Seventeen years later: stateful neural guidance and the tail that sizes a Mars aerocapture mission

Grégory Gelly

gregory.gelly@gmail.com

Preprint, 2026

Abstract

In 2009 we showed that a single-hidden-layer feed-forward network, trained by a genetic algorithm, could fly the aerocapture of a Mars Sample Return vehicle more efficiently than a Cerimele–Gamble feedback law, and we closed that work by asking for the obvious next step: a comparison against predictor–corrector guidance. This paper answers it. We train stateful neural guidance policies and benchmark them, on identical Monte Carlo scenarios drawn from a bit-validated simulator, against six classical schemes including a numerical predictor–corrector (FNPAG) and a reference-tracking feedback law (FTC). Because the mission's correction propellant is sized off the worst-case Δv, we lead every comparison with the tail of its distribution, not the mean. A 962-parameter recurrent (Mamba) policy captures every one of 10^6 pre-registered confirmatory scenarios (a 95% upper bound of 3×10^{-6} on its failure probability) and reaches a far-tail $\mathrm{CVaR}_{99.9}$ of 123.3 ± 0.1 m/s (independent retraining seeds span 122–131). It beats the best classical scheme (FTC with a co-optimized reference) by 16.4 m/s in mean and 27.6 m/s at CVaR_{95}, better on every one of 1000 paired scenarios, at 3.1 ms per simulation – 28 $\times$ faster than FNPAG. The result rests on a training methodology that is itself a contribution: a non-stationary, adaptive-seed Monte Carlo environment turns the genetic algorithm from the **worst** optimizer under fixed scenarios (154 m/s three-seed mean) into the **best** (120). Across cell types, engineered, cost-aligned inputs flatten the median; ablation controls – state reset, matched history, input removal – show it is genuine internal state that compresses the extreme tail that sizes the tanks. The main deployment caveat: under a deliberately harsher off-nominal regime the analytic law generalizes better than the medium-trained network – a gap we trace to the training objective, not to neural guidance itself. A second caveat we surface ourselves: the historical evaluation pipeline conditioned every scenario on a single sample path of the time-varying density noise, and the networks exploit that conditioning 2–4 $\times$ more than the classical schemes. Retraining under per-scenario noise restores 100% capture and full constraint feasibility, and a fresh 10^6-scenario far-tail confirmatory under honest noise restores the architecture story where it matters: the fine-tuned recurrent policy holds $\mathrm{CVaR}_{99.9} = 163.2 \pm 1.3$ m/s (three fine-tune seeds, capturing 99.996% of 10^6 scenarios), while the best classical scheme and the best dense fine-tune, which captures all 10^6, both sit near 237 m/s (Appendix E).

1 Introduction

Aerocapture replaces a propulsive orbit-insertion burn with a single pass through a planet's atmosphere: the vehicle enters on a hyperbolic trajectory and bleeds its excess energy through aerodynamic drag, modulating only the bank angle to steer the lift vector and arrive at a bound target orbit. The maneuver is attractive precisely because it is propellant-free, but it is also unforgiving. The control authority is proportional to dynamic pressure, the entry corridor is narrow, and the dispersions – atmospheric density first among them – are large. There is no closed-form optimal bank-angle law, so the classical practice is to make strong simplifying assumptions (a decoupling of in-plane and out-of-plane motion, a reference trajectory built on apoapsis control alone) in order to obtain a tractable feedback or predictor–corrector law. These simplifications are exactly what costs correction propellant downstream.

Neural networks are especially valuable for guidance problems that admit no analytic solution, and aerocapture is a textbook case. In 2009 we showed that a feed-forward network with a single hidden layer, trained by a genetic algorithm without any reference trajectory or input–output pairs, could perform the aerocapture of a Mars Sample Return (MSR) vehicle at a mean correction cost of 116.7 m/s against 144.8 m/s for a Cerimele–Gamble-derived feedback law – a 19% reduction, approaching the 113 m/s periapsis-raise floor (Gelly and Vernis, 2009), itself building on earlier aerocapture-corridor and soft-landing guidance studies (Vernis *et al.*, 2004; Gelly and Ferreira, 2007). The mechanism was that the network learned the in-plane and out-of-plane logic **jointly**, where the classical law treats them separately. We closed that paper with an explicit hook: "the next step would be to extend our work on the aerocapture to skip entry missions and evaluate the performance of neural guidance compared to classic algorithms such as the predictor-corrector schemes." This paper is that next step, seventeen years later.

In the interval we carried the same evolutionary-training philosophy to recurrent networks for speech, where a coordinated-gate LSTM trained by quantum-behaved particle-swarm optimization (Sun, Feng and Xu, 2004), with a divide-and-conquer initialization and task-aligned differentiable losses, became the working method (Gelly and Gauvain, 2015; Gelly and Gauvain, 2017; 2016). Stateful cells, swarm training, smart initialization, and custom losses are precisely the machinery this paper brings back to where it started. With their innate ability to exploit the trajectory history, recurrent and selective state-space policies are natural candidates for a guidance law that must anticipate the atmospheric pass rather than merely react to the current state.

We make three contributions:

1. **A training methodology, not just a policy.** We show that a genetic algorithm is the **worst** optimizer for this problem under a fixed set of Monte Carlo scenarios – it overfits the repeated cases – and the **best** under a non-stationary, adaptive-seed environment that keeps moving the scenarios beneath it. Switching the seed schedule from fixed to moving takes the mean correction cost from 154 to 120 m/s in three-seed means (first seeds $160.3 \rightarrow 118.0$), the single largest effect in the campaign. The moving environment, a tail-weighting cost transform, and hardest-case seed curation form one matched system.
2. **An architecture finding with a mechanism.** Across dense, gated-recurrent (GRU, LSTM), windowed, attention (Transformer), and selective state-space (Mamba) cells, the engineered, cost-aligned inputs flatten the **median** correction cost to a common 108–112 m/s for every architecture we trained to convergence. The separation appears only on the **tail**: a 962-parameter Mamba policy beats the best dense network at the far-tail depth where the propellant tanks are sized – by 15.0 m/s in three-seed-mean $\mathrm{CVaR}_{99.9}$ on the frozen confirmatory pool, with the deployed artifact below every dense retraining seed. Training loss does not pick the tail winner; internal state does – a mechanism the ablation controls of Section 6.3 measure directly.
3. **A systematic head-to-head of neural versus predictor–corrector aerocapture guidance** – to our knowledge the first for an MSR-class Mars aerocapture to compare an end-to-end learned policy against classical baselines co-tuned on the same objective, on paired dispersed scenarios, under a far-tail correction-Δv risk metric. On identical dispersions, the deployed network beats the best classical scheme by 16.4 m/s in mean and 27.6 m/s at CVaR_{95}, at a per-simulation compute cost $28\times$ below the numerical predictor–corrector – with the caveat that the analytic law is more robust off-nominal, under a deliberately harsh stress regime we return to in Section 7.3.

This answer lands in a field that has moved since 2009. On the classical side, FNPAG (Lu *et al.*, 2015) set the numerical predictor–corrector standard and was carried across Mars aerocapture mission and vehicle design maps (Matz *et al.*, 2017); more recent work replans under uncertainty explicitly – two-stage stochastic and robust formulations that beat a deterministic predictor–corrector under density perturbations (Zucchelli *et al.*, 2021) – solves the full constrained replan by convexification (Rataczak, McMahon and Boyd, 2025), or augments the bank channel with angle-of-attack modulation through optimal control (Sonandres, Palazzo and How, 2025a). Machine learning has so far entered the loop mostly as a **component** inside a classical scheme: LSTM density estimation feeding FNPAG (Sonandres, Palazzo and How, 2025b), generative failure-mode indicators steering a predictor–corrector (Calkins *et al.*, 2025). End-to-end learned aerocapture guidance evaluated against classical baselines on common dispersed scenarios – with the classical side given the same objective and tuning freedom, and the comparison read at the far-tail depth that sizes the propellant – is, to our knowledge, the gap this paper fills.

All of this rests on a high-fidelity simulator – $J_2/J_3/J_4$ gravity, Gauss–Markov density perturbations, thermal limits, pilot dynamics – regression-validated against a legacy reference implementation across all 725 time steps of a guided trajectory, 22 of 24 output channels bit-identical (the two mismatches trace to uninitialized variables in the reference; Appendix A lists the independent physics checks). The simulator also carries EKF navigation, an altitude-dependent wind model, and adaptive integration; this campaign does not exercise them, so every run here uses the bias-filter navigation and fixed-step integration the validation covers. The next section formalizes the aerocapture problem and the sizing-tail objective; Section 3 describes the guidance schemes; Section 4 presents the training methodology; Sections 5–7 give the optimizer, architecture, and classical-versus-neural results; and Section 8 reports what the deployed network actually uses.

2 Problem and objective

2.1 Dynamics and the aerocapture corridor

We study the same robotic MSR mission concept as the 2009 work. The vehicle reaches the atmospheric entry interface at an altitude of 130 km with a relative velocity of 5687 m/s, a flight-path angle of $-10.81°$, and an azimuth of 38.04° (the 2009 study placed the interface at 120 km with a $-10.24°$ flight-path angle; vehicle and target are unchanged); it carries a mass of 1089 kg on a 14.7 m² reference area and is statically trimmed at a fixed angle of attack, so the only control is the bank angle μ, slewed at up to 15°/s. The guidance targets a 500×11 km orbit at atmosphere exit (apoapsis $\times$ periapsis altitude), at 50° inclination, subsequently corrected to a 500 km circular parking orbit. Energy is dissipated by drag during a single atmospheric pass; the bank angle rotates the lift vector to manage how much energy is shed and in which plane.

The natural state space for the maneuver is the (orbital energy, dynamic pressure) plane (Figure 1). The vehicle enters with positive orbital energy (hyperbolic, $E \approx +4.9$ MJ/kg) and must exit with negative energy (bound, $E \approx -5.9$ MJ/kg) without either skipping back out – a hyperbolic escape – or descending so deep that it crashes or violates a thermal limit. The set of trajectories that capture is bounded by two constant-bank profiles: an overshoot boundary (full lift-up, the limit against escape) and an undershoot boundary (full lift-down, the limit against crash). A practical **restricted corridor** tightens these to the bank profiles that land within a bounded apoapsis error of the target. Control authority is proportional to dynamic pressure, so the guidance has the most leverage deep in the pass and almost none on the thin entry and exit legs. The vehicle must respect a peak heat-flux limit of 200 kW/m², a 4 g load limit, and an integrated heat-load limit of 25 MJ/m².

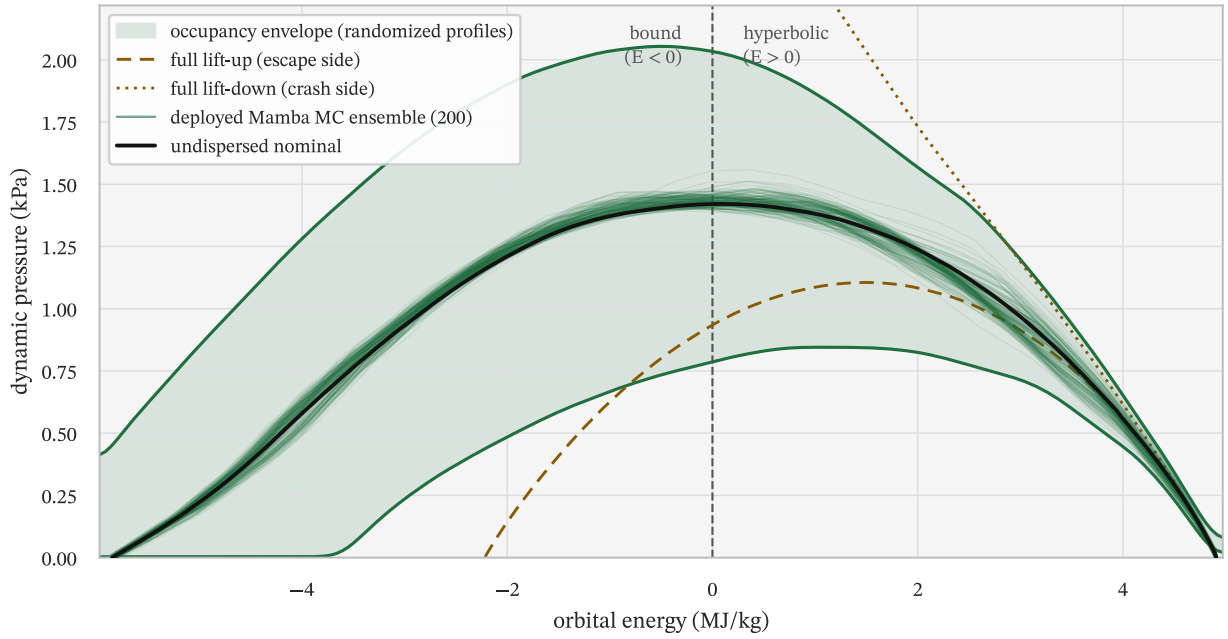


Figure 1: Empirical trajectory-occupancy envelope of the aerocapture corridor in the (orbital energy, dynamic pressure) plane, traced by a $1,000,000$-run dispersed Monte Carlo of randomized signed piecewise-constant bank profiles (roll reversals included), with the undispersed full-lift-up and full-lift-down constant-bank boundary traces overlaid (dashed). The shaded band spans the occupied corridor: the upper edge is the $p_{99.9}$ dynamic pressure of all capturing trajectories (the crash-side limit), the lower edge the $p_{0.5}$ of trajectories capturing below a 5000 km apoapsis (the escape-side limit); as quantiles of sampled profiles these edges are empirical, not a formal reachable set. The vehicle enters hyperbolic ($E > 0$, right) and bleeds energy into a bound orbit ($E < 0$, left); the 200-run Monte Carlo ensemble of the deployed Mamba policy and its undispersed nominal (heavy line) fly well inside the envelope.

2.2 The objective: the tail that sizes the mission

Performance is measured by the **correction Δv** – the propellant cost to carry the captured orbit to the 500 km circular parking orbit, summed over the periapsis-raising, circularization, and plane-change burns. At atmosphere exit the periapsis always sits inside the atmosphere and must be raised, so the correction cost has a floor of roughly 105 m/s at this entry interface, set by the nominal periapsis raise (dispersed cases spread a few m/s around it; the 2009 study's interface put the same floor near 113 m/s); a guidance law cannot do better than deliver the vehicle to that floor across all dispersions. We define a run as a **capture** when it terminates in a bound orbit (the simulator's terminal flag $i_{\text{final}} = 3$ with eccentricity < 1) and compute Δv over captured runs only.

The point we want to make sharply, because it governs every comparison in this paper, is that the **mean** correction cost is operationally almost irrelevant. Aerocapture propellant tanks are sized for the worst credible case, conventionally the 3σ design point ($\approx$ the 99.87th percentile), not the median. Two guidance laws with the same mean but different tails are not equally good: the one with the heavier tail forces larger tanks and a heavier, more expensive mission. We therefore treat the **tail** of the Δv distribution as the objective, and report the conditional value-at-risk – the mean cost in the worst $(1-\alpha)$ fraction of cases, CVaR_α (Rockafellar and Uryasev, 2000) – at $\alpha = 95\%$ and, for sizing decisions, at the far-tail depth $\mathrm{CVaR}_{99.9}$, together with the 95th and 99th percentiles and the sample maximum (a descriptive bound, $\approx p_{99.99}$ at $n = 10,000$). Empirically CVaR_α is the mean of the worst $\max(1, \mathrm{round}((1-\alpha)n))$ captured observations, without interpolation; every tail statistic is reported with the number of observations it averages. The far tail cannot be estimated from a 1000-case ensemble (a single sample

beyond $p_{99.9}$), so every sizing number in this paper is computed on a dedicated $n = 10,000$ pool, training-disjoint by construction. We lead with the tail; the mean is reported for continuity with the 2009 work.

Table 1: The dispersion model. Twenty-six static draws across nine domains, plus a time-varying Gauss–Markov density perturbation (the tenth row). The controlled-study regime uses medium presets except for the winds and density perturbation (low); the wind model itself is disabled in this campaign, so its two draws are inert. Gaussian dispersions are quoted at 3σ; uniform dispersions at their full range.

Domain	Dims	Distribution	Dispersion (controlled regime)
Entry state	6	Gaussian	altitude ± 0.3 km, velocity ± 3 m/s, flight-path/azimuth ± 0.15–$0.3°$ (3σ; medium)
Atmospheric density	1	Uniform	$\pm 50\%$ multiplicative bias (medium)
Aerodynamics	3	Uniform	drag $\pm 5\%$, lift $\pm 10\%$, angle of attack $\pm 1°$ (medium)
Navigation errors	7	Gaussian	altitude ± 2 km, horizontal $\approx \pm 9$ km per axis, velocity ± 1.2 m/s, drag-accel ± 0.3 m/s² (3σ; medium)
Mass	1	Uniform	$\pm 1\%$ (medium)
Vehicle	2	Uniform	reference area $\pm 2\%$, max bank rate $\pm 10\%$ (medium)
Pilot dynamics	3	Uniform	time constant / damping / frequency $\pm 10\%$ (medium)
Nav-filter gain	1	Gaussian	± 0.3 absolute (3σ; medium)
Winds	2	Uniform	speed $\times [0.7, 1.3]$, direction $\pm 5°$ (low; model disabled – draws inert)
Density perturbation	process	Gauss–Markov	correlation time 120 s, 5% RMS, time-varying (low)

The mission is flown under 26 dispersed parameters across nine domains, summarized in Table 1, plus a time-varying Gauss–Markov (Ornstein–Uhlenbeck) density perturbation that evolves during each run. The controlled-study regime uses medium presets for the initial-state, atmospheric, aerodynamic, navigation, mass, vehicle, pilot, and navigation-filter domains and low presets for the winds and the density perturbation. The atmospheric density bias alone spans $\pm 50\%$ – it is the dominant driver of apoapsis error, and a guidance law blind to it cannot reject it. The simulator's wind model follows a parametric Mars profile (Forget *et al.*, 1999) but is disabled for this campaign (its two draw dimensions are carried but inert); the density perturbation is an Ornstein–Uhlenbeck process layered on the static bias. Within a multi-scenario batch, draws are generated by Latin-hypercube sampling (McKay, Beckman and Conover, 1979) for space-filling coverage; the evaluation pools draw one scenario per seed and are therefore plain independent samples – which is what the replicate- and bootstrap-based intervals of Sections 6–7 assume.

Note that one dispersion dimension is narrower than the table suggests, and we surface it here rather than in a footnote. In every one-scenario-per-seed batch – which is how all training, validation, and evaluation pools of this paper are executed – the Ornstein–Uhlenbeck process draws its innovations from a stream seeded independently of the scenario seed, so all scenarios in a pool share a **single** sample path of the density perturbation (the same holds for the then-unused EKF sensor streams). The perturbation is time-varying within each run, but it is not a cross-scenario random variable: every number in the main body conditions on one realization of it. The comparison across schemes remains fair – every scheme trains and evaluates under the identical conditioning – but the absolute Δv statistics are tighter than the marginal ones, and the neural policies, trained on the shared path, exploit it more than the analytic laws do. Appendix E quantifies the gap (it is a tail effect: 2–4 $\times$ larger for the networks than for the classical schemes), introduces the per-scenario seeding mode that repairs it, and re-quotes the headline cells retrained under it.

3 Guidance schemes

We benchmark the neural policies against six classical schemes that span the spectrum of aerocapture guidance, from analytic feedback through numerical prediction. All schemes share the same simulator, the same navigation chain, and – where they need one – the same kind of reference trajectory, so the comparison isolates the guidance law itself (Table 2).

3.1 Classical schemes

Feedback Trajectory Control (FTC) is the Cerimele–Gamble-derived scheme from the 2009 study (Cerimele and Gamble, 1985; Gelly and Vernis, 2009) and our continuity baseline. It builds a virtual reference trajectory on apoapsis control alone – a constant-bank profile that reaches the target apoapsis without inclination control, tabulated as $\cos\mu_{\text{ref}}$, $\dot{h}_{\text{ref}}$, and q_{ref} versus orbital energy – and enslaves the commanded bank to it through a proportional law on the altitude-rate and dynamic-pressure errors,

$$\cos\mu_{\text{com}} = \cos\mu_{\text{ref}} + G_{\dot{h}}\frac{\dot{h}-\dot{h}_{\text{ref}}}{q} + G_q\frac{q-q_{\text{ref}}}{q}, \tag{1}$$

where q is the dynamic pressure and $G_{\dot{h}}$, G_q are the altitude-rate and dynamic-pressure feedback gains; the command is clamped to 0° or 180° when $|\cos\mu_{\text{com}}| > 1$. It decouples in-plane (apoapsis) from out-of-plane (inclination) motion, the latter handled by a roll-reversal logic that flips the bank sign whenever the inclination error, projected a few seconds ahead, exceeds a threshold inside an energy-gated activation window. FTC is analytic, fast, and – as we will show – only as good as its reference.

FNPAG is Lu's fully numerical predictor–corrector (Lu, 2014; Lu *et al.*, 2015), subsequently carried across Mars aerocapture mission and vehicle design maps (Matz *et al.*, 2017). Every two-second replan cycle it integrates the equations of motion forward to atmosphere exit and bisects the constant capture-bank magnitude until the predicted osculating exit apoapsis matches the target, scaling the onboard atmosphere by the navigation-estimated density factor so the predictor tracks the measured atmosphere rather than a nominal model; the command is held between replans. It is the most accurate classical scheme and, at roughly eleven forward integrations per replan, by far the most expensive.

PredGuid is the Apollo/Shuttle-heritage drag-tracking law (Harpold and Graves, 1979; Bairstow, 2006): it tracks a drag-versus-energy reference profile with negative feedback.

Energy controller tracks an energy-dissipation reference through dynamic-pressure and altitude-rate feedback.

Equilibrium glide holds the equilibrium-glide condition with altitude-rate damping and a velocity bias, using the navigation-filtered density rather than a static table.

Piecewise constant flies an N-segment constant-bank profile; it is the simplest scheme, produces the reference trajectory and corridor the other schemes consume, and – like the full-neural network – emits a **signed** bank, so it bypasses the shared roll-reversal, exit, and thermal-limiter logic.

3.2 Neural guidance

The neural policy maps an observation vector to a bank command. We generalize the 2009 single-hidden-layer feed-forward network – five hand-picked inputs (orbital energy, eccentricity, inclination, velocity, non-gravitational acceleration) feeding a two-output bank decoder – along three independent axes: the **inputs** the policy reads, the **bank decoder** it writes through, and the **cell type** that carries state across the pass. We take them in turn.

3.2.1 Inputs

The modern policy draws from a 35-element candidate vector; a learned input mask selects the subset that actually reaches the network (the deployed atan2 policies use 17). Sixteen entries are instantaneous orbital, aerodynamic, and thermal state variables. The other nineteen are **engineered** signals that hand the policy temporal context and a cost-aligned summary of the flown trajectory, so it need not integrate the history itself. A few are genuinely autoregressive – seam-free $(\sin, \cos)$ encodings of the recent bank history and the roll-reversal telemetry – but

most are instantaneous functions of the current osculating orbit: reference-trajectory interpolations ($\dot{h}$ and q at the current energy), a closed-form exit-bank teacher signal, the periapsis altitude, and – most important, as the ablation of Section 8 shows – the three **predicted correction-Δv** components (the energy-closing, periapsis-correction, and plane-change burns). Those last three are smooth, causal, and cost-aligned: they tell the network what the maneuver would cost if it stopped now.

3.2.2 Bank decoder

The 2009 paper read the bank from a two-element output,

$$\mu = \operatorname{atan2}(o_1, o_2), \tag{2}$$

which we keep as the default (`atan2_signed`). It wastes half its range when only the magnitude is needed, so for magnitude-only policies the `acos_tanh` decoder maps a single output smoothly onto $[0, \pi]$,

$$\mu = \arccos(\tanh(o_1)). \tag{3}$$

Two further single-output decoders attack the $\pm\pi$ wrap seam a raw angle output suffers near $\mu = \pi$. The `scaled_pi` decoder pushes the seam out of the operating region,

$$\mu = \operatorname{wrap}_{\pi(n\pi\tanh(o_1))}, \tag{4}$$

while the `delta` decoder applies a bounded increment on the previous realized bank,

$$\mu = \operatorname{wrap}_{\pi(\mu_{\text{prev}}+\Delta_{\max}\tanh(o_1))}. \tag{5}$$

3.2.3 Cell type

Where 2009 had a single hidden layer, we span six architecture families behind one common runtime, each carrying a different kind of internal state across the atmospheric pass:

- **Dense** feed-forward – memoryless; maps the current observation straight to a command. The 2009-style baseline.
- **GRU** (Cho *et al.*, 2014) – a gated recurrent cell carrying one hidden-state vector, updated each tick through reset and update gates.
- **LSTM** (Hochreiter and Schmidhuber, 1997) – a recurrent cell that adds a separate long-term cell state alongside the hidden state, regulated by input, forget, and output gates.
- **Windowed** – a zero-parameter FIFO buffer of the last N observations flattened into a dense stack: explicit recent history rather than a learned state.
- **Transformer** (Vaswani *et al.*, 2017) – a causal-attention block attending over a fixed-length key/value window of recent steps.
- **Mamba** (Gu and Dao, 2023) – a selective state-space core: a linear recurrence whose gates depend on the input, carrying a compact SSM state.

We use **recurrent** loosely for any cell that carries internal state across the pass. All six train and deploy through the same bit-validated Rust runtime, and all are sized so the comparison across cell types holds the parameter budget roughly fixed. Appendix B probes three further recent recurrent families – closed-form continuous-time cells (Hasani *et al.*, 2022), the exponential-gated and matrix-memory xLSTM variants (Beck *et al.*, 2024), and the discretization and complex-state axes of Mamba-3 (Lahoti *et al.*, 2026) – against these cells at matched budget; none improves on them.

Table 2: The benchmarked schemes. "Reference" marks dependence on a tabulated reference trajectory ("inputs": the network reads two reference interpolations as observations but does not enslave to them); "Compute" classes are quantified in Section 7.2 (fast: 1–3 ms/sim; slow: 87 ms/sim). Signed-bank schemes (full-neural, piecewise-constant) bypass the shared roll-reversal, exit-phase, and thermal-limiter logic.

Scheme	Bank command	Reference	Compute	Heritage / note
FTC	magnitude + roll reversal	yes	fast	Cerimele–Gamble apoapsis enslavement
FNPAG	magnitude + roll reversal	no	slow	onboard forward integration, bisection corrector
PredGuid	magnitude + roll reversal	yes	fast	Apollo/Shuttle drag tracking
Energy controller	magnitude + roll reversal	yes	fast	energy-dissipation tracking
Equilibrium glide	magnitude + roll reversal	no	fast	equilibrium-glide condition, nav density
Piecewise constant	signed (N segments)	no	fast	produces reference + corridor
Neural network	signed or magnitude	inputs	fast	35-input candidate vector, stateful cells

4 Training methodology

The training methodology is the load-bearing contribution. The policies are trained without input–output pairs or any reference-tracking objective (two observation inputs interpolate the reference table; nothing enslaves to it): each candidate network is simulated on a batch of dispersed Monte Carlo scenarios, and its fitness is the resulting correction-Δv cost (with soft constraint penalties), so the optimizer searches directly on mission performance. The same recipe trained the 2009 networks. What changed – and what makes the modern policies work – is the realization that **how the scenario batch is chosen, generation to generation, matters more than the optimizer or the architecture**. The right environment is non-stationary: it keeps moving the scenarios beneath the population so that no individual can memorize a fixed set. Four design choices – a genetic algorithm, an adaptive (moving) seed schedule, a tail-weighting cost transform, and a hardest-case seed curation – form one matched system. We take them in turn.

4.1 Seed strategy: the genetic algorithm needs a non-stationary objective

The single largest effect in the entire campaign is the seed strategy. We compared three: **fixed** (a deterministic scenario batch, identical every generation), **rotating** (a fresh random batch every generation), and **adaptive** (a curated batch, refreshed on a validated improvement or periodically, drawn from the cost distribution of the best individuals). The decisive cells carry three independent training seeds each (this revision's repeats). Under fixed seeds the genetic algorithm is the **worst** optimizer we tested and an unstable one – three trainings land at 160.3, 166.9, and 133.5 m/s mean (CVaR_{95} 215.8 on the first seed) – because it overfits the repeated scenarios, evolving a policy that is excellent on those particular draws and mediocre elsewhere. Rotating the seeds rescues it outright and **reliably**: 119.4 m/s three-seed mean with a 3.5 m/s seed range (first seed 120.0 mean, 144.5 CVaR_{95}) – a 34 m/s drop in three-seed means, a 71 m/s drop on the tail on the first seeds, and the two distributions do not touch: the fixed schedule's luckiest seed sits 12.5 m/s above the moving schedule's worst. Adaptive curation is indistinguishable from rotating on the **mean** at this allocation (119.6 versus 119.4, seed ranges overlapping; the first-seed 118.0 was a favorable draw); its contribution is the hardest-seed tail shaping of Section 4.2, which a plain rotating schedule cannot express.

The control that makes this a statement about the **objective** and not about compute is CMA-ES. Run on the identical fixed-versus-rotating change, CMA-ES does not move – 125.7 versus 126.1 m/s in three-seed means, seed ranges within ± 1 (first seeds 126.9 $\rightarrow$ 127.3) (Figure 2). Under fixed seeds the genetic algorithm's selection converges onto

the quirks of those particular draws; CMA-ES neither suffers under the fixed batch nor benefits from the moving one. We state that asymmetry empirically rather than mechanistically: a plausible reading is that CMA-ES's continuous re-estimation of its parameter-space search distribution already decorrelates successive generations, while scenario noise chiefly perturbs its rank-based updates and step-size control – consistent with its self-termination on the noisy objective (Section 5) – but our experiments were not designed to isolate the mechanism. Rotating costs marginally more compute, but CMA-ES given the same marginal compute gains nothing, so the lever is the non-stationarity of the objective, not the extra evaluations. The practical consequence is striking: under a fixed objective one would deploy CMA-ES and discard the genetic algorithm; under a moving objective the genetic algorithm becomes the best optimizer in the study. The moving environment does not make the genetic algorithm **robust** to a moving objective – it makes the genetic algorithm **need** one.

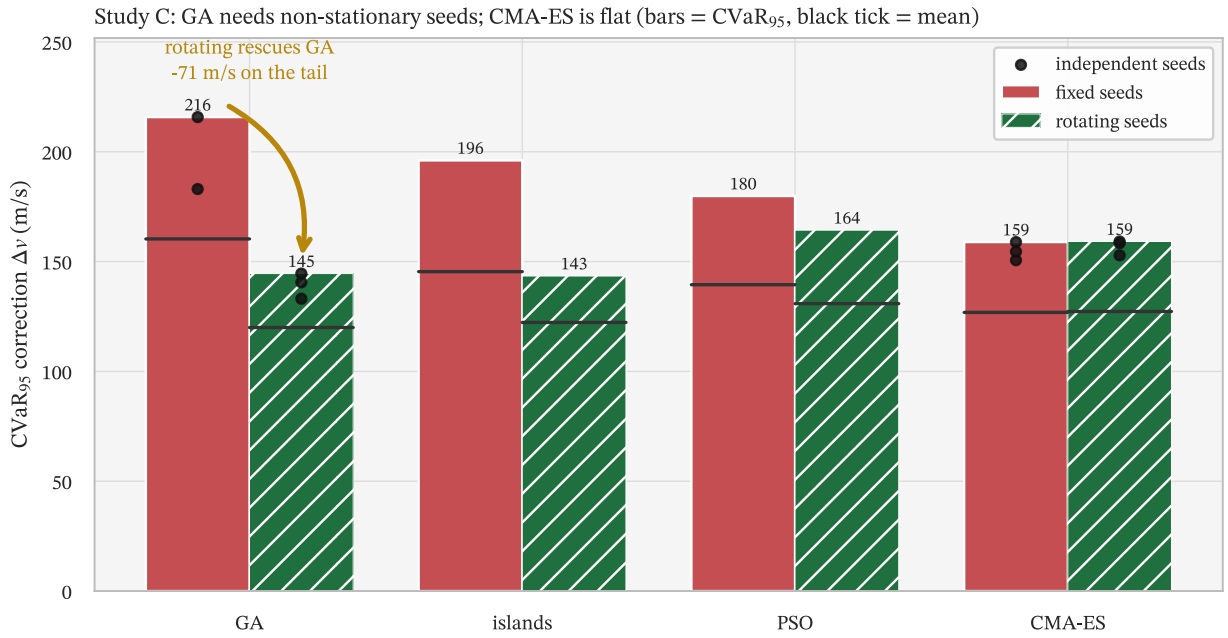


Figure 2: Fixed versus rotating seeds, per optimizer. The genetic algorithm is the worst optimizer under fixed seeds and is rescued by rotating them (−34 m/s in three-seed means, −71 m/s at CVaR_{95} on the first seeds); CMA-ES is essentially unchanged (candidate mechanisms are discussed in the text). Black dots: three independent training seeds for the repeated GA and CMA-ES cells – the fixed-seed pathology is itself high-variance. The lever is the non-stationary objective, not the extra compute.

4.2 Cost transform, curation, and allocation

The same worst-case-leaning logic that makes us report the tail also shapes how we train. Because the mission is sized off the far tail, we apply a monotonic **cost transform** to each per-simulation cost before aggregating, so that the optimizer feels expensive scenarios more sharply. Applied per scenario before the root-mean-square aggregation over the individual's batch (Appendix A), the cubed transform makes the per-individual objective

$$J = \left(\frac{1}{n} \sum_{i=1}^{n} C_i^6 \right)^{1/2}, \tag{6}$$

a monotone function of the L_6 norm of the per-scenario cost vector – a high-moment objective that weights an individual's worst scenarios far more heavily than its typical ones. (For a single scenario a monotone transform is ranking-neutral; across a batch it deliberately is not.) We prefer this smooth high-moment proxy over optimizing an empirical tail quantile directly because the deployed allocation evaluates as few as two scenarios per individual per

generation – far too few to estimate a quantile – while the moving batch supplies tail coverage across generations. Evaluated at the depth that matters – a far-tail $n = 10,000$ pool – the cubed transform compresses the extreme tail best ($\mathrm{CVaR}_{99.9}$ 153.0 m/s, sample max 160.1) against linear (156.7/162.2), square-root (158.4/167.1), squared (162.7/180.9), and logarithmic (162.3/180.6). The logarithm is worst across the shallow and mid tail because it over-compresses and starves the gradient between captures; only at the very extreme is the squared transform worse still. These are single training runs, and the cubed-versus-linear margin (3–5 m/s) is within the run-to-run scatter we measure in Section 6, so we read the direction – deeper tail-weighting pays at deeper sizing depth – rather than the exact ranking as the finding. A shallower metric (CVaR_{95}) would have mildly favored square-root; the deeper we look into the tail, the more the tail-weighting pays, which is exactly why the sizing depth must decide (Figure 3).

Seed **curation** is the same mechanism applied to the scenarios rather than the cost. At each refresh the adaptive strategy bins the cost distribution of the best individuals into quantiles and picks one representative seed per bin; the choice of representative is the lever. Picking the **hardest** seed per bin (the "max" bucket) dominates the far tail ($\mathrm{CVaR}_{99.9}$ 153.0 m/s) against the bin median (193.9), a random pick (173.1), and the **easiest** seed (225.9). The easiest-seed bucket is the cautionary tale: the best mean, 117.8 m/s, and a catastrophic worst case – it drops captures and reaches 245 m/s, optimizing the average at the expense of the worst case. Trimming the cost distribution helped nothing (Figure 5). These too are single runs, but the min- and middle-bucket gaps (40–70 m/s) sit far outside run scatter. The cost transform and the curation bucket are therefore one idea – force the policy onto the hard cases – expressed twice.

Finally, **allocation**: given a compute budget, is it better spent on more scenarios per generation or more generations? Under rotating seeds with a fixed generation count the sweet spot is moderate ($n_{\mathrm{sims}} = 10$). But under the adaptive strategy with the budget spent on generations, the answer flips hard: just $n_{\mathrm{sims}} = 2$ per generation, run for many generations, gives the best mean (109.9 m/s) and CVaR_{95} (117.5), beating 5, 20, and 100 (Figure 4). The few-sample noise per generation is bought back, and then some, by the far greater scenario diversity the moving environment sees over the run. The deployed headline policy uses this allocation.

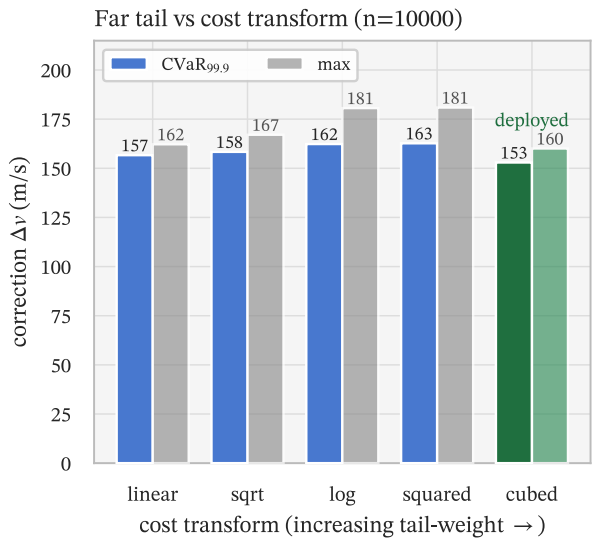


Figure 3: Cost transform versus the sizing tail, at the sizing depth ($n = 10,000$ pool). The cubed transform minimizes both the far-tail $\mathrm{CVaR}_{99.9}$ and the sample maximum, even though a shallow CVaR_{95} read would mildly favor square-root.

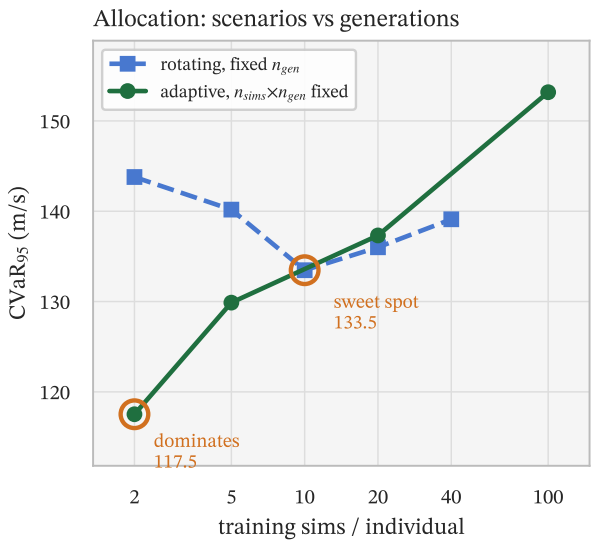


Figure 4: Allocation of the compute budget (CVaR_{95}; the mean orders both series identically). Under the adaptive schedule, two scenarios per generation over many generations dominates larger per-generation batches.

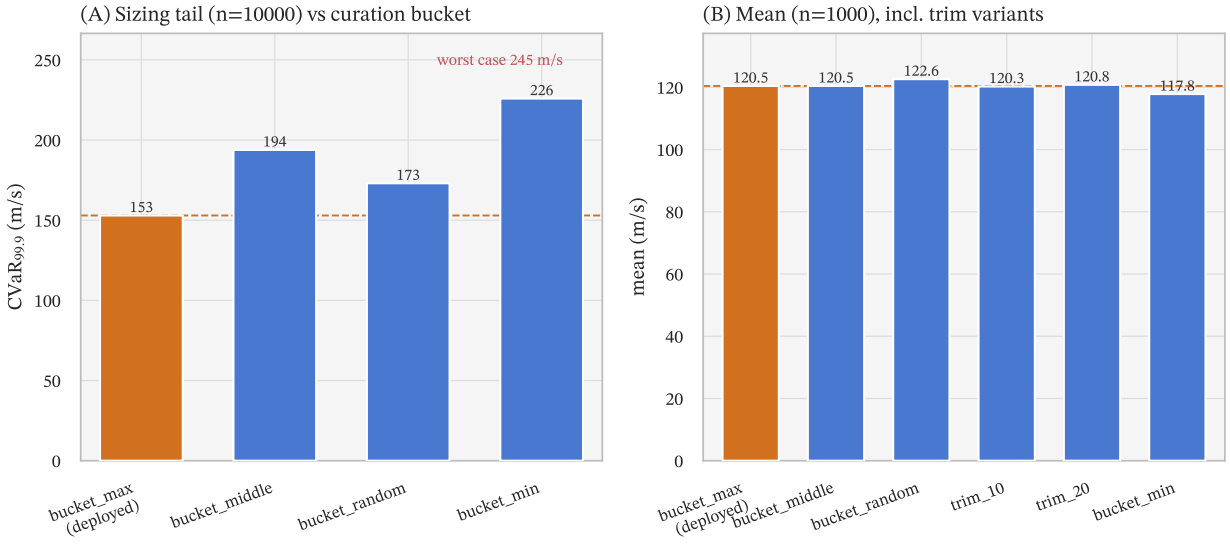


Figure 5: Seed curation. Left: far-tail $\mathrm{CVaR}_{99.9}$ ($n = 10,000$) per curation bucket – selecting the hardest seed per cost-CDF bin (the "max" bucket) compresses the sizing tail, and the easiest-seed bucket blows it up (worst case 245 m/s). Right: mean ($n = 1000$) – the easiest bucket wins the mean, the optimize-the-average trap; the trim variants match the max bucket and were not carried to the far-tail pool.

That the optimal allocation pours the budget into generations points to the last methodological fact: training here is **compute-bound, not overfitting-bound**. A stationary objective eventually overfits, and one stops early. The moving objective never converges to a fixed landscape, so the validation error – the RMS on the reserved selection pool of Table 3; we keep the conventional name for the loss curve – keeps falling for nearly the whole twenty-thousand-generation run and then plateaus rather than degrading, i.e. the policy does not memorize scenarios (Figure 6). The plateau also exposes a counter-intuitive dimensionality effect that recurs in the architecture results: the 972-parameter dense network learns **faster** early – more plasticity – but the 515-parameter network overtakes it and plateaus **lower** (validation RMS 1.326×10^6 versus 1.433×10^6). For the gradient-free genetic algorithm, the extra dense parameters are more search burden than added capacity; the "more parameters, learn faster" intuition does not transfer.

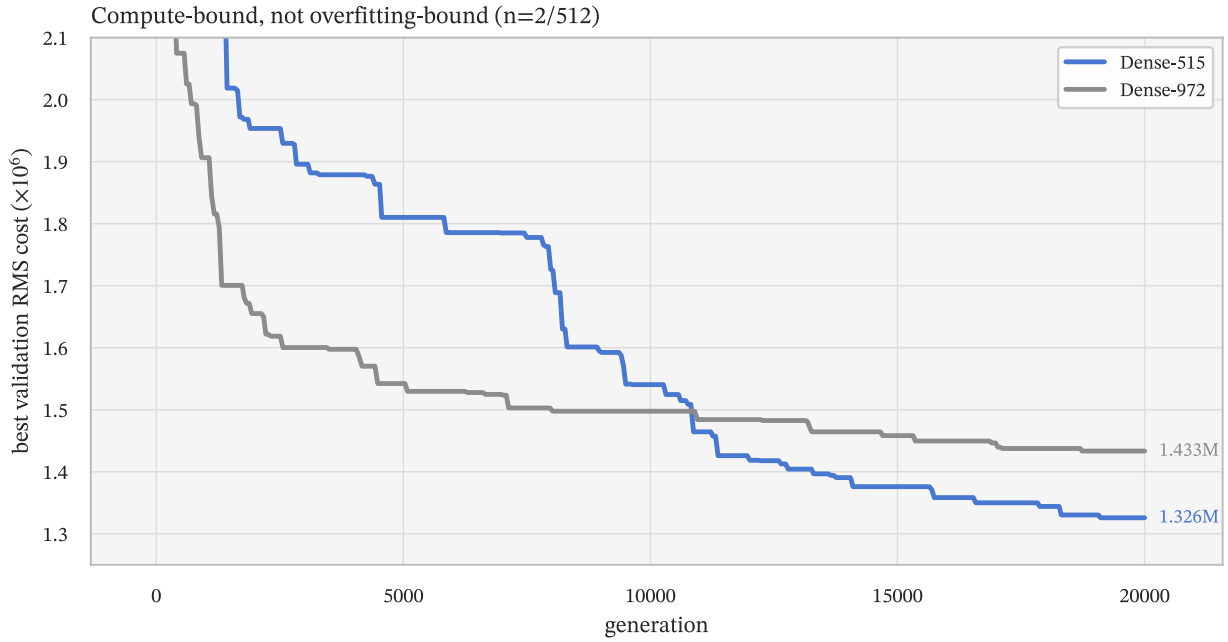


Figure 6: Best validation RMS versus generation for the two dense reference networks. Both keep improving until late in the twenty-thousand-generation run and then plateau – under the non-stationary objective the policy generalizes across the dispersion distribution rather than memorizing scenarios. The 515-parameter network plateaus below the 972-parameter one: beyond a few hundred parameters, extra dense capacity hurts the gradient-free search.

Table 3: Scenario-pool roles and the decisions each pool influenced. The pools above the last three rows are development quantities: the selection pool is adaptively reused by the promotion gate, and the development far-tail pool informed the methodology and architecture choices, so neither is an unbiased test set. The confirmatory pool was generated from a seed range disjoint from every earlier draw, after all methodology, architecture, and checkpoint choices were frozen, and each cell was evaluated on it exactly once.

Pool	**n**	**Queries**	**Decisions taken on it**
Training batches	2/gen	every generation	weight updates (moving, curated)
Selection pool (offset 1M)	1000	13, 442 over the run	in-training argmin promotion
Development far tail (offset 2M)	10, 000	tens	cost transform, curation bucket, allocation, cell type, headline choice
Fresh re-quote (offset 8M)	1000	once; quantization grid	quantization cell choice (Appendix C)
Confirmatory sizing (Appendix A)	$10 \times 100,000$	once, post-freeze	none – every quoted sizing number
Off-nominal stress (offset 9M)	1000	once per policy	none (robustness probe)
Architecture probes (offset 10M)	1000	once per arm-repeat	none (Appendix B verdicts)

The studies of Sections 4–5 are exploratory – single training runs unless stated otherwise; the confirmatory statements of this paper are the frozen-pool quantities of Sections 6–7.

5 Optimizer and dimensionality

Every optimizer in this study is a gradient-free population method, a deliberate choice rather than an omission. The training objective is the correction Δv of a complete atmospheric pass – produced by a simulator with discrete capture, crash, and atmosphere-exit termination, threshold-triggered phase transitions, and hard constraint limits – so it is a black-box function of the network weights with no usable gradient. Policy-gradient reinforcement learning (PPO (Schulman *et al.*, 2017), SAC (Haarnoja *et al.*, 2018)) does not require a differentiable simulator or a differentiable reward – it estimates gradients from sampled rollouts and can in principle optimize a terminal-only objective. We implemented and trained both, with potential-based per-step shaping aligned to the predicted correction cost plus the true terminal cost (the standard remedy for sparse terminal rewards), and the best policies still underperformed the population methods by a wide margin: 636 m/s mean (1047 at CVaR_{95}) for the dense PPO policy and 513 (893) for the recurrent one, against 119 (138) for the population-trained dense network on the same simulator regime[1] – consistent with the stochastic shaped return optimizing a different quantity than the deterministic mission cost. Population search on the mission cost itself was simply the stronger tool here, so throughout we optimize the mission cost directly.

Having established that the genetic algorithm is the right optimizer under a moving objective, two questions remain: does it need a population that scales with the search dimension, and does the optimizer choice even matter for the low-dimensional classical-gain problems? We compared the genetic algorithm (Goldberg, 1989) against CMA-ES (Hansen and Ostermeier, 2001), particle-swarm optimization (Kennedy and Eberhart, 1995) and its quantum-behaved variant (Sun, Feng and Xu, 2004), differential evolution (Storn and Price, 1997), and a three-island heterogeneous model, on an optimizer-by-budget grid at the largest dense network (3998 weights) and an optimizer-by-dimension grid spanning the 26-parameter FTC-gain problem and the 515-parameter dense network.

At 3998 weights the population size is decisive (Figure 7). The genetic algorithm is best at a population of 150 (118.0 m/s mean; three-seed 119.6, range 3.0) and 300 (120.5 m/s) – a gap smaller than the run-to-run scatter we measure on retrained cells in Section 6, so we report them as indistinguishable – but at a population of 60 it **collapses** to 166.3 m/s. Sixty individuals cannot cover a four-thousand-dimensional weight space; selection drifts. So the tempting generalization that the genetic algorithm dominates at any budget is wrong: at a starved population it is no better than a single restart. The corrective is simple and worth stating plainly – the population must scale with the search dimension. CMA-ES improves smoothly with budget (133.3 $\rightarrow$ 126.3 $\rightarrow$ 121.8 m/s) but never reaches the genetic algorithm's optimum and self-terminates on the noisy objective before exhausting a generous generation count, an asymmetry to keep in mind for compute-matched comparisons. The three-island heterogeneous trainer (particle swarm, genetic, and differential evolution, with periodic migration) is the most budget-robust of all, 120–124 m/s across every budget, but the well-populated single genetic algorithm edges it.

The dimensionality grid carries the more useful lesson. On the 26-parameter FTC-gain problem every gradient-free optimizer we ran lands in a tight band, roughly 170–178 m/s – islands 169.8, particle swarm 170.9, CMA-ES 171.7, quantum-behaved swarm 172.9, differential evolution 178.2 – an 8 m/s spread we do not attempt to rank (Section 9). Optimizer choice barely matters when there are only twenty-six gains to tune. It is at neural-network dimensionality that the optimizers separate: at 515 weights the genetic algorithm is best (117.4 m/s), the islands next (118.6), and particle swarm worst (129.8), a 12 m/s spread. The confound is that the 26-parameter cell is a different guidance scheme (FTC), not a narrowed version of the same network, so the comparison conflates dimension with law; but the direction is clear – the optimizer earns its keep on the high-dimensional weight search, not on low-dimensional gain tuning.

[1] The reinforcement-learning cells predate several later simulator fixes and are quoted on their own contemporaneous evaluation pool; the 4–5 $\times$ gap, not the absolute values, is the result.

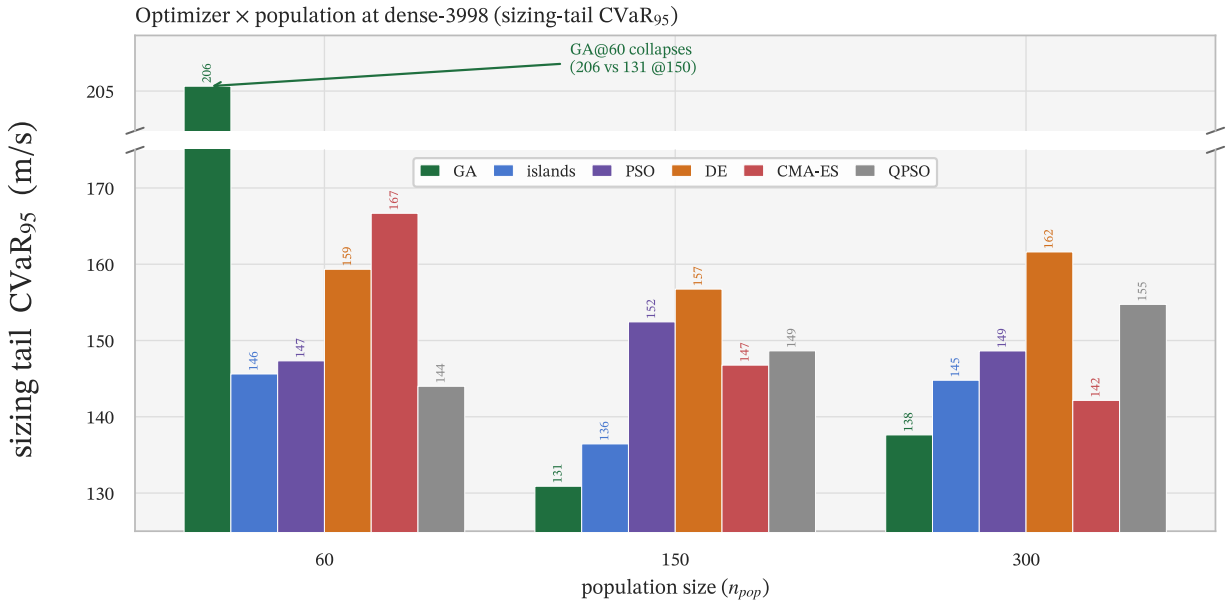


Figure 7: Optimizer by population budget at 3998 weights. The genetic algorithm is best at populations of 150–300 but collapses at 60 (the population must scale with the search dimension); the heterogeneous island model is the most budget-robust; CMA-ES improves with budget but trails the genetic optimum.

The island model – the most budget-robust optimizer above – was built for exactly that robustness (Figure 8). It splits one population budget across three islands running complementary operators (particle swarm, genetic, and differential evolution) on the same scenarios, and every k generations migrates each island's best n individuals into the others, overwriting their worst. Two properties follow. The heterogeneous operators explore the weight space differently and the migration cross-pollinates their discoveries, so a sub-population trapped in a local optimum is pulled out by a migrant from another island instead of having to escape on its own – the diversity a single homogeneous method lacks. And because the three strategies share one run's evaluations through migration, one need not run each optimizer separately and keep the best. The result never collapses the way a starved genetic algorithm does (mean 120–124 m/s across every budget); the price is that at a well-chosen budget a single large genetic algorithm still edges it, so the islands buy robustness to the budget choice rather than a lower optimum.

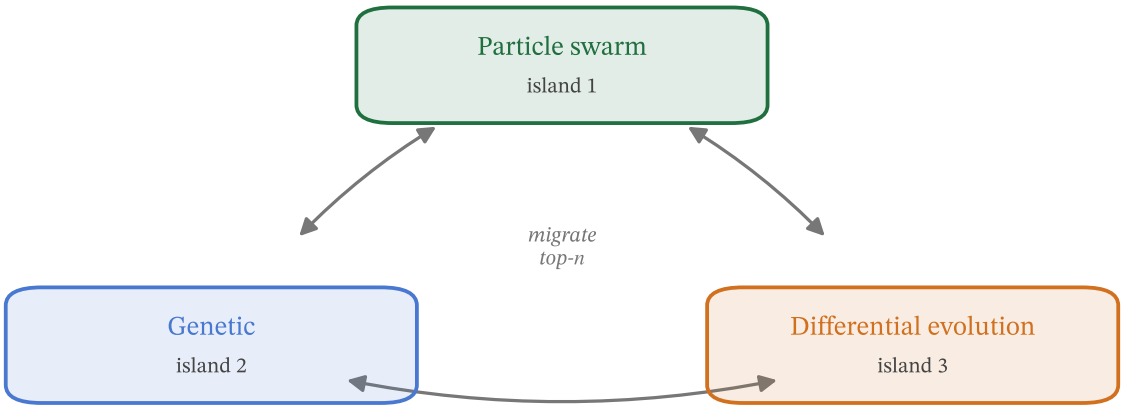


Figure 8: The three-island heterogeneous optimizer. Three islands run complementary gradient-free operators (particle swarm, genetic, differential evolution) on the same scenarios; periodic migration exchanges each island's best individuals for the others' worst. The heterogeneity plus migration is what escapes local optima, and one population budget covers all three searches.

6 Architecture: the headline result

We now sweep the cell type. The finding, which the next two subsections establish, comes in two halves: a flat median across cell types, and a separation that appears only on the **tail**, where a policy with internal state wins.

6.1 Parameter budget and the capability floor

A parameter-budget sweep across six families (dense, GRU, LSTM, windowed, Transformer, Mamba), each trained identically at two scenarios per generation for five thousand generations, makes the first half of the thesis concrete (Figure 9). Every cell in the sweep captures 100% of the time – there is no capability collapse, and a dense network with as few as 102 parameters still guides the maneuver at 100% capture and 120.8 m/s mean. Within the dense family the cost is flat from a few hundred to a few thousand parameters (515 $\rightarrow$ 972 $\rightarrow$ 1957 weights give 117.4 $\rightarrow$ 116.9 $\rightarrow$ 116.8 m/s) and a 3998-weight network gains nothing further – it is over-parameterized for the genetic search, consistent with the plateau result of the training methodology (Section 4). At a matched budget the recurrent and state-space families edge the best dense cell (GRU 112.8, Mamba 114.9, LSTM 116.0 versus the best dense 116.8), while the Transformer pays an attention-overhead penalty at small budgets – its worst cell, at 762 parameters, is the worst in the whole sweep (121.9 m/s), and not for lack of parameters (762 $>$ 515): a small budget forces the attention block to a tiny model width and two dimensions per head, which the genetic algorithm trains poorly. The Pareto front is therefore shallow on the mean; the interesting structure is in the tail, which a five-thousand-generation sweep cannot resolve.

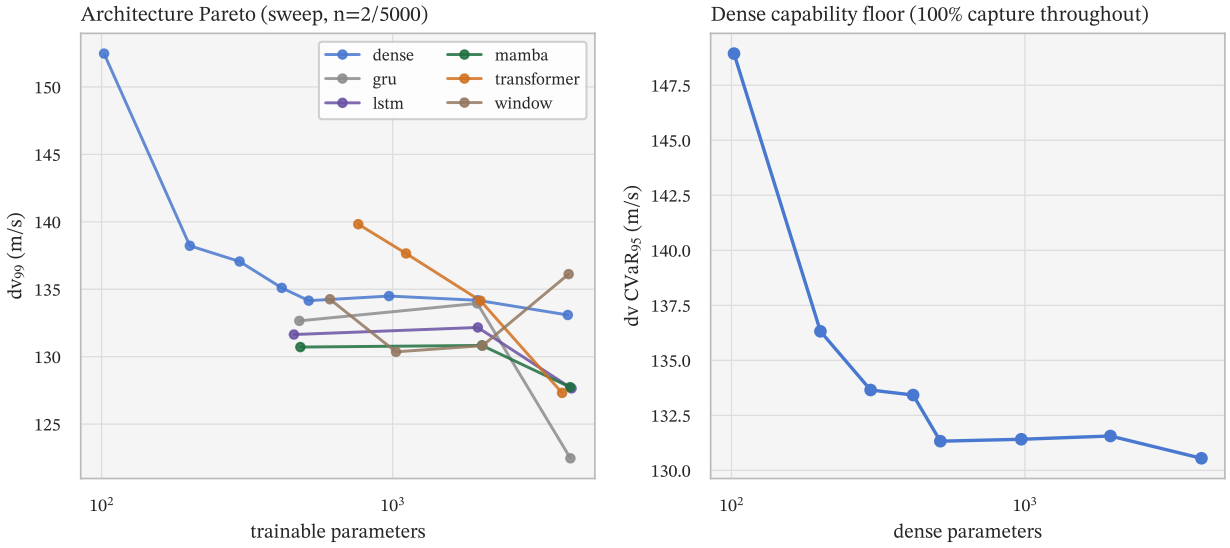


Figure 9: Parameter budget versus sizing tail per architecture family (left) and the dense-family capability floor (right). Every cell captures 100%; the dense cost is flat above a few hundred parameters; recurrent and state-space families edge dense at matched budget; the Transformer is penalized at small budgets. The left panel plots the 99th-percentile cost and the right panel CVaR_{95}; the body quotes means, under which the flat cells are indistinguishable.

6.2 The tail reversal

To resolve the tail we took the strongest recurrent and state-space candidates plus the dense reference to convergence – the headline allocation of two scenarios per generation, a population of 512, run for roughly fifteen to twenty thousand generations until each plateaued – and evaluated each once on the frozen confirmatory pool ($10 \times 100,000$ scenarios; the development $n = 10,000$ pool guided the campaign and agrees within 1–2 m/s throughout). Because a single run carries real run-to-run scatter, we repeated the deciding cells over three independent seeds and report the mean and range (Figure 10). (The GRU, which had the best sweep mean, was also taken to convergence; its single confirmatory run lands between the Mamba and the LSTM three-seed means – $\mathrm{CVaR}_{99.9}$ 126.4 [126.0, 126.8] – but we did not repeat it over seeds, so it stays out of the three-seed comparison.)

On the far-tail $\mathrm{CVaR}_{99.9}$, the depth at which the tanks are sized, the confirmatory-pool ordering is

$$\underbrace{\mathrm{Mamba}_{962}}_{125.5} < \underbrace{\mathrm{LSTM}_{1082}}_{131.5} < \underbrace{\mathrm{Dense}_{515}}_{140.5} \tag{7}$$

m/s (three-seed means; $10 \times 100,000$ scenarios per seed, replicate standard errors of 0.1–0.9 m/s, so the per-seed values are essentially exact and the residual spread is training-run variance). The deployed artifact separates cleanly: its 123.3 (95% CI [123.0, 123.6]) sits below every dense retraining seed (128.7 / 139.2 / 153.7) and every feasible LSTM seed, with a paired margin of 5.4 [4.4, 6.4] m/s over even the best dense seed. Across retrainings the three-seed mean gap to dense is 15.0 m/s against a combined seed-scatter standard error near 8; the per-seed ranges touch once (the Mamba's worst seed, 131.0, against the dense reference's lucky first seed, 128.7), so the architecture-level claim is a strong ordering of means and of consistency – the Mamba's seeds span 8.8 m/s where the dense reference's span 25.0 – rather than disjoint ranges. Crucially, the advantage is invisible at shallow depth – on the shared $n = 1000$ pool the Mamba and the dense reference are a statistical tie on the mean (+0.1 m/s; 95% CI [−0.1, +0.3]) and the Mamba leads by only 1.6 m/s at CVaR_{95} – and it grows monotonically with tail depth. The dense network's tight median masks a fat, high-variance extreme tail: its three runs span $\mathrm{CVaR}_{99.9}$ from 129 to 154 m/s. Sizing from a single dense run could quote 129 m/s and still be unlucky in flight; the recurrent policy's tail estimate is consistent across retraining. The sample maximum, by contrast, retires as a comparison statistic at this depth: at 10^6 scenarios it is a single draw from the extreme tail – the Mamba seed with the **lowest** $\mathrm{CVaR}_{99.9}$ (122.2) also

logged the campaign's deepest single excursion (412 m/s) – which is precisely why the sizing metric is an expected shortfall and the maximum is reported as descriptive only.

One feasibility asterisk belongs on the LSTM. Its best run – the seed with the lowest training loss and the tightest tail of its three – exceeds the integrated heat-load limit on 13.7% of the confirmatory pool (15.6% of the $n = 1000$ pool: its p_{95} heat load sits above the 25 MJ/m² limit), so its Δv tail is bought partly with heat. Its two repeats violate nothing, and neither do the GRU or the Mamba on any pool (the dense reference grazes the limit on 2 of $10,000$ development draws and 0.01% of the confirmatory pool). The LSTM three-seed mean therefore mixes one infeasible run with two feasible ones. We adopt a **feasibility-first** rule for every ranking and deployment statement: a run must satisfy all three constraints on every pool it was evaluated on, or it is excluded from the comparison. Under that rule the LSTM ranks by its feasible-seeds mean (135.2 m/s $\mathrm{CVaR}_{99.9}$) and the ordering Mamba $<$ LSTM $<$ dense is unchanged – the Mamba wins the sizing tail entirely inside the constraint envelope with or without the rule; Table 4′s violation column makes the same check for the classical schemes. The infeasible seed also shows that the soft constraint penalty does not by itself enforce feasibility – one of eleven converged runs bought tail performance with heat – which is why the deployment rule is feasibility-first rather than penalty-trusting.

The control that pins this on architecture rather than parameter count is the equal-capacity pair. At roughly 960 parameters, the Mamba (three-seed 125.5 m/s $\mathrm{CVaR}_{99.9}$; deployed seed 123.3) beats the dense network of the same size (972 weights, 131.1 [130.5, 131.6], single run) – the state, not the parameters, buys the tighter tail. And the dense family does not reward extra capacity: the 972-weight network plateaus to a worse validation loss than the half-size 515-weight reference and is beaten by it on the $n = 1000$ pool (−2.7 m/s mean, −4.7 m/s at CVaR_{95}, paired), exactly the dimensionality effect from the plateau. On the far-tail metric itself the two dense nets are not cleanly separable: the 972-weight net's single confirmatory run, 131.1 m/s, sits inside the 515-weight net's three-seed spread, and we did not repeat the 972 net, so its tail carries no measured scatter; the dense-versus-dense comparison is conclusive only on validation loss and the median. More dense parameters do not buy a better policy; in this campaign internal state does, and the deployed Mamba's 123.3 undercuts both dense nets regardless.

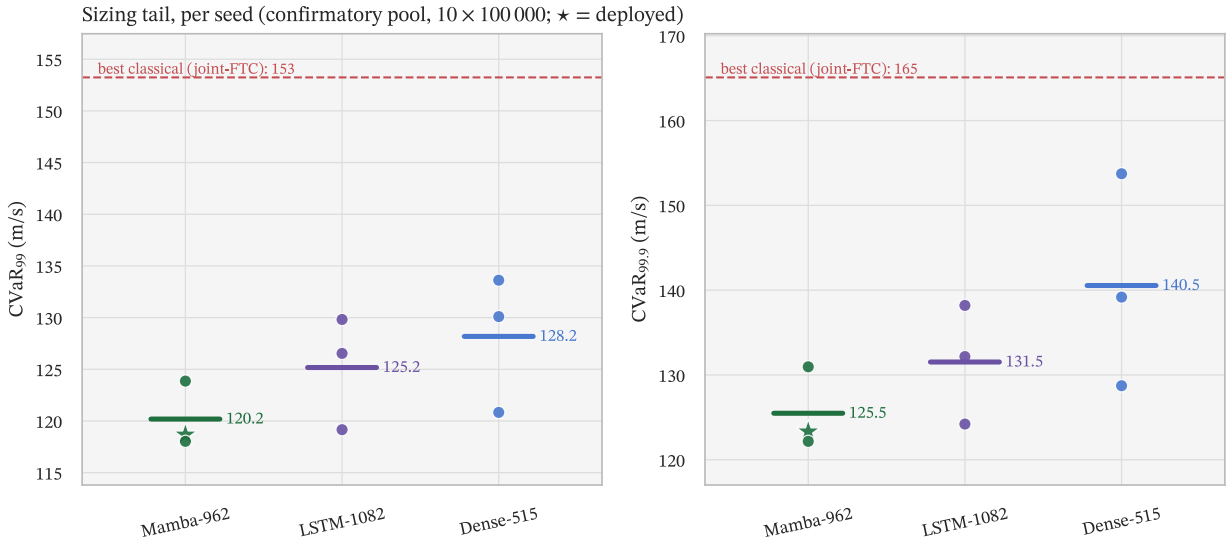


Figure 10: Per-seed far-tail $\mathrm{CVaR}_{99.9}$ on the confirmatory pool ($10 \times 100,000$ scenarios per seed; replicate 95% CIs are narrower than the markers). Mamba is lowest and tightest across retraining, the deployed seed sits below every dense seed, and every network seed sits well below the best classical scheme (the LSTM's best seed carries the heat-load caveat of the text).

6.3 Why: training loss does not pick the tail winner

The mechanism is the most surprising part. The three policies reach nearly the same training objective – validation RMS 1.331×10^6 (Mamba), 1.326×10^6 (dense), and 1.276×10^6 (LSTM, the **lowest** training loss of the three) – yet their deployed far tails order Mamba below LSTM below dense. Validation loss is not blind: across the eleven converged runs it orders the seeds **within** each family exactly as the tail does (Figure 11). What it cannot see are the offsets **between** families: the lowest-loss run of the campaign (the LSTM) has neither the lowest nor a feasible tail, and at indistinguishable loss the dense reference concedes 6 m/s of far tail per run (15 in three-seed mean) to the Mamba. Selecting the deployed model on validation loss would have picked the wrong cell. Why the selective state space edges the gated cell – input-conditioned recurrence better matched to the density process, or simply a friendlier search landscape at this budget – our three-seed evidence cannot separate, and we claim no mechanism for the intra-recurrent ordering. What we can say is that more sophisticated memory does not help: Appendix B probes three further recent recurrent families (CfC, the xLSTM cells, Mamba-3's axes) at matched budget, and none beats the plain cells – two arms are significantly worse. The reason is visible in the median: every architecture we trained to convergence, dense included, reaches the same 108–112 m/s typical cost, because the engineered, cost-aligned inputs (the predicted-Δv components above all) already encode most of what a memory cell could recover. Recurrence is redundant in the bulk. It earns its place only on the handful of hardest scenarios – the deep-tail draws where the static inputs are not enough and genuine internal state, carried across the pass, lets the policy anticipate rather than react. That is precisely the part of the distribution that sizes the mission, and precisely the part a validation-RMS objective under-weights.

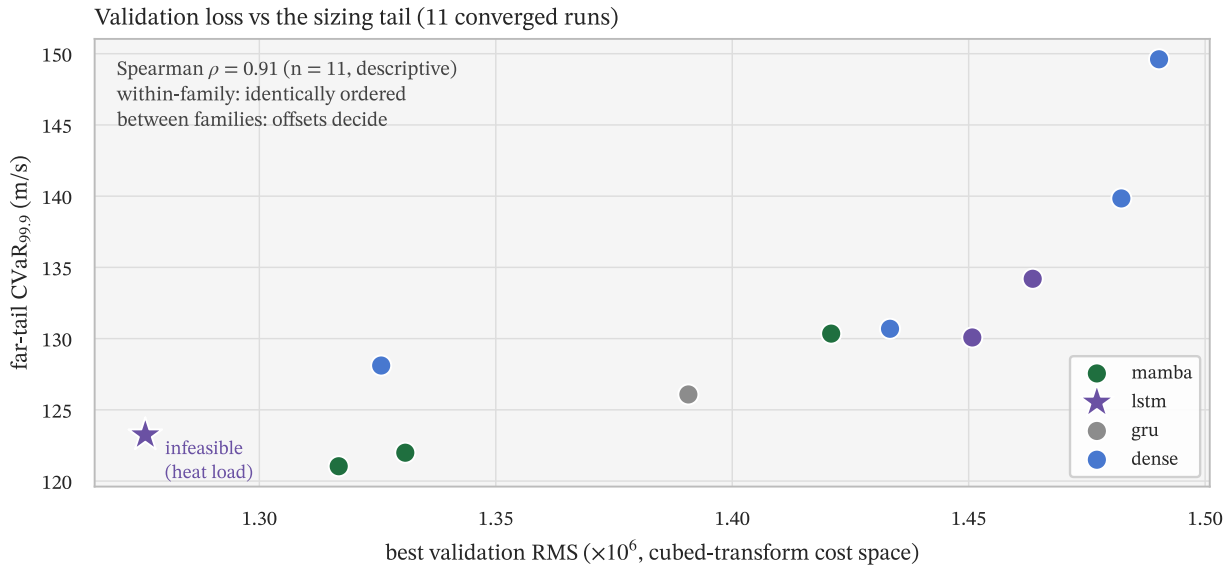


Figure 11: Best validation RMS versus far-tail $\mathrm{CVaR}_{99.9}$ ($n = 10,000$) for the eleven converged runs. Within every family the runs order identically on both axes (overall Spearman $\rho = 0.91$, pooled over families and read as descriptive – the runs are not exchangeable across families); what the loss cannot see are the offsets between families – the lowest-loss run (the LSTM, starred: the heat-load-infeasible one of Section 6.2) is not the best tail, and the dense cells sit well above the Mamba at matched loss.

Three controls, run for this revision and evaluated once on the frozen confirmatory pool of Section 7, pin the mechanism. **State reset**: the deployed Mamba evaluated with its recurrent state zeroed at every guidance tick collapses from $\mathrm{CVaR}_{99.9} = 123.3$ to 414.5 m/s at unchanged 100% capture – the deployed policy computes with its state; it is not a feedforward law in disguise. **Matched history**: a dense cell fed an explicit five-tick observation window (970 parameters, identical budget and regime) reaches 142.2 ± 0.3 – inside the dense family's seed-to-seed spread, 19 m/s above the intact policy, with a 423 m/s worst case at 10^6 scenarios; short temporal context does not substitute

for learned state. **No predicted-Δv**: the same Mamba architecture retrained without the three cost-aligned inputs reaches 138.9 ± 0.2 (median 113.1) – the engineered inputs matter for bulk and tail alike, so inputs and state are complements rather than substitutes; notably, even this stripped network beats the best classical scheme on every reported statistic (CVaR_{95} 125.3 versus joint-FTC's 144.3), so the network's edge does not ride on its privileged observations. In sum: state is necessary for the deployed tail (the first two controls) and not sufficient without the cost-aligned inputs (the third).

The deployed headline policy is therefore the Mamba network: a 962-parameter dense-to-selective-state-space-to-dense stack (a 17-input dense encoder, a Mamba core of inner width 16 and state size 12, a two-output dense decoder with the atan2 bank decoder), trained under the full methodology of Section 4. It captures 100% of the time at 109.9 m/s mean and 115.2 m/s CVaR_{95} on a fresh, never-trained-or-selected-on pool – within rounding of its 2 M-pool numbers (the 115.4 of Table 4) – and, on the frozen confirmatory pool, captures 10^6 of 10^6 scenarios at $\mathrm{CVaR}_{99.9} = 123.3 \pm 0.1$, within 1.3 m/s of the development-pool estimate: there is no selection optimism in the headline. The 515-parameter dense network remains the **efficiency reference**: half the parameters, no internal state, and a competitive median, at the cost of the fat tail just described. If compute or simplicity is the binding constraint it is the better pick; if the mission is sized off the tail, the Mamba wins.

A final architectural detail concerns the bank decoder. Among the single-output decoders that attack the $\pm\pi$ wrap seam, the classical two-output atan2 decoder of Equation 2 still wins: it reaches 117.4 m/s mean and 128.7 m/s CVaR_{95} against the delta decoder (119.9/141.6) and the scaled-π decoder (122.2/140.4), with the edge concentrated on the tail (roughly 12–13 m/s at CVaR_{95}, paired) (Figure 12). The decoder we inherited from 2009 is still the right one.

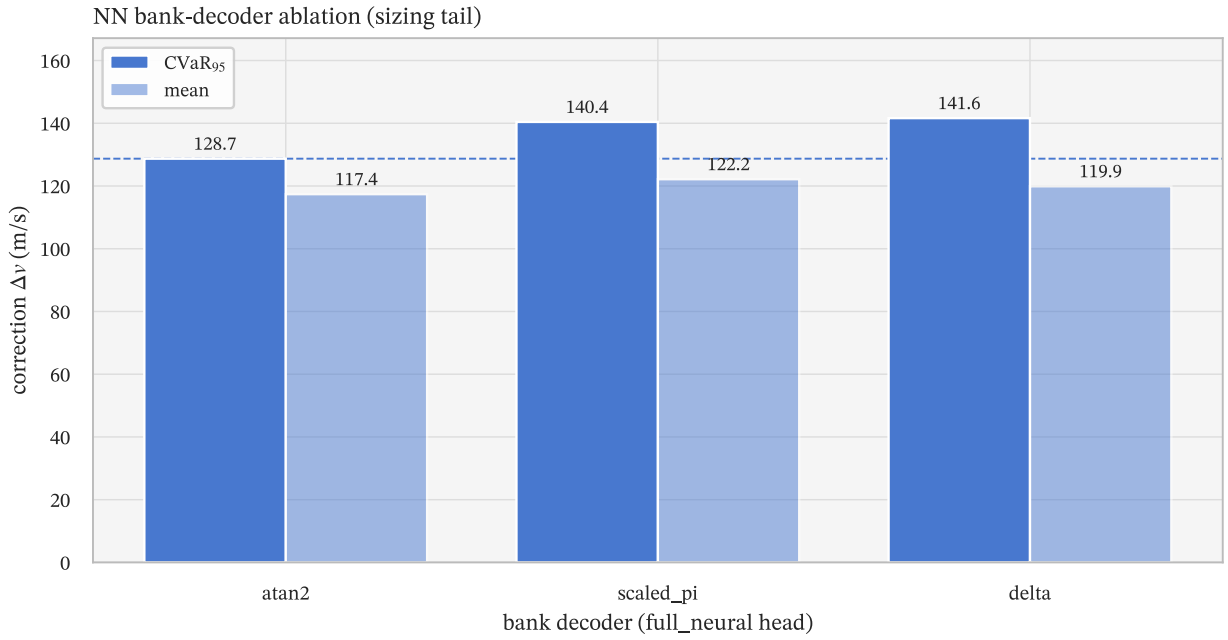


Figure 12: Bank-decoder variants on the 515-parameter dense network. The two-output atan2 decoder inherited from the 2009 work wins, with most of its advantage on the tail.

7 Classical versus neural network

We now place the neural policy against the classical schemes on identical Monte Carlo scenarios – the same seed pools, the same dispersions, the fair comparison we have always insisted on. Two things have to be established: that the classical baselines are tuned to their best, and that the comparison is read at sizing depth.

7.1 A co-optimized reference recovers the predictor–correctors

Before comparing against the classical schemes we must give them their best shot. The reference-tracking laws (FTC, the energy controller, PredGuid) are only as good as the reference trajectory they enslave to, and the legacy constant-bank reference is not optimal. We therefore let the genetic algorithm co-optimize the reference: a single extra gene sets the constant bank angle that generates the reference table, regenerated per individual, so the law and the trajectory it tracks adapt together. The effect is large (Figure 13). FTC falls from 170.7 to 126.3 m/s mean (a 44 m/s improvement) and from 244.1 to 142.9 m/s at CVaR_{95}, and the network beats it on every one of 1000 paired scenarios ($p < 10^{-15}$, saturated). The energy controller recovers by 35 m/s and PredGuid by 23. The reference **was** FTC's weakness: a feedback law tracking a poor target cannot out-perform the target.

With its reference repaired, FTC (126.3 m/s / 142.9 CVaR_{95}) becomes the best classical scheme, within about 2 m/s in mean of the far more expensive FNPAG (124.3 / 144.0) – the paired gap is statistically resolvable ($p \approx 10^{-23}$) but operationally negligible, and at the far-tail sizing depth the repaired FTC pulls decisively ahead (165.1 versus 198.7, Section 7.2) – and it is analytic and fast. The aerocapture story we wrote in 2009, in which FTC was the baseline to beat, becomes: a **well-referenced** FTC is the classical state of the art, and it costs a fraction of a numerical predictor–corrector to run.

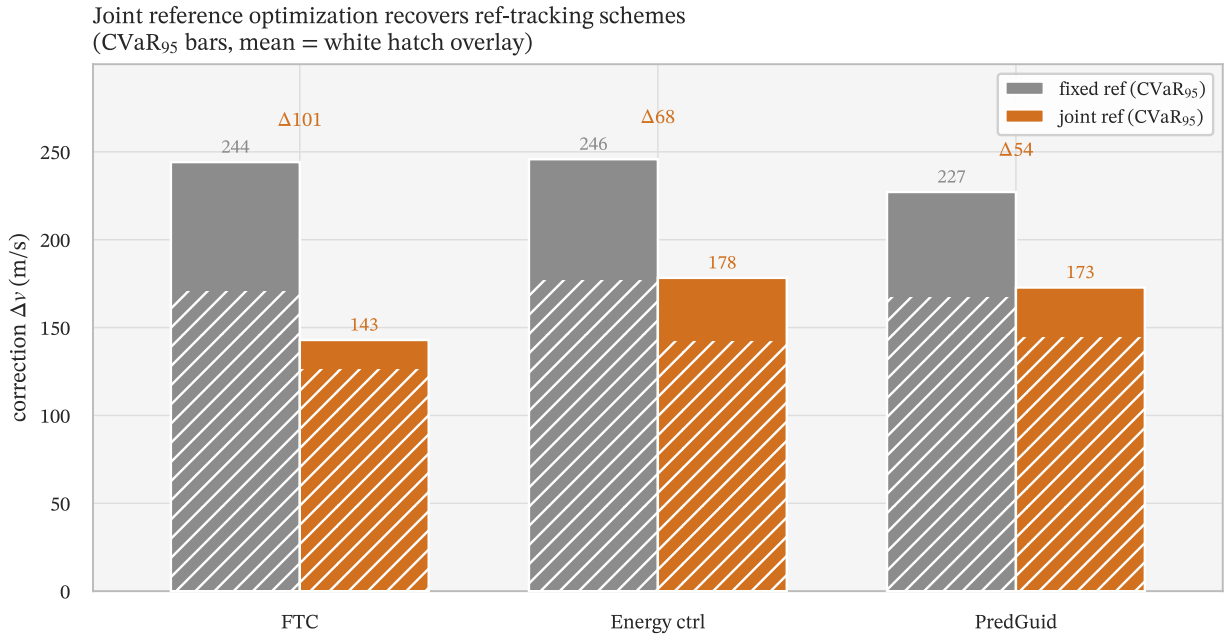


Figure 13: Fixed versus co-optimized reference for the three reference-tracking schemes. Co-optimizing the constant-bank reference recovers most of FTC's deficit and lifts it to the best classical scheme.

7.2 The deployability triangle

Three schemes define the deployability frontier: the neural network, the well-referenced FTC, and FNPAG. They trade off along three axes – accuracy, compute, and robustness – and no single scheme dominates all three.

Accuracy. On the sizing tail the network wins outright. Its far-tail $\mathrm{CVaR}_{99.9}$ of 123.3 ± 0.1 m/s sits 41.8 m/s [41.2, 42.4] below joint-FTC (165.1) and 75.4 [71.4, 79.3] below FNPAG (198.7) on the frozen confirmatory pool. FNPAG's extreme tail fattens with depth in a way its shallow statistics hide – CVaR_{95} matches the development pool at 143, but $\mathrm{CVaR}_{99.9}$ grows from 165 at $n = 10^4$ to 198.7 at 10^6, with 163 physical crashes in 10^6 (all surface impacts on individual re-runs at 12 × the evaluation timeout, not censoring) – so the repaired FTC is decisively the better classical scheme at sizing depth. On the shared paired pool it beats joint-FTC by 16.4 m/s in mean, 23.8 at p_{95}, and 27.6 at CVaR_{95}, winning all 1000 scenarios; against FNPAG the margins are 14.4 / 23.4 / 28.7 m/s, winning

998 of 1000 (Table 5). The tail margin is consistently **larger** than the mean margin – the network's advantage is precisely where the mission is sized.

Compute. On a single idle core, the dense network runs at 1.88 ms per simulation and the stateful Mamba at 3.14 ms, against 0.90 ms for FTC and 87.1 ms for FNPAG (Figure 14). The network is roughly three and a half times FTC – the same fast class – and 28 × faster than the numerical predictor–corrector. On accuracy and compute FNPAG is dominated – joint-FTC matches its accuracy and the network beats it, both at a small fraction of its cost – though the off-nominal stress below keeps robustness a separate axis. The selective-state-space core costs about 1.7 × the dense network, the price of the tail it buys. Per control action rather than per trajectory: the network's forward pass costs ≈ 4 µs per 1 s guidance update (whole-simulation cost divided by the ≈ 734 updates flown – an upper bound on pure inference), while FNPAG's predictor work amounts to ≈ 0.27 ms per 2 s replan cycle. Flight processors run one to two orders of magnitude slower than the laptop core measured here; at a conservative 100 × scaling neither scheme breaches its own deadline (≈ 27 ms per replan against 2 s; ≈ 0.4 ms per update against 1 s), but the deadline margins differ by a factor of thirty, and the relative cost ordering is host-independent. None of these measurements establish worst-case execution time or memory on qualified hardware (Section 9).

Robustness – the honest caveat. We trained the network on the medium dispersion regime. Under a deliberately harsher off-nominal regime (atmosphere, density perturbation, navigation, and filter all set high), the picture inverts on robustness, and we report it plainly because it is the one place the network loses (Figure 15). All stress-regime tail statistics are conditional on capture – $\mathrm{CVaR}_{95}(\Delta v \mid \text{capture})$ – and a conditional tail can improve by failing the hardest scenarios, so we read every stress comparison lexicographically: capture probability first, conditional tail cost second, and no tail win is claimed across a capture-rate deficit. The analytic joint-FTC degrades least – its capture rate falls by 5.5 points and its CVaR_{95} inflates by 197 m/s – against the network's 9.9-point capture drop and +402 m/s inflation; PredGuid (−9.3 pts, +297) sits between, FNPAG loses less capture (−7.1 pts) but inflates its tail the most (+490), and the **fixed**-reference FTC collapses entirely (−33 points), which again ties the robustness of FTC to its reference. The lesson is not that the network is fragile – it captures 90% even far outside its training regime – but that a training-free analytic law extrapolates better than a policy trained on a narrower distribution. The network wins the nominal sizing tail it was trained for; widening its training regime to recover off-nominal robustness is future work, not a property we can claim.

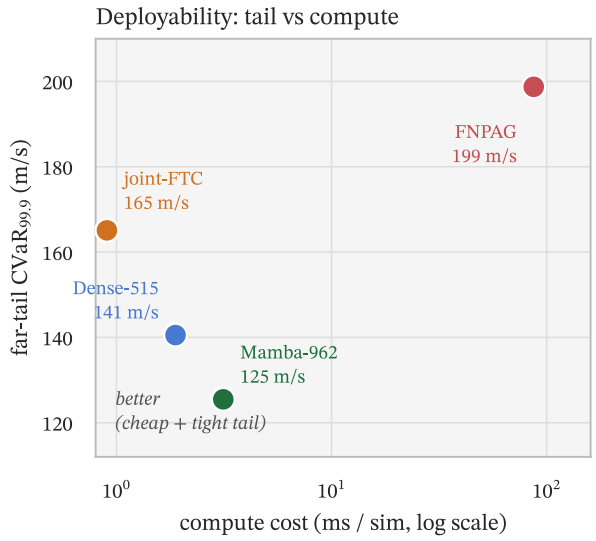


Figure 14: Deployability: per-simulation compute versus far-tail $\mathrm{CVaR}_{99.9}$. Both network points sit lower-left – a tighter tail than every classical scheme and far cheaper than FNPAG.

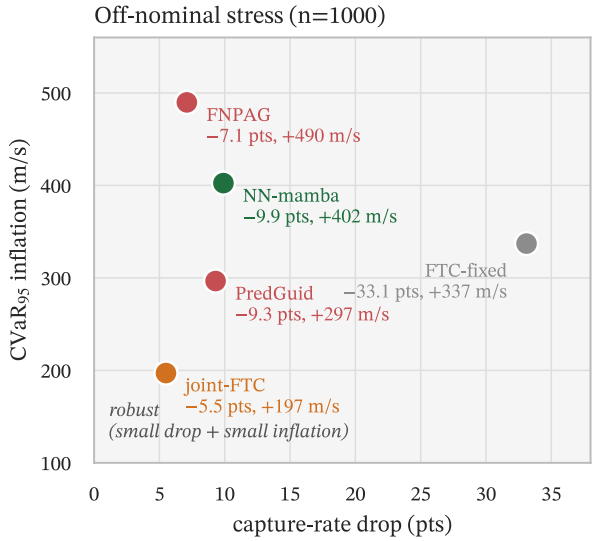


Figure 15: Off-nominal stress. The analytic joint-FTC is the most robust to distribution shift; the medium-trained network generalizes less well, the paper's robustness caveat.

Table 4: Final Monte Carlo performance, correction Δv in m/s, ordered by CVaR_{95}. Capture / Viol. / mean / p_{95} / CVaR_{95} are on the $n = 1000$ final-evaluation pool; †$\mathrm{CVaR}_{99.9}$ is the far-tail sizing metric on the frozen confirmatory pool ($10 \times 100,000$ scenarios per scheme; replicate standard errors 0.1–3.6 m/s; each average pools 1000 tail observations). Network rows tabulate the value the feasibility-first rule of Section 6.2 ranks the cell by – the deployed Mamba seed, the dense efficiency-reference seed, and the LSTM feasible-seeds mean – with per-seed values in Section 6.2. Reference-tracking schemes appear in both their fixed- and co-optimized (joint) reference forms (Section 7.1). Viol. is the fraction of draws exceeding any of the heat-flux, g-load, or heat-load limits: the deployed network, the tuned predictor–correctors, and the joint-reference trackers violate none; the fixed-reference FTC, equilibrium glide, and piecewise constant carry heat-flux exceedances of at most 1.1%. Sub-100% captures are the off-corridor draws of the weaker schemes; a 100% capture rate means "no failures observed" – with zero failures in n independent scenarios the one-sided 95% upper bound on the failure probability is $\approx 3/n$ (3×10^{-4} at $n = 1000$; 3×10^{-6} at the confirmatory $n = 10^6$). The mean is reported for continuity with the 2009 work but is operationally secondary to the tail. ‡The LSTM's best seed – its lowest-training-loss one – exceeds the heat-load limit on 15.6% of this pool (13.7% of the confirmatory pool); the tabulated 135.2 is its two feasible seeds' mean, and the raw three-seed mean including the infeasible seed is 131.5. Confirmatory capture is 100% for the network rows, joint-FTC, and fixed-reference FTC; 99.85–99.98% for the remaining classical schemes (their † values are conditional on capture; FNPAG's 163 failures in 10^6 were individually re-run at $12 \times$ the evaluation timeout and are all physical crashes).

Scheme	**Capture %**	**Viol. %**	**Mean**	p_{95}	$\mathbf{CVaR_{95}}$	$\mathbf{CVaR_{99.9}}$†
NN – Mamba (deployed)	100.0	0.0	109.9	114.0	**115.4**	**123.3**
NN – LSTM‡	100.0	15.6	108.4	114.0	116.0	135.2
NN – dense (efficiency ref.)	100.0	0.0	109.7	114.9	117.0	128.7
FTC (joint reference)	100.0	0.0	126.3	137.8	142.9	165.1
FNPAG	100.0	0.0	124.3	137.4	144.0	198.7
PredGuid (joint reference)	100.0	0.0	144.2	164.2	172.8	225.8
Energy controller (joint reference)	100.0	0.0	142.1	166.3	178.3	304.3
PredGuid (fixed reference)	100.0	0.0	167.4	209.8	227.1	301.6
FTC (fixed reference)	100.0	0.2	170.7	208.9	244.1	341.4
Energy controller (fixed reference)	99.6	0.0	176.7	226.0	245.8	308.1
Equilibrium glide	99.5	0.5	200.3	290.0	327.6	410.1
Piecewise constant	99.8	1.1	258.3	374.6	421.1	598.2

Table 5: Paired comparisons on the shared $n = 1000$ pool, correction Δv in m/s; negative Δ favors A. The CI is a 10,000-resample bootstrap on the paired mean difference; the tail deltas carry the same paired-resample construction (Δp_{95} / $\Delta\mathrm{CVaR}_{95}$ 95% CIs – Mamba vs joint-FTC $[-24.9, -22.5]$ / $[-29.7, -25.5]$; vs FNPAG $[-25.1, -21.5]$ / $[-31.4, -26.0]$; vs dense $[-1.6, -0.1]$ / $[-2.6, -0.6]$; vs LSTM $[-0.6, +0.5]$ / $[-1.3, +0.1]$, the one interval straddling zero – the intra-network separation lives at the far-tail depth, not here). Win-rate and p (Wilcoxon signed-rank) are computed on the per-scenario cost; p is truncated at 10^{-15} – the normal-approximation statistic saturates (near 10^{-165}) at or near sign unanimity at $n = 1000$, certifying a (near-)unanimous direction, not a resolved tail probability. Deltas are computed on unrounded per-scenario values, so a delta may differ from the difference of the rounded marginals in Table 4 by 0.1 m/s. ‡For the two intra-network rows the win-rate is driven by the bulk of the per-scenario differences, where the dense and LSTM networks match or slightly beat the Mamba; the headline ordering lives in the tail ($\Delta\mathrm{CVaR}_{95} < 0$ and, at the far-tail sizing depth, confirmatory paired replicate deltas of $\Delta\mathrm{CVaR}_{99.9} = -41.8$ $[-42.4, -41.2]$ vs joint-FTC, -5.4 $[-6.4, -4.4]$ vs the dense reference seed, and -15.0 in three-seed means). The LSTM row additionally carries the heat-load feasibility caveat of Section 6.2.

Comparison (A vs B)	**Δmean**	**95% CI**	**Δp_{95}**	**$\Delta\mathrm{CVaR}_{95}$**	**A-win %**	**p**
Mamba vs FTC (fixed ref.)	-60.8	$[-62.4, -59.2]$	-95.0	-128.8	100.0	$< 10^{-15}$
Mamba vs FTC (joint ref.)	-16.4	$[-16.8, -16.0]$	-23.8	-27.6	100.0	$< 10^{-15}$
Mamba vs FNPAG	-14.4	$[-14.8, -14.0]$	-23.4	-28.7	99.8	$< 10^{-15}$
Mamba vs dense (eff. ref.)	$+0.1$	$[-0.1, +0.3]$	-0.9	-1.6	44.9‡	0.02
Mamba vs LSTM	$+1.4$	$[+1.2, +1.6]$	-0.0	-0.6	29.2‡	3×10^{-46}
FTC: joint vs fixed reference	-44.4	$[-45.9, -42.9]$	-71.2	-101.2	100.0	$< 10^{-15}$
FTC (joint) vs FNPAG	$+2.0$	$[+1.5, +2.5]$	$+0.4$	-1.1	33.9	1×10^{-23}

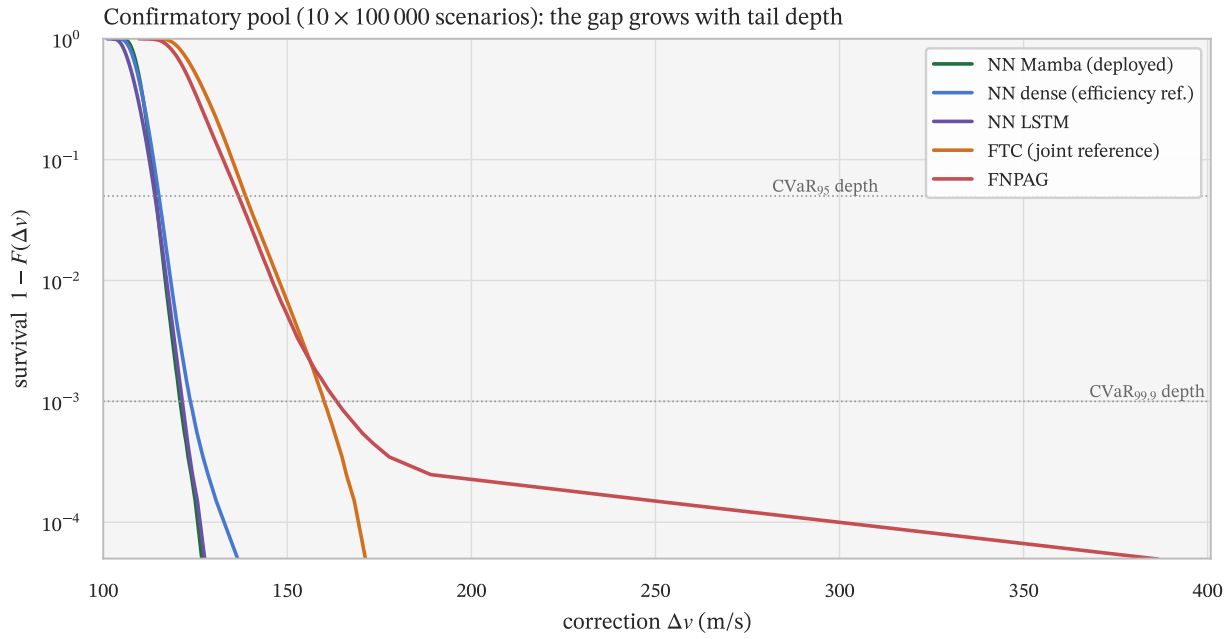


Figure 16: Empirical survival curves of the correction Δv on the confirmatory sizing pool ($10 \times 100,000$ scenarios per scheme; log scale, curves subsampled every $\approx$ 100th order statistic). The network-to-classical separation grows with tail depth – the sizing thesis in one picture.

7.3 Matching the objective to the regime closes the gap

The robustness caveat of Section 7.2 invites a question: is the off-nominal gap intrinsic to neural guidance, or an artifact of a training objective tuned for the wrong regime? Before answering it, the regime itself deserves scrutiny. The high-dispersion stress pool is not a realistic operating point – no mission would be flown on an entry profile that fails roughly one pass in twenty under **any** guidance scheme. In practice the entry interface would simply be re-targeted so that the 3σ dispersion envelope still captures, restoring a near-certain margin by construction. We therefore read this regime not as a deployment scenario but as a deliberate **stress probe of the training method**: it pushes the optimizer into a regime where roughly 5% of scenarios are catastrophic, and asks whether the genetic algorithm can still make progress.

Under that probe, it cannot – not with the objective that wins in the medium regime. Retraining the Mamba head on the high regime with the deployed objective (the cubed transform and hardest-seed curation of Section 4) stalled: the validation cost plateaued for thousands of generations, and the deployed policy came out **worse** than the medium-trained one it was meant to replace – 27% capture on the stress pool with a CVaR_{95} of 1216 m/s, against the medium-trained network's 90% and 518. The mechanism is the one the medium regime hides: when failures dominate, cubing the cost makes the objective a spiky near-discrete failure count, max-bucket curation feeds the optimizer only the hardest seeds, and a per-individual scenario batch as small as the deployed allocation's two cannot estimate any of it – so selection has no gradient to climb. The worst-case shaping that is a virtue in the medium regime becomes a liability once the noise outruns the sample budget.

Centering the objective – more scenarios per individual (a real cost estimate), a central curation bucket, and a milder transform – restores the gradient. On the dense vehicle a one-lever-at-a-time sweep (Figure 17) attributes the recovery: more sims is the clean lever (it halves the tail, CVaR_{95} $1031 \to 523$, while **holding** capture at 95%); a milder transform helps moderately; and the central bucket **alone** is a trap – it drops capture to 84.5%, because one cannot pick a central representative from a cost distribution two samples cannot resolve. The three knobs are a coupled system, exactly as in the medium regime, but now the coupling has teeth: only all three together recover both capture and the tail (dense CVaR_{95} $1031 \to 276$, mean $574 \to 157$). The effect transfers cleanly to the deployed architecture – the centered Mamba reaches CVaR_{95} 273 m/s at 94.9% capture against the stalled stack's 1216 at 27% – so the stall was the objective, not the cell type.

The payoff, stated with its caveats, is that on this evidence the off-nominal gap is not intrinsic to neural guidance. With a regime-matched objective the centered Mamba (CVaR_{95} 273 m/s at 94.9% capture) beats the best classical scheme on the very regime where the medium-trained network lost to it – joint-FTC retrained on the same regime sits at 424 m/s at 95.0% capture, and the medium-deployed joint-FTC at 340 at 94.5%: capture parity within half a point, so the lexicographic comparison is decided by the conditional tail. The analytic law's edge in Section 7.2 was a property of the **mismatched** training objective, not of neural guidance. The result now carries measured run-to-run scatter: three independent centered-Mamba retrainings hold capture at 94.8–95.0% with $\mathrm{CVaR}_{95}(\Delta v \mid \text{capture})$ spanning 231–273 m/s – every seed beating both the retrained joint-FTC (424) and the medium-deployed one (340) – so the reversal is stable across training runs. The remaining caveats: these are $n = 1000$ figures (a sizing-grade number wants the $n = 10,000$ depth used elsewhere), and the regime that produces the gap is itself one a real mission would design away. The durable lesson is methodological and reinforces Section 4: the optimal worst-case weighting is matched to the environment's noise and the per-individual sample budget, not fixed once.

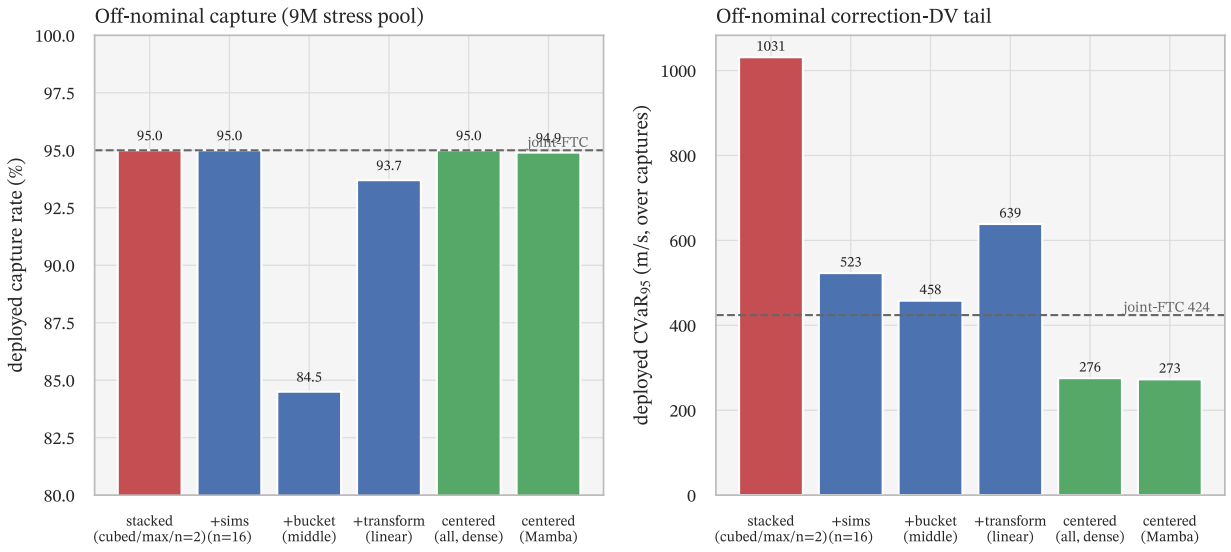


Figure 17: Objective-centering under the high-dispersion stress probe ($n = 1000$ on the 9M pool). Left: deployed capture rate; right: deployed correction-DV CVaR_{95} over captured runs. The medium-regime stack (red) carries an enormous tail; centering recovers it (green) and transfers to the Mamba, beating the retrained joint-FTC (dashed). The central bucket alone trades capture for the tail (left, 84.5%), so the three levers are read together. Capture and tail are shown separately because the levers trade them off.

8 What the network uses

To close the architecture argument we ask which inputs the deployed Mamba policy actually relies on, through a closed-loop input-sensitivity analysis: each input is zeroed in turn and the resulting cost increase measured (Figure 18). Zeroing a normalized input perturbs every subsequent state of the closed loop, so the deltas measure closed-loop dependence, not isolated feature importance. The ranking is informative. The largest degradations come from the orbital tracking signals – the eccentricity excess relative to the target, the nominal altitude rate, and the dynamic-pressure error – the quantities a reference-tracking law would feed back on. Immediately behind them are two of the engineered, cost-aligned **predicted correction-Δv** components: the periapsis-correction burn (`predicted_dv2`) and the plane-change burn (`predicted_dv3`), each evaluated on the current osculating orbit. The energy-closing burn (`predicted_dv1`) and the bank-history encodings contribute much less.

This is the mechanism behind the bulk-versus-tail split of Section 6 seen from the inside. The network's two strongest dependencies after the orbital errors are signals that tell it, causally and smoothly, what the maneuver would cost if it stopped now – a cost-to-go surrogate handed to the policy as an input. Because the osculating orbit is itself an integral of the flown trajectory, those signals already summarize most of the history a memory cell could recover, and a dense network without any memory matches the recurrent ones on the median: the engineered inputs do the work that recurrence would otherwise do. What they cannot capture is the small set of hardest scenarios where the future of the pass depends on more than the present osculating orbit, and that is exactly where the Mamba's internal state pays off and the dense tail frays. A per-input behavior report over the deployed policy shows no clustered failure mode; we did not attempt a per-scenario lower-bound analysis, so we do not claim the residual cost irreducible. The network is using the physics we handed it, and reserving its memory for the tail.

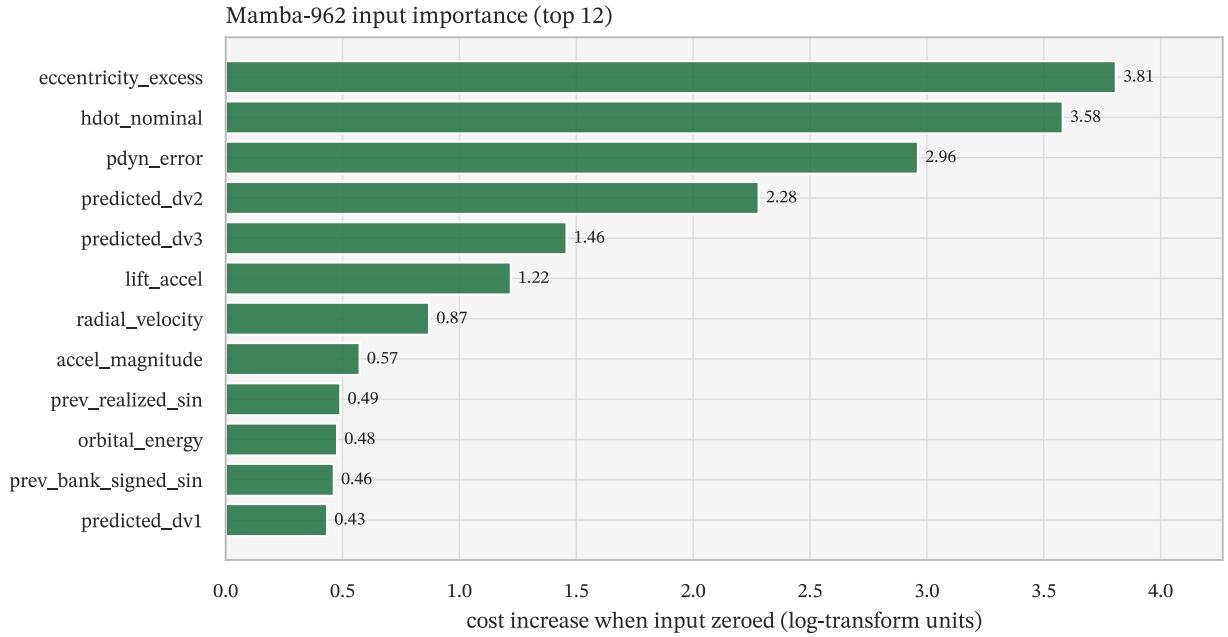


Figure 18: Closed-loop input sensitivity: per-input cost increase when each input is zeroed, for the deployed Mamba policy. The orbital tracking errors (eccentricity excess, altitude rate, dynamic-pressure error) dominate, followed by the engineered, cost-aligned predicted-Δv components.

9 Discussion and limitations

The clearest limitation is the off-nominal robustness gap of Section 7.2 – the deployed network wins the nominal sizing tail it was trained for and loses, off-nominal, to a training-free analytic law. Section 7.3 traces that gap to the training objective rather than to neural guidance, with the markers stated there: the centered-retrain demonstration is now three-seed (capture steady at $\approx 95\%$, conditional tail 231–273 m/s, every seed beating both FTC references) but remains $n = 1000$, and the stress regime is one a real mission would design away. What remains open is the far-tail depth used for the headline.

The shared-noise-path conditioning of Appendix E belongs in the same family, and it sharpens the same lesson from a different direction. The main-body numbers are internally consistent and the cross-scheme comparison is fair – every scheme saw the identical conditioning – but the networks overfit the shared density path where the analytic laws cannot, so the **margins** are regime-dependent: under per-scenario noise the scratch-retrained networks beat FNPAG by only about one run-to-run standard deviation at CVaR_{95}, the decisive shallow-tail margin comes from a fine-tune recipe, and the CVaR_{95} inter-architecture ordering compresses into σ_{run}. But the 10^6-scenario far-tail re-run separates the architectures again, at the depth Section 6 always claimed: the fine-tuned Mamba holds $\mathrm{CVaR}_{99.9} = 163.2 \pm 1.3$ m/s over three fine-tune seeds, with 99.996% of scenarios captured, while the dense fine-tune – the CVaR_{95} winner – and FNPAG both blow past 236. The recurrent advantage lives at the extreme tail, and only a million-scenario pool can see it. Twice now – off-nominal dispersions and per-scenario noise – the broader pattern is the same: the network is exactly as good as the distribution it trains on, and widening the training environment recovers what the narrow one gave away.

A second tradeoff is the cost of state. The deployed Mamba runs at 3.14 ms per simulation against 1.88 ms for the dense network – about $1.7\times$ for the selective-state-space core – which is the price of the tighter tail. The head itself accounts for ≈ 1.2 ms of that cost; single-precision arithmetic would roughly halve it, while integer quantization buys memory rather than speed at this scale (Appendix C). Both remain in the fast compute class, an order of magnitude below the numerical predictor–corrector, so the choice is between the network and FTC, not between the

network and FNPAG. If on-board compute or implementation simplicity is the binding constraint, the memoryless 515-parameter dense network is the efficiency reference: a competitive median at half the parameters, paying only on the tail.

A flight-qualification path for a stateful network remains open, and the deployability triangle suggests its shape: a simplex arrangement in which the analytic joint-FTC – whose off-nominal robustness Section 7.2 established – runs in parallel as an onboard monitor with authority to take over on envelope violation, while the network flies the nominal corridor it wins on. The deployed policy is a fixed forward pass of 962 parameters in double precision with no data-dependent control flow, so worst-case execution time and memory bound trivially; verifying the **decision** behavior across the dispersion envelope, rather than the code, is the open problem.

Two threads earlier drafts left open are now closed, and one remains. Appendix C quantizes the deployed head (weight-only): 8-bit is free, the SSM dynamics parameters are the 4-bit bottleneck, and a quantization-aware fine-tune holds the sizing tail ($\mathrm{CVaR}_{99.9}$ 122.8 versus 123.3) at a $4.9\times$ memory reduction – pruning remains open. The state-ablation thread is now closed by the three controls of Section 6.3 – state reset, matched history, and no predicted-Δv – so the tail mechanism is measured rather than hypothesized; what remains open there is only the intra-recurrent ordering (why the selective state space edges the gated cells), which the three-seed evidence cannot separate. Run-to-run variance is calibrated at the tail through the three-seed architecture repeats and, for the decisive seed-strategy and CMA-ES cells, at the mean through dedicated repeats (Section 4.1; the fixed-seed pathology is itself high-variance, a 33 m/s seed range); the remaining tight optimizer differences (the genetic algorithm at populations of 150 versus 300, for instance) stay reported as indistinguishable rather than ranked.

Finally, a methodological note for anyone reproducing this. The training is not bit-reproducible from a seed alone – it never was – because the non-stationary objective and the operator randomness make each run a fresh draw; the deployed policy is reproduced from its saved weights and checkpoints, not re-derived. This is the same property that forced us to measure run-to-run variance directly rather than assume it away.

10 Conclusion

Seventeen years ago we showed that a feed-forward network trained by a genetic algorithm could fly an MSR aerocapture more efficiently than a Cerimele–Gamble feedback law, and we asked for a comparison against predictor–correctors. This paper delivers it, and the answer is favorable to neural guidance on the metric that matters. A 962-parameter recurrent (Mamba) policy captures every one of 10^6 pre-registered confirmatory scenarios and, on the far tail that sizes the propellant tanks, reaches $\mathrm{CVaR}_{99.9} = 123.3 \pm 0.1$ m/s – 42 m/s below the best classical scheme and beating a well-referenced FTC by 16.4 m/s in mean and 27.6 at CVaR_{95}, on every one of a thousand paired scenarios, running $28\times$ faster than the numerical predictor–corrector.

Two findings carry beyond the headline number. The first is methodological: a genetic algorithm is the wrong optimizer for a fixed objective and the right one for a moving one, and the moving Monte Carlo environment – adaptive seeds, a tail-weighting cost transform, and hardest-case curation – is a matched system that converts it from the worst optimizer we tested (154 m/s three-seed mean) to the best (120). The second is architectural: engineered, cost-aligned inputs flatten the typical cost across every cell type we tried, so the network's internal state earns its place only on the hardest scenarios – the extreme tail – which is exactly the part of the distribution that sizes the mission and exactly the part a validation-loss objective under-weights. Training loss did not pick the tail winner; architecture did.

A third finding arrived while preparing this revision, and we report it with the same candor we ask of others. The historical pipeline conditioned every scenario on one sample path of the time-varying density noise; under per-scenario noise the shared-path-trained networks lose 54–102 m/s of CVaR_{95} where the classical schemes lose 11 –31 (Appendix E). Retraining under the repaired seeding restores 100% capture and full constraint feasibility for every cell – including the LSTM whose deployed champion had been heat-load infeasible – and the correction ends by **strengthening** the thesis it tested. At CVaR_{95} the architectures compress into run-to-run variance and a dense fine-tune takes the shallow tail (129.8 m/s against FNPAG's 154.3); but on a fresh 10^6-scenario far-tail confirmatory under honest noise, the fine-tuned recurrent policy holds $\mathrm{CVaR}_{99.9} = 163.2 \pm 1.3$ m/s (three fine-

tune seeds, 99.996% capture) while the dense fine-tune and FNPAG both sit near 237. The internal state earns its keep exactly where the shared-path study said it did – on the extreme tail that sizes the tanks – and that claim now stands on the marginal distribution, not on one noise path.

The honest drawback that closed the 2009 paper – that the training is too heavy to run on board – remains, but its sting is gone: the deployed policy is a fixed forward pass that costs a few milliseconds, and the heavy optimization happens once, on the ground – ground compute buying in-flight margin. The next step would be to widen the training environment – annealing the tail-weighting and the dispersion envelope over the run, or stratifying the curated batch across cost quantiles – until the network's off-nominal robustness matches its nominal accuracy, and to carry these stateful policies beyond the single capture maneuver – to skip-entry and Earth-return legs, and to on-line adaptation of the deployed policy in flight.

References

Bairstow, S.H. (2006) *Reentry Guidance with Extended Range Capability for Low L/D Spacecraft*. Master's thesis.

Beck, M. *et al.* (2024) "xLSTM: Extended Long Short-Term Memory," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Calkins, G.E. *et al.* (2025) "Risk-Aware Aerocapture Guidance Through a Probabilistic Indicator Function."

Cerimele, C.J. and Gamble, J.D. (1985) "A Simplified Guidance Algorithm for Lifting Aeroassist Orbital Transfer Vehicles," in *AIAA 23rd Aerospace Sciences Meeting*.

Cho, K. *et al.* (2014) "Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734.

Forget, F. *et al.* (1999) "Improved General Circulation Models of the Martian Atmosphere from the Surface to Above 80 km," *Journal of Geophysical Research: Planets*, 104(E10), pp. 24155–24175.

Gelly, G. and Ferreira, E. (2007) "Guidance Algorithm Using Neural Networks for a Soft-Landing Application," in *European Conference for Aerospace Sciences (EUCASS)*.

Gelly, G. and Gauvain, J.-L. (2015) "Minimum Word Error Training of RNN-based Voice Activity Detection," in *Interspeech*, pp. 2650–2654.

Gelly, G. and Gauvain, J.-L. (2017) "Spoken Language Identification Using LSTM-Based Angular Proximity," in *Interspeech*, pp. 2566–2570.

Gelly, G. and Vernis, P. (2009) "Neural Networks as a Guidance Solution for Soft-Landing and Aerocapture," in *AIAA Guidance, Navigation, and Control Conference*. Chicago, Illinois.

Gelly, G. *et al.* (2016) "A Divide-and-Conquer Approach for Language Identification Based on Recurrent Neural Networks," in *Interspeech*, pp. 3231–3235.

Goldberg, D.E. (1989) *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

Gu, A. and Dao, T. (2023) "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752* [Preprint].

Haarnoja, T. *et al.* (2018) "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in *International Conference on Machine Learning (ICML)*, pp. 1861–1870.

Hansen, N. and Ostermeier, A. (2001) "Completely Derandomized Self-Adaptation in Evolution Strategies," *Evolutionary Computation*, 9(2), pp. 159–195.

Harpold, J.C. and Graves, C.A. (1979) "Shuttle Entry Guidance," *Journal of the Astronautical Sciences*, 27(3), pp. 239–268.

Hasani, R. *et al.* (2022) "Closed-form Continuous-time Neural Networks," *Nature Machine Intelligence*, 4, pp. 992–1003.

Hochreiter, S. and Schmidhuber, J. (1997) "Long Short-Term Memory," *Neural Computation*, 9(8), pp. 1735–1780.

Kennedy, J. and Eberhart, R. (1995) "Particle Swarm Optimization," in *IEEE International Conference on Neural Networks*, pp. 1942–1948.

Lahoti, A. *et al.* (2026) "Mamba-3: Improved Sequence Modeling using State Space Principles," *arXiv preprint arXiv:2603.15569* [Preprint].

Lu, P. (2014) "Entry Guidance: A Unified Method," *Journal of Guidance, Control, and Dynamics*, 37(3), pp. 713–728.

Lu, P. *et al.* (2015) "Optimal Aerocapture Guidance," *Journal of Guidance, Control, and Dynamics*, 38(4), pp. 553–565.

Matz, D.A. *et al.* (2017) "Application of a Fully Numerical Guidance to Mars Aerocapture," in *AIAA Guidance, Navigation, and Control Conference*.

McKay, M.D., Beckman, R.J. and Conover, W.J. (1979) "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code," *Technometrics*, 21(2), pp. 239–245.

Rataczak, J.A., McMahon, J.W. and Boyd, I.D. (2025) "Convex Predictor–Corrector Aerocapture Guidance," *Journal of Guidance, Control, and Dynamics* [Preprint].

Rockafellar, R.T. and Uryasev, S. (2000) "Optimization of Conditional Value-at-Risk," *Journal of Risk*, 2(3), pp. 21–41.

Schulman, J. *et al.* (2017) "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347* [Preprint].

Sonandres, K.A., Palazzo, T.R. and How, J.P. (2025a) "ABAMGuid+: An Enhanced Aerocapture Guidance Framework Using Augmented Bank Angle Modulation."

Sonandres, K.A., Palazzo, T.R. and How, J.P. (2025b) "Real-Time Density Estimation for Uranus Aerocapture Using Deep Learning," in *AIAA SciTech Forum*.

Storn, R. and Price, K. (1997) "Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces," *Journal of Global Optimization*, 11(4), pp. 341–359.

Sun, J., Feng, B. and Xu, W. (2004) "Particle Swarm Optimization with Particles Having Quantum Behavior," in *IEEE Congress on Evolutionary Computation (CEC)*, pp. 325–331.

Vaswani, A. *et al.* (2017) "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30 (NIPS)*, pp. 5998–6008.

Vernis, P. *et al.* (2004) "Guidance Trade-Off for Aerocapture Missions," in *ESA Guidance, Navigation and Control Conference*.

Zucchelli, E.M. *et al.* (2021) "Two Stage Optimization for Aerocapture Guidance," in *AIAA SciTech Forum*.

Appendix A: reproduction details

One compact reference for the settings and systems behind every number.

Simulator. All schemes fly through one native (Rust) simulator: fixed-step fourth-order Runge–Kutta integration (Gill variant), J_2–J_4 zonal gravity, a tabulated Mars atmosphere carrying the static Monte Carlo bias and the Ornstein–Uhlenbeck perturbation, pilot dynamics, thermal tracking, and the navigation–guidance–control chain sequenced on its own cadences; the bias-filter navigation recovers density through lift-corrected inverse dynamics. The implementation is regression-validated against the 2009 study's legacy code – across all 725 time steps of a guided trajectory, 22 of 24 output channels are bit-identical (the two mismatches trace to uninitialized variables in the reference) – which establishes equivalence with the flight-heritage implementation, not physical validation. Independent physics checks run in the test suite: an analytic potential-gradient oracle for the J_2–J_4 gravity expansion, integrator convergence on analytic systems, vacuum energy and angular-momentum conservation, and cross-language forward-pass parity for the network runtime at machine epsilon.

Seed pools. Every Monte Carlo pool derives from the mission base seed through disjoint reserved streams: the selection pool (offset 10^6, $n = 1000$, the in-training promotion gate – queried $13,442$ times over the headline run, so it is adaptively reused and is not an unbiased estimate of generalization), final evaluation (offset 2×10^6: the $n = 1000$ paired pool of Table 4 and Table 5, extended to $n = 10,000$ for the development far tail), the fresh re-quote pool (offset 8×10^6), the off-nominal stress pool (offset 9×10^6), and the architecture-probe pool (offset 10^7, Appendix B). Training scenarios are drawn outside every reserved stream; evaluation pools draw one scenario per seed (independent samples), and multi-scenario batches use Latin-hypercube draws. Table 3 states which decisions each pool influenced.

Confirmatory sizing pool. All methodology, architecture, and checkpoint choices were frozen at a recorded revision before the pool was generated. Seeds are drawn without duplicates from $[2^{31}, 2^{32})$ – structurally disjoint from every historical draw, since all earlier pools, training batches, and curation probes live in $[0, 2^{31})$ – arranged as ten replicate pools of $100,000$ scenarios sharing seeds across schemes (so per-replicate differences are paired). Every cell is evaluated on it exactly once; tail statistics use the estimator of Section 2.2 ($\mathrm{CVaR}_{99.9}$ averages 100 observations per replicate, 1000 pooled), and replicate-level dispersion gives the quoted standard errors and difference intervals with no distributional assumptions.

Off-nominal stress regime. The Section 7 stress pool raises four domains to their high presets: the atmospheric density bias to $\pm 100\%$ (from $\pm 50\%$); the Gauss–Markov density perturbation to a 30 s correlation time at 20% RMS (from 120 s at 5%); navigation errors to 3σ altitude ± 3 km, horizontal $\approx \pm 18$ km per axis, velocity ± 3 m/s, drag acceleration ± 0.6 m/s²; and the navigation-filter gain draw to ± 0.45 absolute (3σ).

Dispersion rationale. The $\pm 50\%$ nominal density span is a deliberately conservative envelope for Mars seasonal and dust-loading variability about a tabulated mean profile (Forget *et al.*, 1999); domains are drawn independently as a modeling choice (no cross-correlations are claimed). The static bias and the Ornstein–Uhlenbeck process model different frequencies – profile-scale uncertainty versus along-track variability – and do not double-count; the process is zero-initialized at entry and reaches its stationary variance within roughly one correlation time. The two wind draw dimensions are retained inert to keep the 26-dimensional draw layout stable across configurations with and without the wind model.

Classical tuning parity. Every classical scheme was tuned by the same genetic algorithm on the same cost C above – identical penalties, virtual costs, and transform – co-optimizing its full parameter set including the shared navigation-filter and actuator gains the network also tunes (26 parameters for FTC against the network's weights plus 3 actuator-side parameters), at 2000 generations $\times$ 300 individuals $\times$ 10 scenarios per generation. The classical searches plateau far inside that budget (FNPAG's best individual stopped improving before generation 60), so the network's longer run buys the comparison nothing. The joint-reference co-optimization of Section 7.1 is the classical counterpart of the network's co-tuned actuator parameters, and the information asymmetry is tested directly: retrained without the three predicted-Δv observations the classical schemes cannot consume, the network still beats the co-tuned joint-FTC on every reported statistic (Section 6.3, third control).

Correction burns. The reported Δv is the three-burn plan from the captured orbit (apsis radii r_a, r_p) to the 500 km circular parking orbit (radius r_c), using the elliptical apsis speed $v(r_1, r_2) = \sqrt{2\mu r_2/(r_1(r_1 + r_2))}$: a periapsis correction at apoapsis, $\Delta v_1 = v(r_a, r_c) - v\big(r_a, r_p\big)$; circularization at the new periapsis, $\Delta v_2 = \sqrt{\mu/r_c} - v(r_c, r_a)$; and a plane change $\Delta v_3 = 2v_n \sin(\Delta i/2)$ at the cheaper node, v_n the smaller of the target-orbit speeds at the two nodes. The total is $|\Delta v_1| + |\Delta v_2| + |\Delta v_3|$.

Cost and objective. Per simulation the cost is

$$C = \Delta v + s(\Delta v - T) + \frac{s(\Delta v - T)^2}{2S} + \sum_j w_j \, s\big(\big(x_j - L_j\big)/L_j\big), \tag{8}$$

with s a softplus (a C^∞ knee), $T = 1000$ m/s, S the quadratic scale, and the penalty sum over the peak heat-flux, peak-load, and integrated heat-load exceedances normalized by their limits L_j, weights $w_j = 1$. Non-captures receive a virtual cost above any captured correction: $10,000 + v_\infty$ m/s for hyperbolic escape (v_∞ the excess speed), and $3000 + 1000 \min\big(|E - E_{\text{target}}|, 50\big) - 500\, t/t_{\max}$ for crash and timeout outcomes (energies in MJ/kg) – so captures are always preferred while the optimizer keeps a gradient toward the capture boundary. The per-simulation cost is cubed (the deployed transform) and aggregated across an individual's scenario batch by root-mean-square – the L_6-norm-equivalent objective of Equation 6.

Optimizer (deployed headline). pymoo genetic algorithm (SBX crossover $\eta = 3$, polynomial mutation $\eta = 5$ at rate 0.15), population 512, two scenarios per individual per generation, run to plateau ($15,000$–$20,000$ generations). The adaptive seed curation, precisely: every second generation or on a promotion, (1) draw 1000 fresh probe scenarios outside every reserved stream; (2) score them with the current best individual; (3) sort the per-scenario costs into n_{sims} equal-count quantile bins; (4) take the hardest scenario of each bin as the next training batch – scenarios are replaced as a set, never accumulated. The selection gate re-runs each new argmin on the reserved $n = 1000$ selection pool and promotes on strict RMS improvement.

Training harness. Population evaluation is batched through in-process Python bindings to the native core: each generation's individuals are simulated scenario-parallel across CPU cores with the interpreter lock released, network weights pass in memory rather than through files, and the dispersion-independent tables (atmosphere, winds, reference trajectories) are shared read-only across the batch. At the measured single-core cost of Section 7.2, the headline run's $\approx 2 \times 10^7$ dispersed passes amount to roughly twenty core-hours – an overnight training on a laptop.

Deployed architecture. Dense($17 \to 16$, swish) $\to$ Mamba($d_{\text{inner}} = 16$, $d_{\text{state}} = 12$) $\to$ Dense($16 \to 2$, asinh), atan2 bank decoder; 962 trainable parameters. The input mask selects 17 of the 35 candidate observations (indices 0, 2, 3, 5, 6, 7, 11, 12, 18, 19, 27–30, 32–34: instantaneous orbital/aerodynamic state, two reference-trajectory interpolations, the seam-free bank-history pairs, and the three predicted correction-Δv components). Each input is normalized by a calibrated per-input affine or asinh transform mapping its $[p_5, p_{95}]$ span to $[-1, 1]$. Three actuator-side parameters – the navigation density-filter gain and its rate limit, and the command-shaping acceleration limit – are co-optimized with the network weights.

FNPAG. Two-second replan cycle; bisection corrector on the constant capture-bank magnitude (about eleven forward integrations per replan); forward-predictor integration step GA-tuned at 3.8 s; the onboard atmosphere is scaled by the navigation-estimated density factor so the predictor tracks the measured atmosphere.

One runtime, training to flight. The policy that flies is the artifact that trained: candidates are evaluated, selected, and deployed through the same native forward pass, so no export or re-implementation step separates the training loop from the flight code. An independent PyTorch implementation of every cell type (used by the supervised warm-start path, not by the deployed runs) is held to numerical agreement with the native runtime by cross-language tests – maximum absolute forward-pass difference near machine epsilon (10^{-16}–10^{-14}) over 100-step stateful sequences.

Timing. Wall-clock per complete simulation over 200 sequential runs of each deployed scheme on one idle core of an Apple M4 Pro laptop (native code compiled with rustc 1.97 at the release profile with link-time optimization; 64 -bit floats throughout; single-threaded). Quoted values are the median of five timed batch repeats after one warm-up; repeat spreads are 0.1–2.2% of the median. A flown simulation spans $\approx$ 480–770 guidance updates depending on the scheme, giving the per-update and per-replan shares quoted in Section 7.2 (whole-simulation cost divided by update count – a loose upper bound on pure guidance inference). These are implementation-and-host measure-

ments supporting the relative comparison; flight-processor worst-case execution time and memory footprint are not established here (Section 9).

Artifacts. The simulator, training harness, analysis code, every configuration, the deployed network weights, the committed per-run evaluation records behind each table, and the scripts that regenerate every figure and number are released publicly under an MIT license at https://github.com/GovitPowerz/Aerocapture. Every number in the tables regenerates, without retraining, from those records.

Appendix B: architecture probes – CfC, xLSTM, and the Mamba-3 axes

A controlled negative result supporting the Section 6 headline: does any recent recurrent family beat the plain selective SSM on the sizing tail? Three probes, each anchored on a Section 6 cell at matched parameter budget: a closed-form continuous-time cell (CfC (Hasani *et al.*, 2022), hypothesis: input-dependent time constants suit the fast-near-periapsis, static-in-vacuum phase structure) against the GRU anchor; the exponential-gated sLSTM and matrix-memory mLSTM of xLSTM (Beck *et al.*, 2024) (hypothesis: sharp revision of a stored estimate at the bounce or a density shock) against the LSTM anchor; and a 2×2 over Mamba-3's (Lahoti *et al.*, 2026) two axes – exponential-trapezoidal discretization and complex (rotational) state – at the deployed Mamba anchor, whose Euler-real arm is bit-identical to the deployed Mamba layer.

Every arm shares one training regime (the genetic algorithm at population 300 for 5000 generations, two scenarios per individual, adaptive hardest-seed curation, live actuator scaffolding) and one reserved evaluation pool, the probe pool (offset 10^7, $n = 1000$; each arm scored with its co-trained scaffolding), with three seed-repeats per arm; σ_{run} is the standard deviation over repeats. Capture is 99.97–100% everywhere and every arm passes the feasibility check of Section 6.2 – zero heat-flux, g-load, and heat-load violations on every repeat – so the comparison is a pure tail-Δv story. Two caveats apply throughout: these are p_{95}/CVaR_{95} statistics at $n = 1000$, not the far-tail $\text{CVaR}_{99.9}$ sizing metric, and the probe budget is deliberately sub-headline (1.5M evaluations versus the deployed $512 \times 20,000$), so the absolute values sit above the headline numbers and are not mission figures.

Table 6: The nine probe arms: three-seed means, correction Δv in m/s on the shared $n = 1000$ probe pool. Baselines are retrained in-regime (not the higher-budget reference runs – see the budget caveat below). Viol. is the any-constraint violation rate; every arm is feasible on every repeat.

Arm	Params	Capture %	Viol. %	p_{50}	$p_{95} \pm \sigma_{\text{run}}$	$\text{CVaR}_{95} \pm \sigma_{\text{run}}$
Mamba (baseline)	962	99.97	0.0	114.0	121.6 ± 0.5	124.1 ± 0.3
Mamba-3 trapezoidal	978	100.0	0.0	115.5	124.9 ± 2.1	128.8 ± 2.3
Mamba-3 complex	1154	100.0	0.0	113.7	121.1 ± 1.8	123.8 ± 2.1
Mamba-3 both	1170	100.0	0.0	114.0	121.6 ± 1.3	124.2 ± 1.2
GRU (baseline)	1014	100.0	0.0	114.7	123.7 ± 1.5	126.7 ± 1.3
CfC	1003	100.0	0.0	116.5	126.5 ± 0.7	130.4 ± 0.1
LSTM (baseline)	1082	100.0	0.0	115.4	124.3 ± 1.4	127.3 ± 1.4
sLSTM	1042	100.0	0.0	115.7	124.7 ± 2.6	127.6 ± 3.1
mLSTM	1078	100.0	0.0	118.1	127.4 ± 3.5	130.8 ± 4.8

Table 7: Within-family significance: gap = treatment minus baseline (positive = worse), cleared when $|\text{gap}| > \sigma_{\text{comb}} = \sqrt{\sigma_{\text{base}}^2 + \sigma_{\text{arm}}^2}$, with the per-arm σ_{run} taken from Table 6.

Treatment vs baseline	Metric	Gap	σ_{comb}	Verdict
CfC vs GRU	p_{95} / CVaR_{95}	$+2.8$ / $+3.7$	1.6 / 1.3	**significantly worse**
Trapezoidal vs Mamba	p_{95} / CVaR_{95}	$+3.3$ / $+4.8$	2.2 / 2.3	**significantly worse**
Complex vs Mamba	p_{95} / CVaR_{95}	-0.5 / -0.3	1.9 / 2.1	within run variance
Both vs Mamba	p_{95} / CVaR_{95}	$+0.0$ / $+0.1$	1.4 / 1.2	within run variance
sLSTM vs LSTM	p_{95} / CVaR_{95}	$+0.4$ / $+0.3$	2.9 / 3.4	within run variance
mLSTM vs LSTM	p_{95} / CVaR_{95}	$+3.1$ / $+3.5$	3.8 / 5.0	within run variance (high σ_{run})

The within-family rows are the rigorous claims. The CfC is significantly worse than the GRU on both tail statistics, and its tiny σ_{run} (0.1 on CVaR_{95}) makes the loss stable and reproducible, not a bad-luck draw: continuous-time time constants add a harder optimization landscape for a timescale adaptation the fixed-cadence gates already learn.

Mamba-3′s trapezoidal discretization is significantly worse than the plain cell – the seed-repeats upgrade an earlier single-run "no benefit" to a measured degradation. Complex state, both axes combined, the sLSTM, and the mLSTM all sit within run variance of their baselines; the mLSTM is notably high-variance (σ_{run} up to 4.8) and trends worse without clearing the bar – matrix memory buys instability, not tail robustness. Cross-family, all nine arms share the regime and pool but the anchors differ slightly (962/1014/1082 parameters), so the ranking is suggestive rather than matched: the plain Mamba tops the field (tied with its own complex arms), about 2 m/s ahead of the GRU and 2.7 ahead of the LSTM at p_{95} – consistent with the deployed headline.

One caveat is load-bearing. Each probe also scored a higher-budget reference row – the Section 6 sweep cell for the GRU and the LSTM (population 512 for the same 5000 generations, 1.7 × the probe evaluations) and a full-budget plain-Mamba run ($512 \times 10,000$, 3.4 ×) for the Mamba – and those sit 4–6.5 m/s better at p_{95} than the retrained in-regime baselines (Mamba 121.6 versus 116.6; GRU 123.7 versus 117.3; LSTM 124.3 versus 120.2): a training-budget effect, not architecture. That gap is exactly why every treatment compares against its retrained in-regime baseline and never against a higher-budget reference. Two smaller caveats: the probe cells are trained through the gradient-free path only (no warm-start), and the sLSTM's 40-parameter deficit against the LSTM is a cell-definition cost (single bias), not a budget mismatch – it does not explain its null.

The consistent null across three independent families points at the task, not the cells. A single atmospheric pass is a few hundred guidance ticks whose latent state worth remembering is a handful of slowly-varying dispersion parameters – density bias, the Ornstein–Uhlenbeck perturbation state, aerodynamic dispersions – and the engineered inputs already carry most of the temporal signal (Section 8). A diagonal selective-SSM state of dimension 12–16 saturates what little internal memory the problem rewards; the CfC's timescale adaptation, the xLSTM's revision and associative recall, and Mamba-3′s long-context and state-tracking axes all target capacity this smooth, low-bandwidth control signal never exercises. The deployed cell wins not by more sophisticated memory but by just enough memory, cheaply. Framed positively, the probes validate the methodology: the adaptive-seed, tailled, matched-anchor protocol distinguishes between architectures rather than rubber-stamping the newest one – three 2024–2026 recurrent families tried, none better, and two arms (the CfC and the trapezoidal discretization) significantly worse.

Appendix C: quantizing the deployed Mamba head

Can the deployed 962-parameter policy shrink for flight memory without giving back the tail it was selected for? This appendix quantizes the deployed head: weight-only, symmetric, fake-quantized. Rounded values are stored back as 64-bit floats, so the runtime, the regression goldens, and the simulation are untouched by construction. A post-training quantization (PTQ) grid crosses bit width (8/6/4/3/2), scale granularity (per-channel, per-tensor), and tensor policy (all tensors, or projections only, keeping the SSM dynamics and bias scalars in floating point); two quantization-aware training (QAT) arms then retrain at the best 4-bit cell of the grid. Selection ran on the fresh re-quote pool of Appendix A ($n = 1000$ per grid cell); the finalists below are quoted once on the frozen confirmatory pool of Section 7 ($10 \times 100,000$ scenarios each), preserving the paper's selection-versus-quote discipline.

The grid's shape is simple (Figure 19). Eight-bit per-channel quantization is free (+0.4 m/s at CVaR_{95} even with every tensor quantized); degradation is visible from 6 bits and catastrophic below 4 (2-bit collapses capture entirely). At 4 bits – the target compression – the tensor policy dominates: projections-only holds 100% capture where all-tensors falls to 77–85%, a split far above the single-evaluation noise floor. The few-m/s non-monotone granularity cells at 6 and 3 bits are within that noise floor and should not be over-read. Among the 4-bit cells the selection rule – highest capture rate, then lowest CVaR_{95} – picks per-channel scales, projections only.

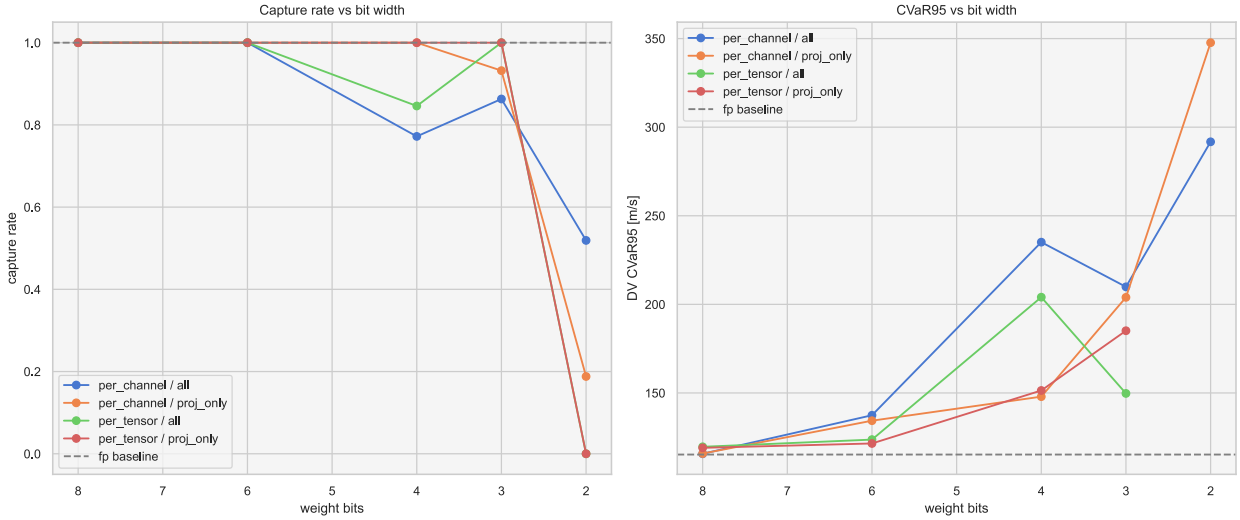


Figure 19: The post-training quantization grid on the re-quote pool ($n = 1000$ per cell): capture rate (left) and CVaR_{95} (right) versus bit width, for the four granularity × policy series.

Table 8: Quantization finalists on the frozen confirmatory pool ($10 \times 100,000$ scenarios each; correction Δv in m/s; replicate standard errors). All four variants capture every scenario with zero constraint violations. The 4-bit cells use per-channel scales on the projection tensors only.

Variant	Capture %	p_{50}	CVaR_{95}	$\mathrm{CVaR}_{99.9} \pm \mathrm{SE}$
Deployed policy (64-bit floats)	100.0	109.7	115.9	123.3 ± 0.1
PTQ, best 4-bit cell	100.0	126.7	149.5	176.4 ± 0.4
QAT fine-tune, 4-bit	100.0	110.2	116.6	122.8 ± 0.1
QAT from scratch, 4-bit	100.0	112.4	124.4	140.7 ± 0.3

The headline is the fine-tuned arm. PTQ at the best 4-bit cell costs +33 m/s at CVaR_{95} on the re-quote pool and degrades further with depth ($\mathrm{CVaR}_{99.9}$ 176.4 – the same fatten-with-depth signature Section 7.2 measured for FNPAG). Quantization-aware fine-tuning recovers the entire gap. The champion checkpoint is resumed for 3000 generations with every candidate's weights fake-quantized before each fitness evaluation, so the genetic search

optimizes the quantized policy directly; the result is $\mathrm{CVaR}_{99.9} = 122.8 \pm 0.1$ against the deployed policy's 123.3 ± 0.1, a paired replicate delta of -0.46 $[-0.74, -0.18]$, at 100% capture and zero violations. The 4-bit head is tail-equivalent at full sizing depth (the point estimate is in fact lower, but the fine-tune's extra generations bias the comparison in its favor, so we claim equivalence rather than improvement), while the shallow tail pays $+0.7$ m/s at CVaR_{95} – within the run-to-run scatter of the Appendix B probe repeats. Training the same 4-bit constraint from scratch at the champion's full $512 \times 20,000$ budget reaches a competent policy (100% capture, $\mathrm{CVaR}_{99.9}$ 140.7 – the dense family's territory) but not the champion's basin: fine-tuning is the better path, and scratch's validation RMS agrees (1.457×10^6 against the fine-tune's 1.350×10^6).

Which parts of a selective SSM tolerate 4 bits? A one-tensor-at-a-time probe quantizes a single tensor group at 4 bits per-channel and keeps everything else in floating point (re-quote pool, $n = 1000$ per cell):

Table 9: One-tensor-at-a-time sensitivity at 4 bits per-channel ($n = 1000$ per cell, re-quote pool). The SSM dynamics parameters are disproportionately sensitive. The deltas do not add up to the all-tensors per-channel cell (+119.9; Δv statistics are conditional on capture, and that cell captures only 77%) – quantization errors interact.

Tensor group	**Params**	**Capture %**	$\mathbf{\Delta CVaR_{95}}$
input dense W	272	100.0	$+59.0$
SSM input projection $W_{x,\mathrm{proj}}$	400	100.0	$+15.8$
SSM Δ-projection W_Δ	16	100.0	$+0.0$
SSM dynamics $a_{\log}$	192	100.0	$+53.0$
SSM residual gains d_{skip}	16	100.0	$+47.4$
output dense W	32	100.0	$+22.5$

The SSM dynamics parameters are the bottleneck. The state matrix parameters $a_{\log}$ cost $+53.0$ m/s – they are exponentiated at runtime ($A = -\exp(a_{\log})$) and one-sided, so a symmetric grid wastes half its levels – and the per-channel residual gains d_{skip} cost $+47.4$ from just sixteen scalars. The Δ-projection is exactly lossless by construction (per-channel scaling of a one-column matrix assigns one scale per element). This is the mechanism behind the projections-only policy – quantize the 720 projection weights, keep the 242 dynamics and bias scalars in floating point – and it is the appendix's SSM-specific finding: the outlier sensitivity of selective-SSM dynamics reported in the language-model quantization literature reproduces on a closed-loop control task under a tail-led metric. Whether $a_{\log}$'s sensitivity is representational (the symmetric grid) or intrinsic is left open; an asymmetric quantizer is the one-line extension that would answer it.

The deployment benefit, stated honestly, is memory – not compute:

Table 10: Analytic memory footprint (packed b-bit weights + 32-bit scales + floating-point remainder; 64-bit baseline 7696 B). The per-tensor projections cell is cheaper than the deployed cell and holds capture, but pays $+3.5$ m/s CVaR_{95} over it on the grid. The 624 B all-tensors cell is shown for contrast only – it is accuracy-broken at 4 bits (capture 85%). The honest headline is the deployed cell, 7696 → 1564 B, bounded below by the 968 B of dynamics parameters the sensitivity study says must stay in floating point.

Cell	**Packed**	**Scales**	**Float**	**Total**	**vs 64-bit**
8-bit per-channel, projections	720 B	236 B	968 B	1924 B	$4.0\times$
4-bit per-channel, projections (deployed QAT cell)	360 B	236 B	968 B	**1564 B**	$\mathbf{4.9}\times$
4-bit per-tensor, projections	360 B	16 B	968 B	1344 B	$5.7\times$
4-bit per-tensor, all tensors	464 B	24 B	136 B	624 B	$12.3\times$

On compute, the head accounts for ≈ 1.22 ms of the deployed policy's 3.14 ms whole-simulation cost (Section 7.2):

Table 11: Forward-pass micro-benchmark (median ns per guidance tick, laptop-class core, scalar code, the SSM recurrence in floating point for every variant; ms per simulation over the measured 644 forward passes of a flown trajectory). Quantization is not a compute win at this scale.

Forward kernel	**ns / tick**	**vs deployed**	**ms / simulation**
64-bit (deployed runtime)	1888	–	1.22
64-bit hand-rolled	1670	-11.6%	1.08
32-bit hand-rolled	1290	-31.7%	0.83
int8 weights, int8 activations	1735	-8.1%	1.12
packed int4 weights, int8 activations	1747	-7.5%	1.13

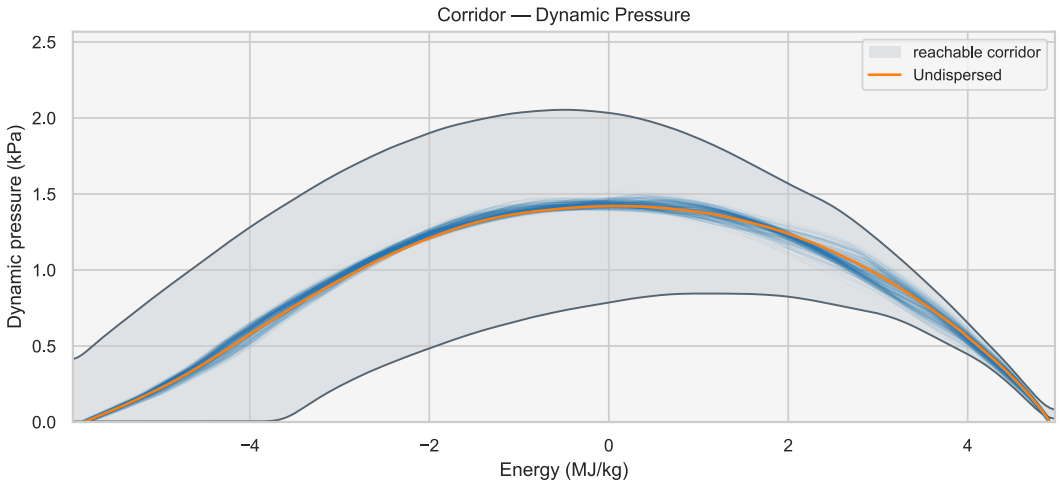
Single precision is the compute sweet spot (-32% per tick); the integer kernels beat the deployed runtime by only $\approx 8\%$, because dynamic activation quantization plus the floating-point SSM recurrence (192 exponentials per tick) dominate a 962-parameter workload, and 4-bit nibble unpacking cancels its bandwidth advantage. Simulation throughput in the accuracy study is unchanged by construction, so no simulation-time claim is made. The deployment case for the 4-bit head is the $4.9\times$ memory footprint and fixed-point-capable projection arithmetic, not speed.

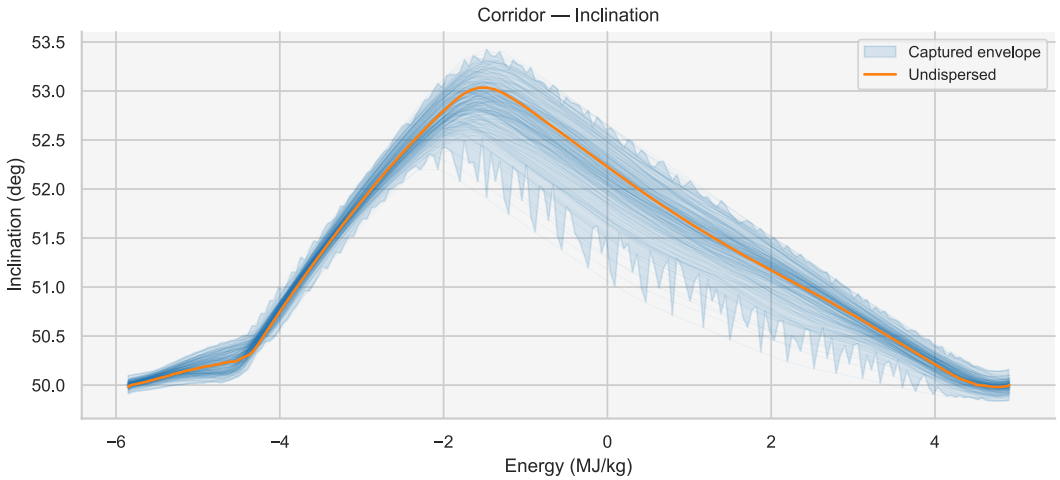
Four caveats bound these claims. The accuracy study is weight-only: activations, the hidden state, and the input normalization stay in 64-bit floats (the integer kernels quantize activations for the compute measurement only). Both QAT arms are single runs. The grid cells are single-model $n = 1000$ evaluations. And the kernel ratios are laptop-class scalar measurements that do not transfer to flight processors – the memory table does.

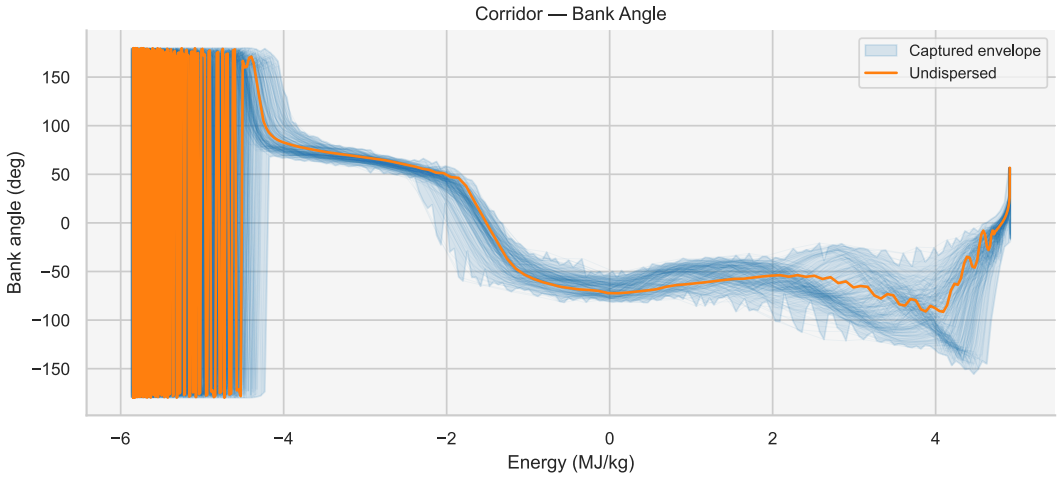
Appendix D: per-scheme mission reports

Each scheme below gets a two-page mission-performance card on the final-evaluation Monte Carlo pool ($n = 1000$), pinned to its deployed policy so the statistics reproduce Table 4; the FTC, PredGuid, and energy-controller cards use their co-optimized-reference variants (Section 7.1), matching those rows of Table 4. The first page shows the corridor behaviour – the classified trajectory ensemble in the (energy, dynamic pressure), (energy, inclination), and (energy, bank) planes, with the undispersed nominal overlaid; the dynamic-pressure panel sets the ensemble against the shared occupancy envelope of Figure 1, so one can read at a glance whether each scheme flies inside it. The second page shows the correction-Δv distribution (total and its three burns), the thermal and load-constraint margins, and the full statistics. Panels reuse the training pipeline's own report charts; captured trajectories are blue, constraint violations orange, failures red.

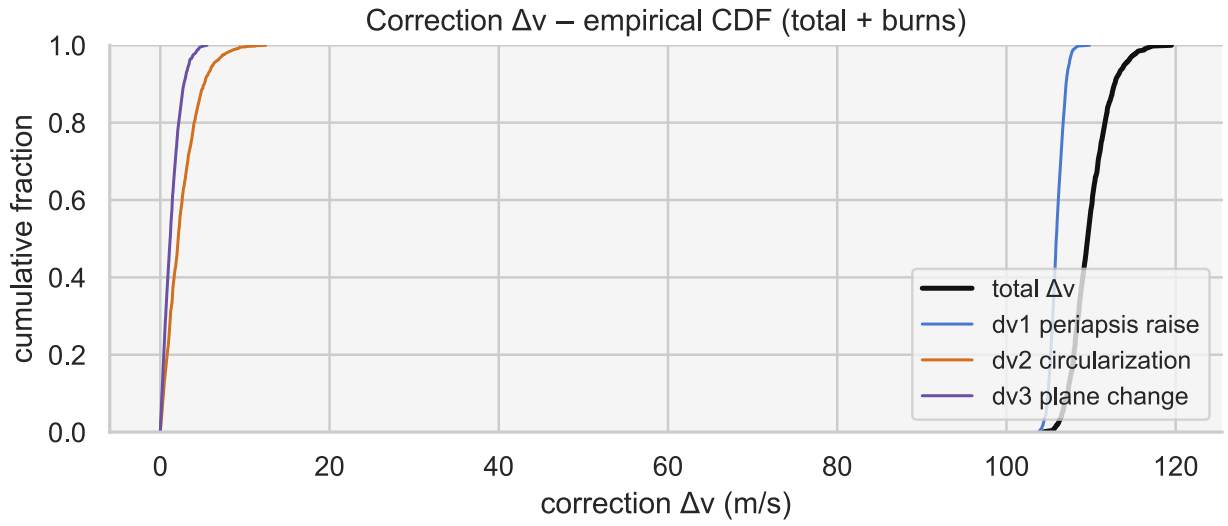
NN -- Mamba (962 params)

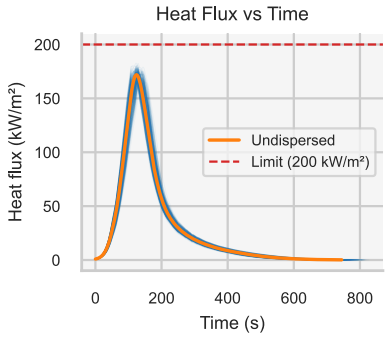


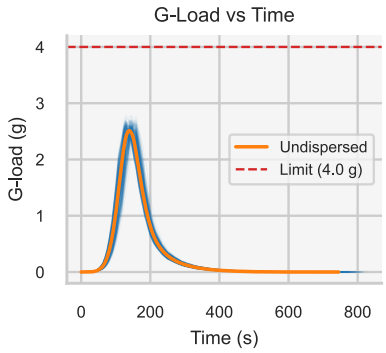


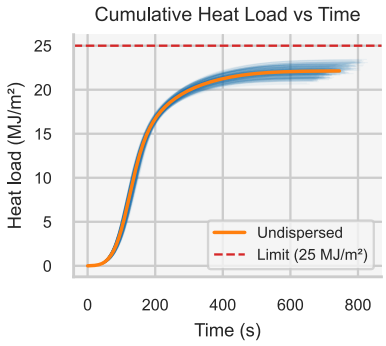


NN -- Mamba (962 params) (continued)



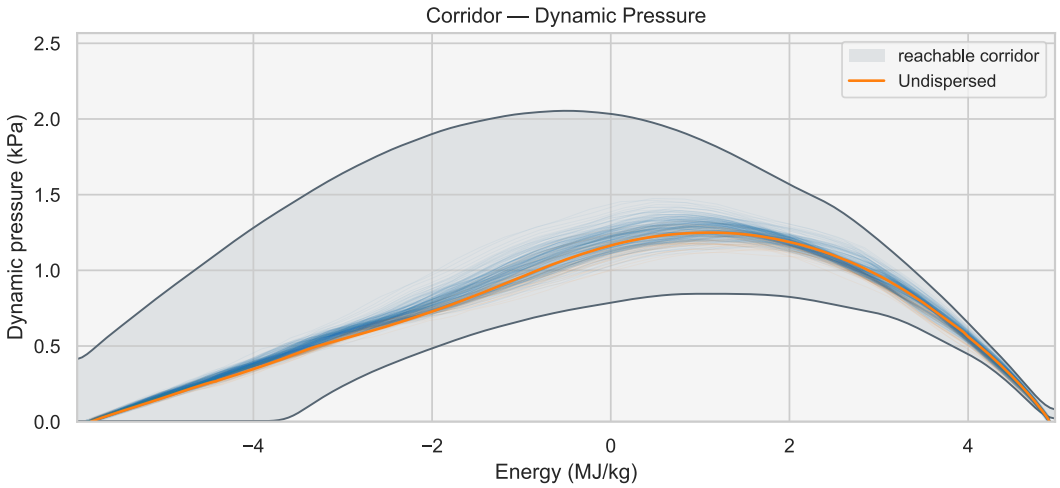


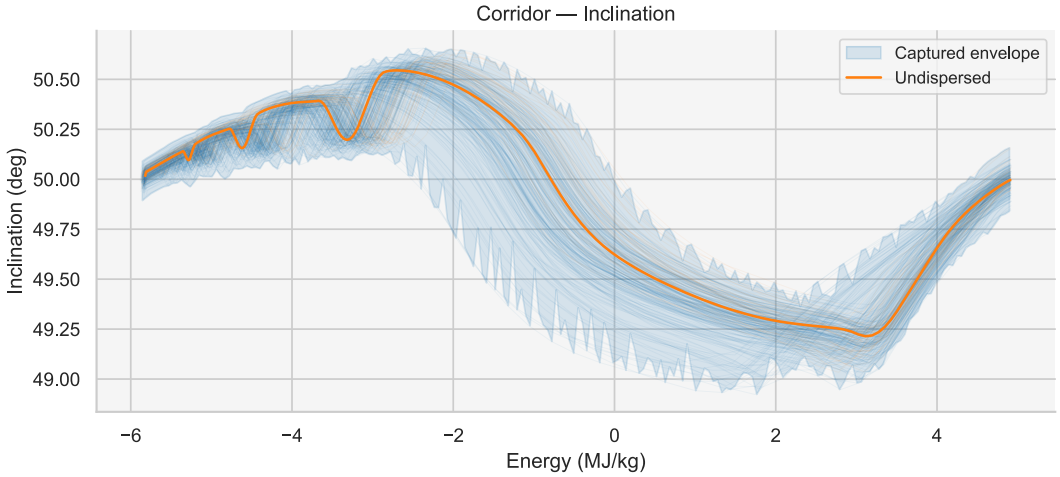


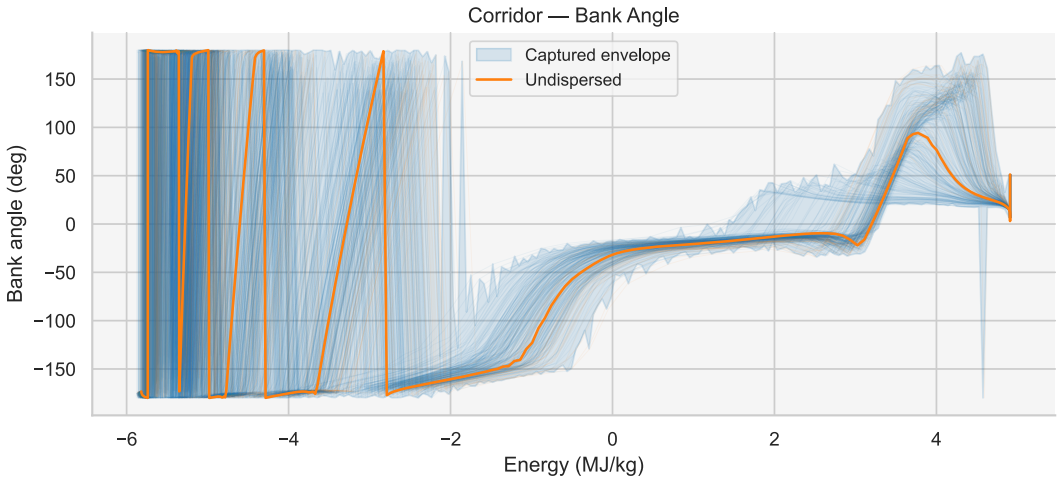


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	109.6	114	109.8	mean
Δv CVaR95 / p99 / max	115.4	116.3	119.5	tail
dv1 periapsis raise	105.9	107.4	109.8	m/s
dv2 circularization	2.1	6.3	12.5	m/s
dv3 plane change	1.2	3.4	5.5	m/s
Apoapsis error (km)	0.8	22.3	0.3	mean
Periapsis error (km)	28.2	33.5	28	mean
Inclination error (deg)	−0.01	0.04	−0.01	mean
Heat flux (kW/m²)	172.5	178.7	182.6	viol 0%
G-load (g)	2.57	2.69	2.81	viol 0%
Heat load (MJ/m²)	22.1	23.1	23.7	viol 0%
Capture rate				100%

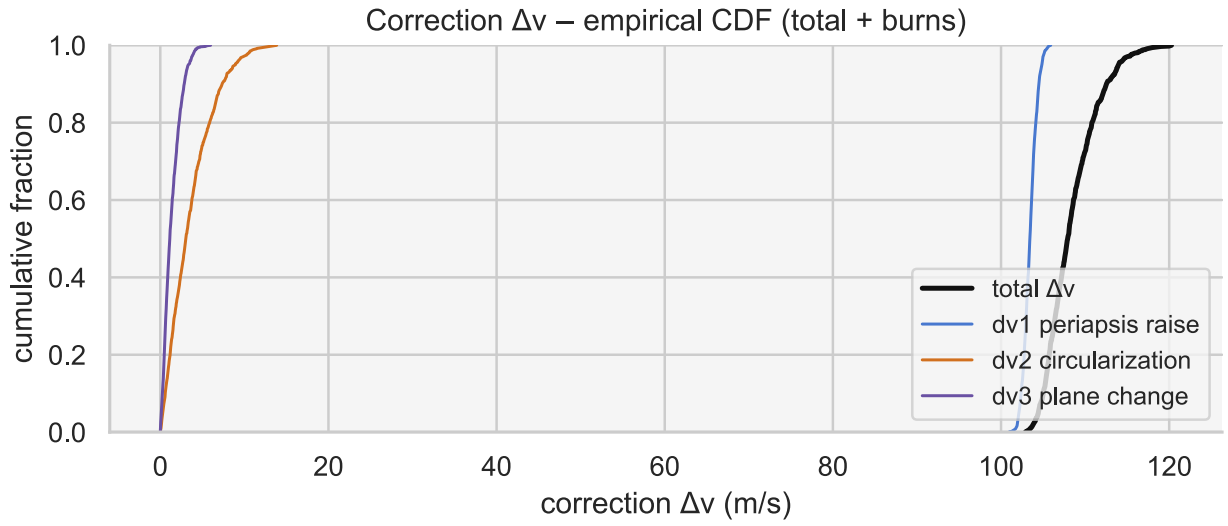
NN -- LSTM (1082 params)

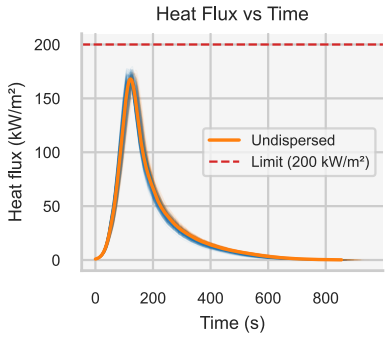


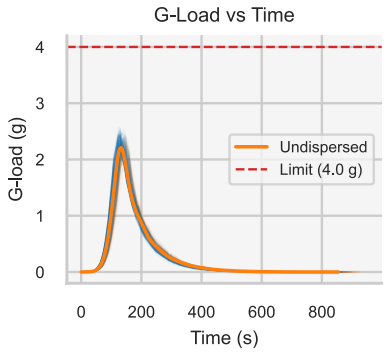


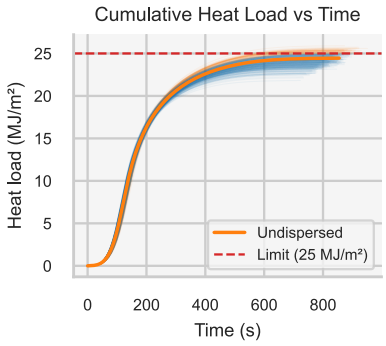


NN -- LSTM (1082 params) (continued)



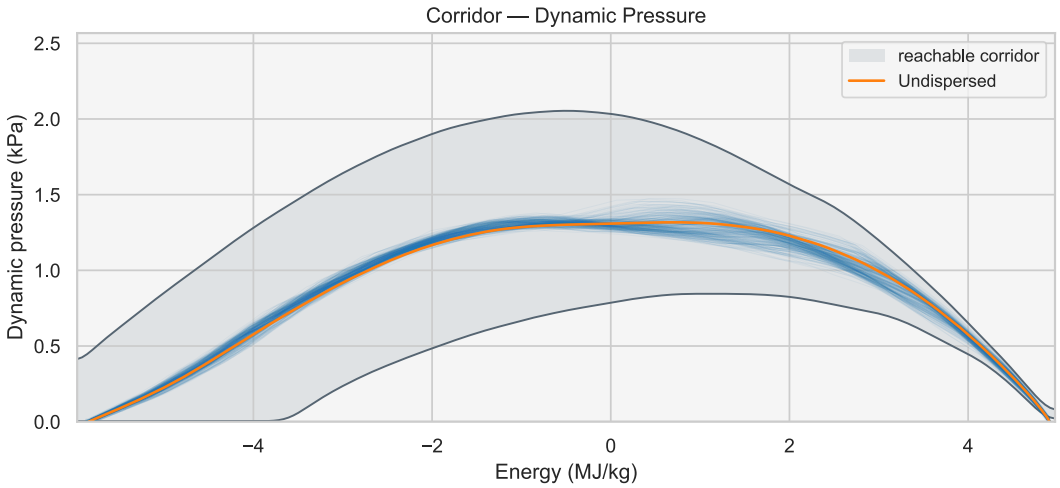


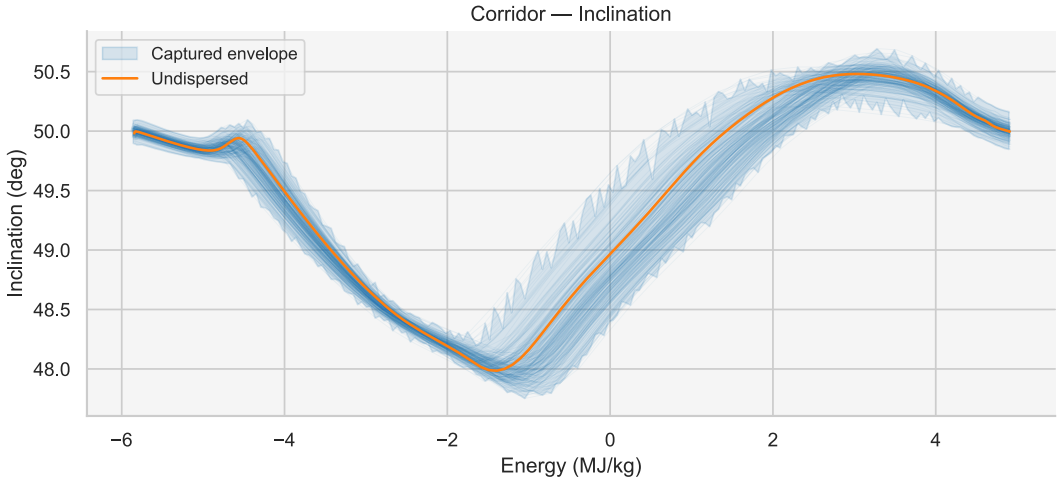


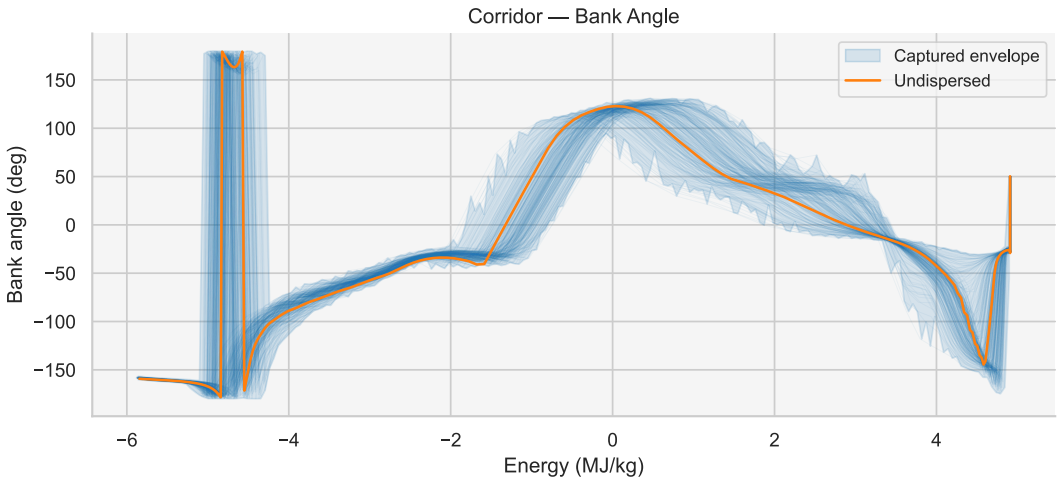


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	107.9	114	108.4	mean
Δv CVaR95 / p99 / max	116	117.2	120.3	tail
dv1 periapsis raise	103.4	104.9	105.9	m/s
dv2 circularization	3	9	13.8	m/s
dv3 plane change	1.1	3.3	6	m/s
Apoapsis error (km)	−1.2	33.8	−1.4	mean
Periapsis error (km)	38.3	43.2	38.2	mean
Inclination error (deg)	0	0.05	0	mean
Heat flux (kW/m²)	168.6	176.3	181.9	viol 0%
G-load (g)	2.27	2.47	2.69	viol 0%
Heat load (MJ/m²)	24.2	25.4	26.7	viol 15.6%
Capture rate				100%

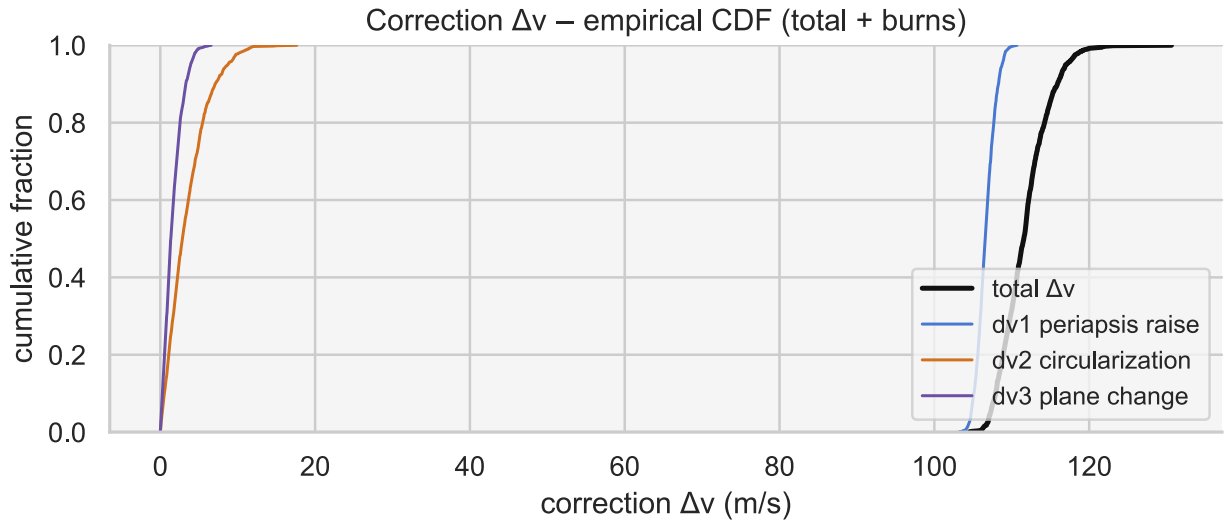
NN -- GRU (1014 params)

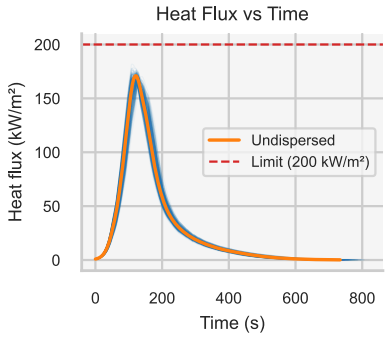


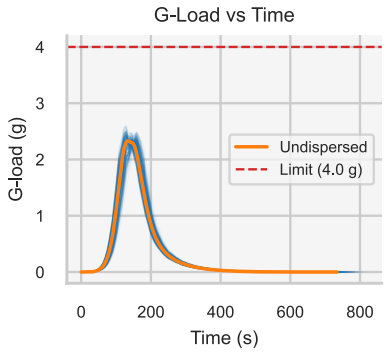


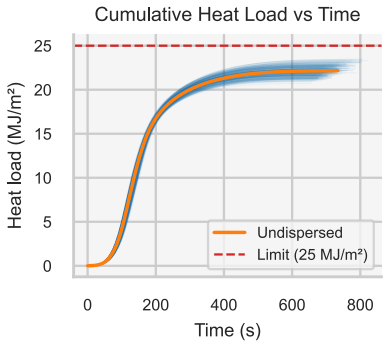


NN -- GRU (1014 params) (continued)



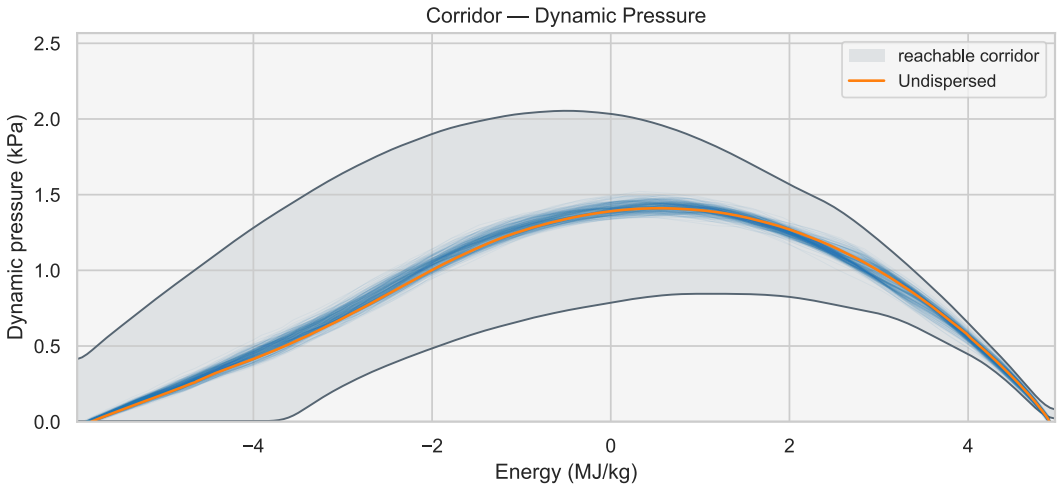


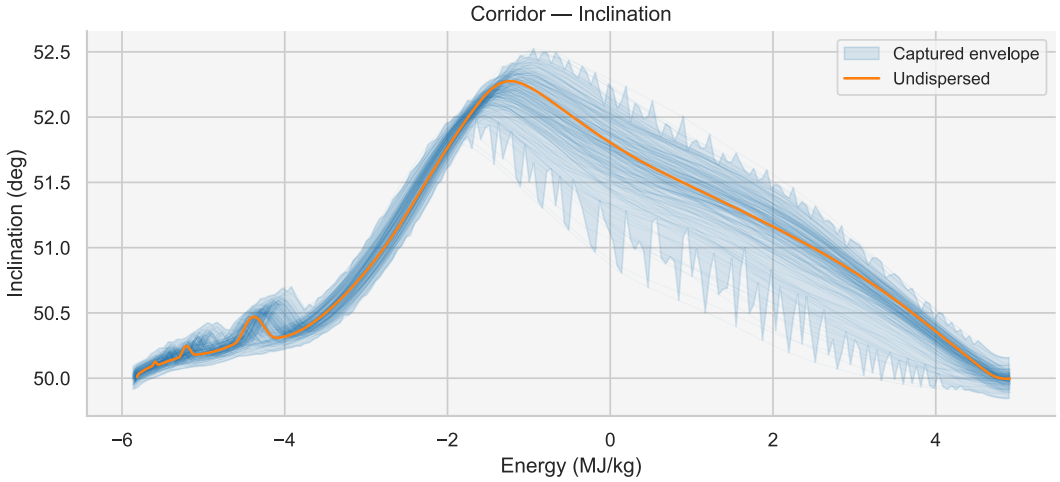


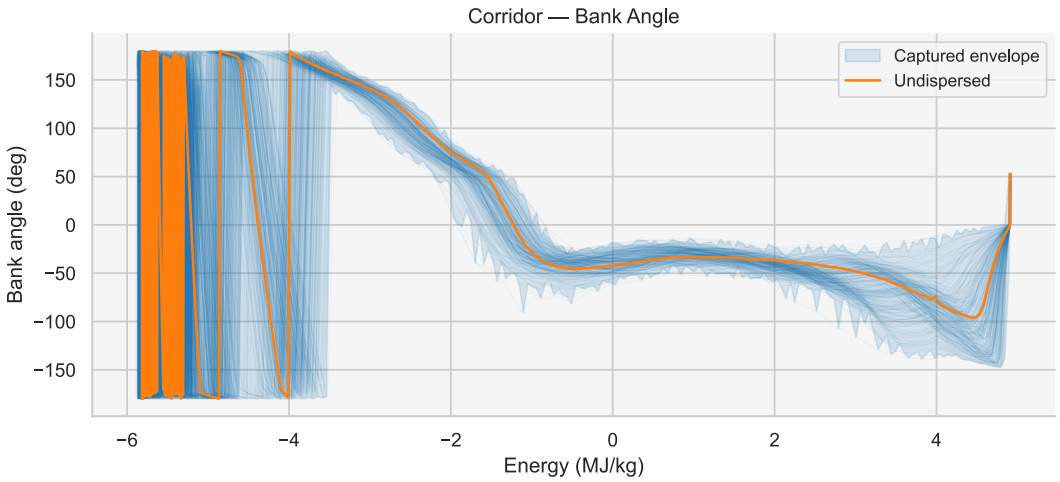


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	111.6	117	111.7	mean
Δv CVaR95 / p99 / max	119	119.8	130.6	tail
dv1 periapsis raise	106.5	108.8	110.7	m/s
dv2 circularization	2.8	8.7	17.6	m/s
dv3 plane change	1.4	3.9	6.6	m/s
Apoapsis error (km)	1.4	37.6	2	mean
Periapsis error (km)	25.6	32.6	25.3	mean
Inclination error (deg)	−0.02	0.04	−0.01	mean
Heat flux (kW/m²)	169.2	175.2	183.1	viol 0%
G-load (g)	2.38	2.5	2.72	viol 0%
Heat load (MJ/m²)	22.2	23.1	24.1	viol 0%
Capture rate				100%

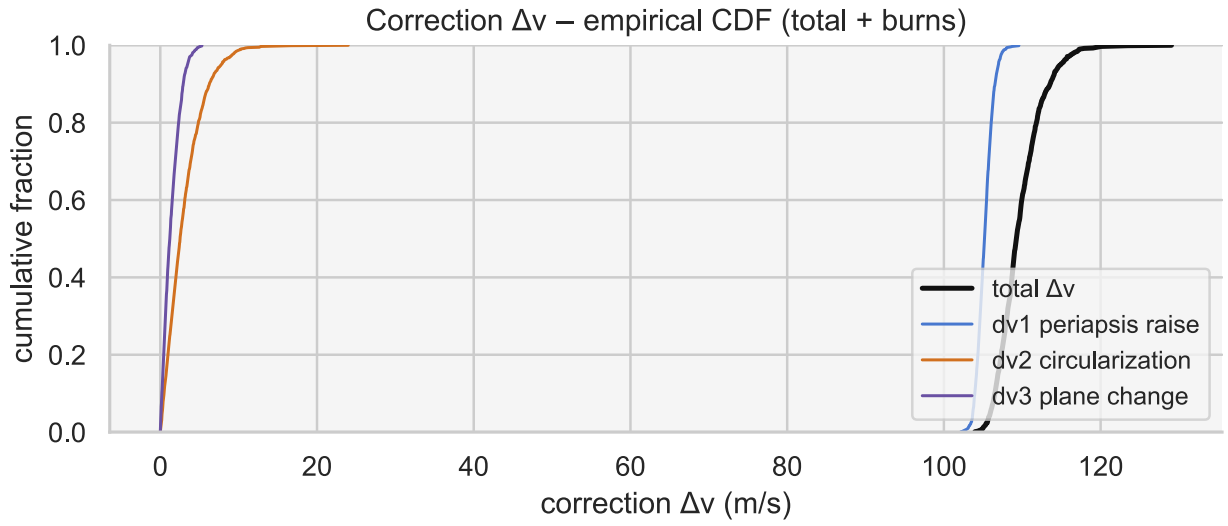
NN -- Dense (515 params)

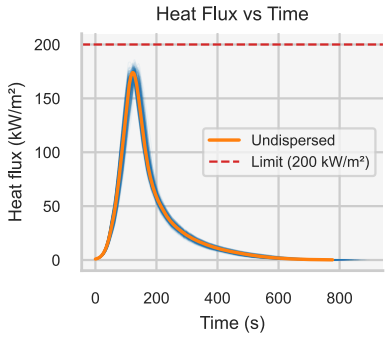


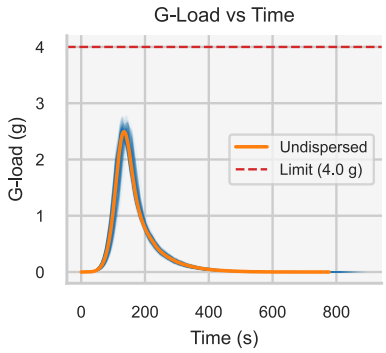


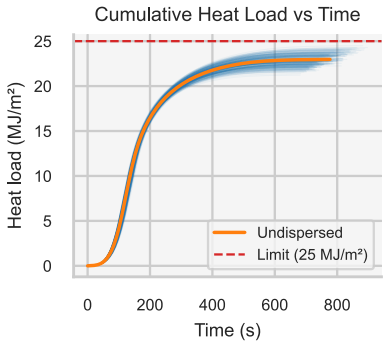


NN -- Dense (515 params) (continued)



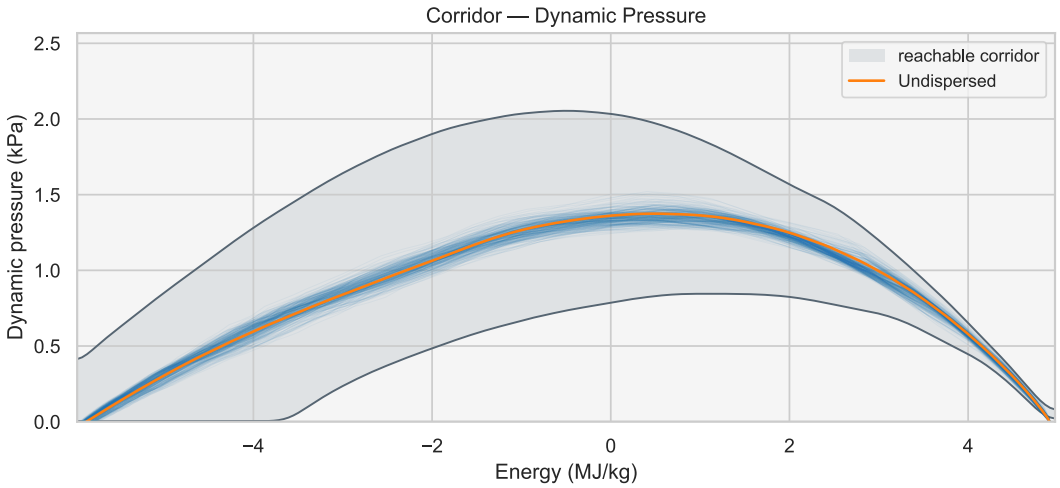


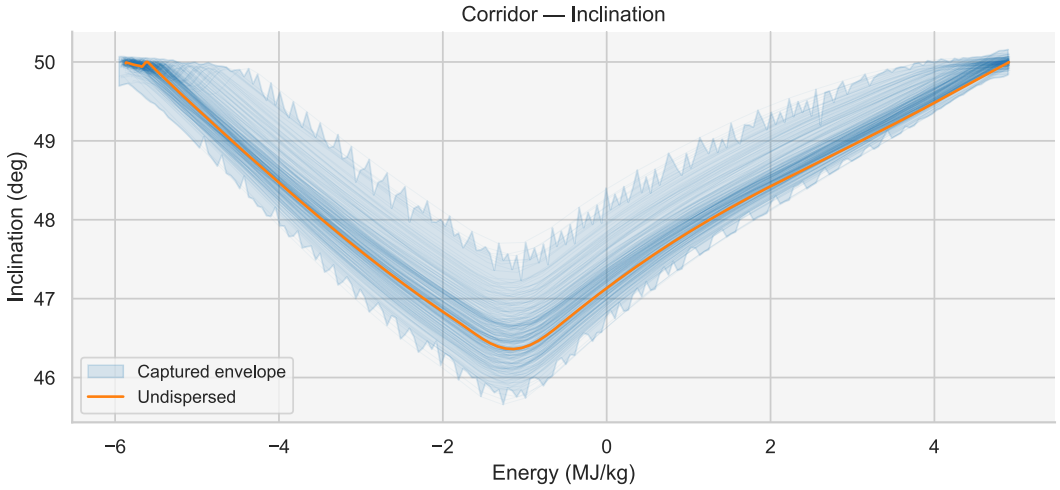


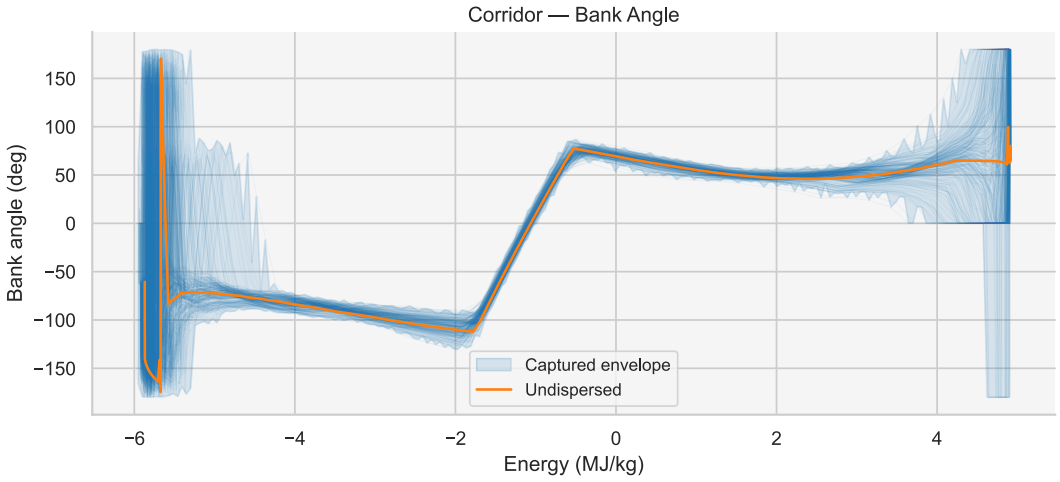


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	109.2	114.9	109.7	mean
Δv CVaR95 / p99 / max	117	117.3	129	tail
dv1 periapsis raise	105.2	107	109.6	m/s
dv2 circularization	2.5	7.8	24	m/s
dv3 plane change	1.2	3.5	5.3	m/s
Apoapsis error (km)	−1.3	26.1	−2	mean
Periapsis error (km)	31.3	36.9	31.1	mean
Inclination error (deg)	0	0.05	0	mean
Heat flux (kW/m²)	174	180.5	187.1	viol 0%
G-load (g)	2.54	2.68	2.81	viol 0%
Heat load (MJ/m²)	22.9	24	24.9	viol 0%
Capture rate				100%

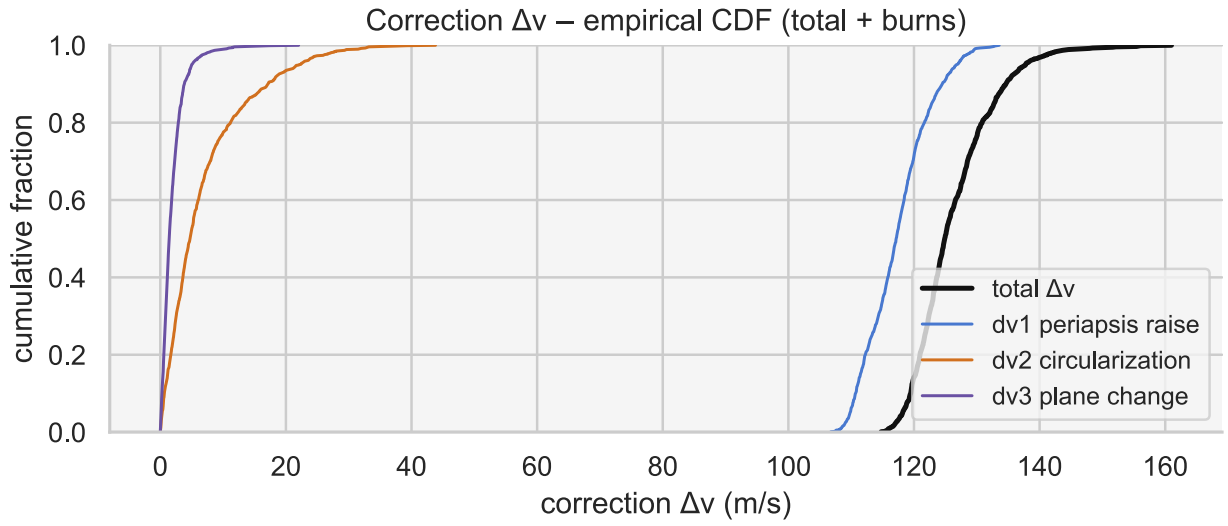
FTC (joint reference)

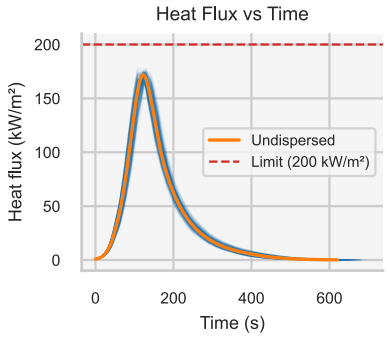


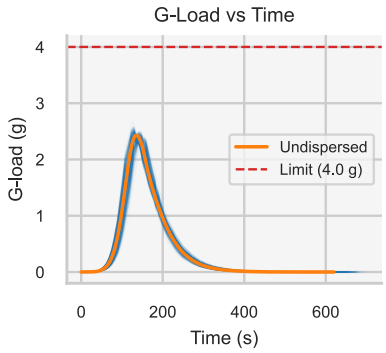


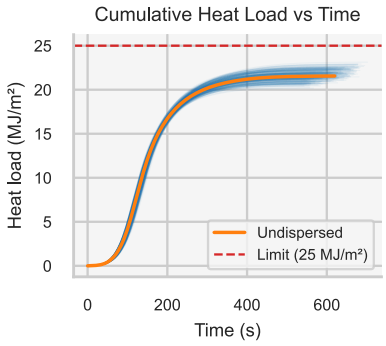


FTC (joint reference) (continued)



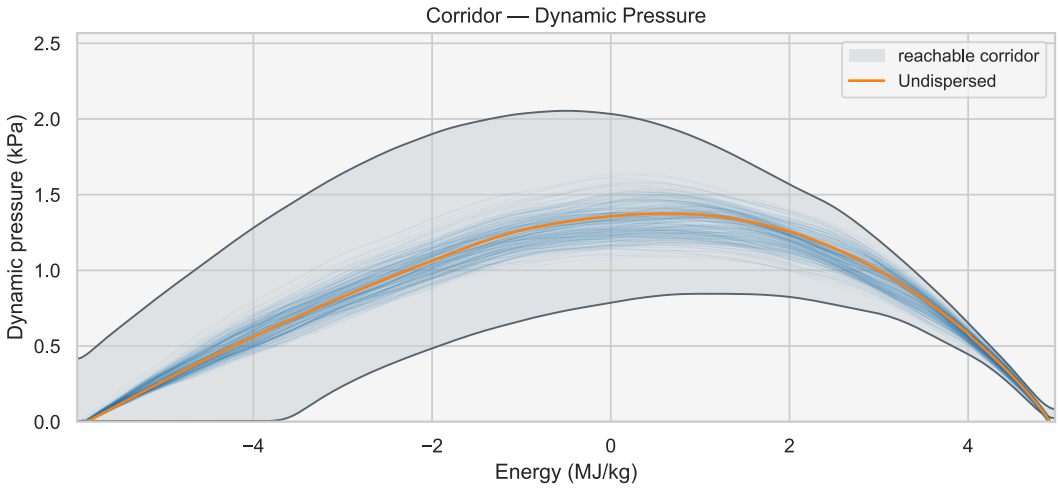


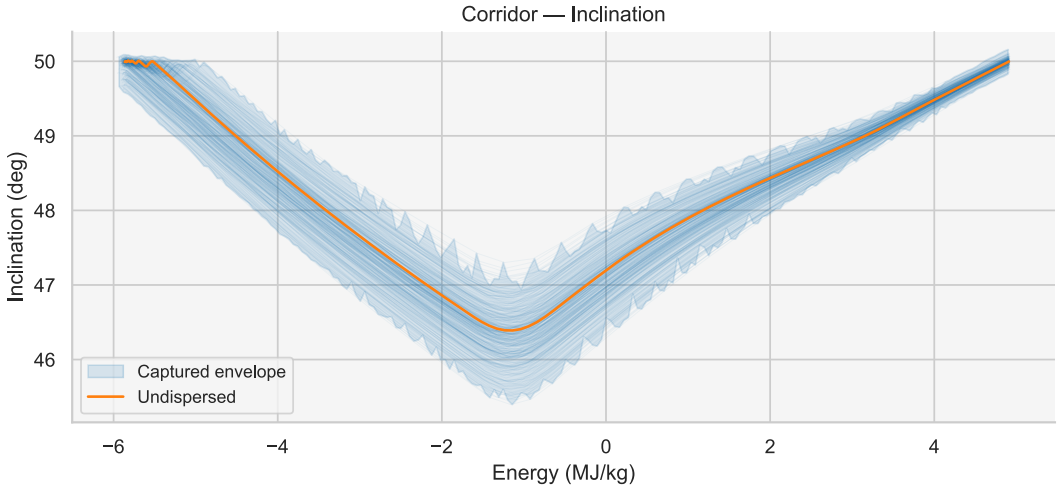


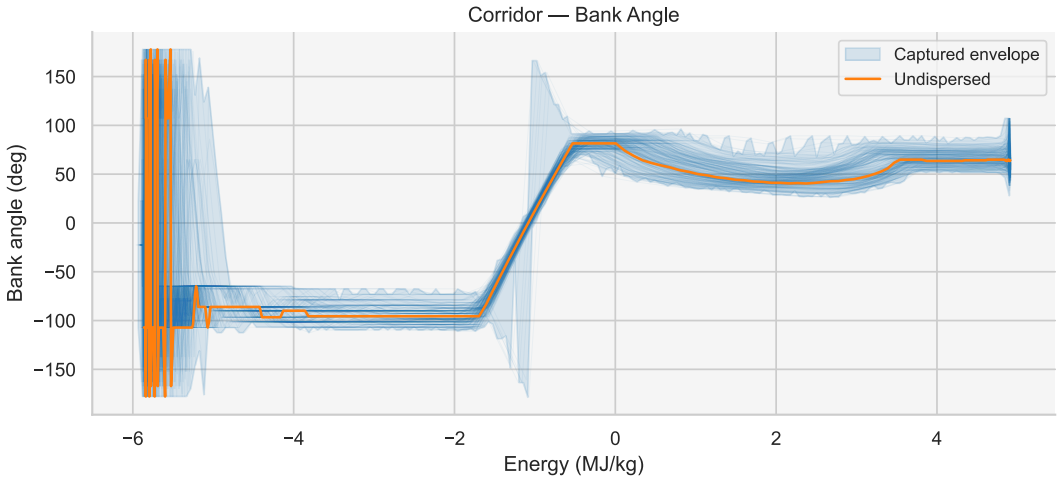


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	124.9	137.8	126.3	mean
Δv CVaR95 / p99 / max	142.9	145.9	161	tail
dv1 periapsis raise	117.2	126.8	133.6	m/s
dv2 circularization	4.8	22.1	43.8	m/s
dv3 plane change	1.4	5	22	m/s
Apoapsis error (km)	3.9	105.6	16.1	mean
Periapsis error (km)	−16.7	9.4	−18	mean
Inclination error (deg)	−0.02	0.04	−0.02	mean
Heat flux (kW/m²)	171.3	176	181.4	viol 0%
G-load (g)	2.43	2.54	2.68	viol 0%
Heat load (MJ/m²)	21.6	22.6	23.4	viol 0%
Capture rate				100%

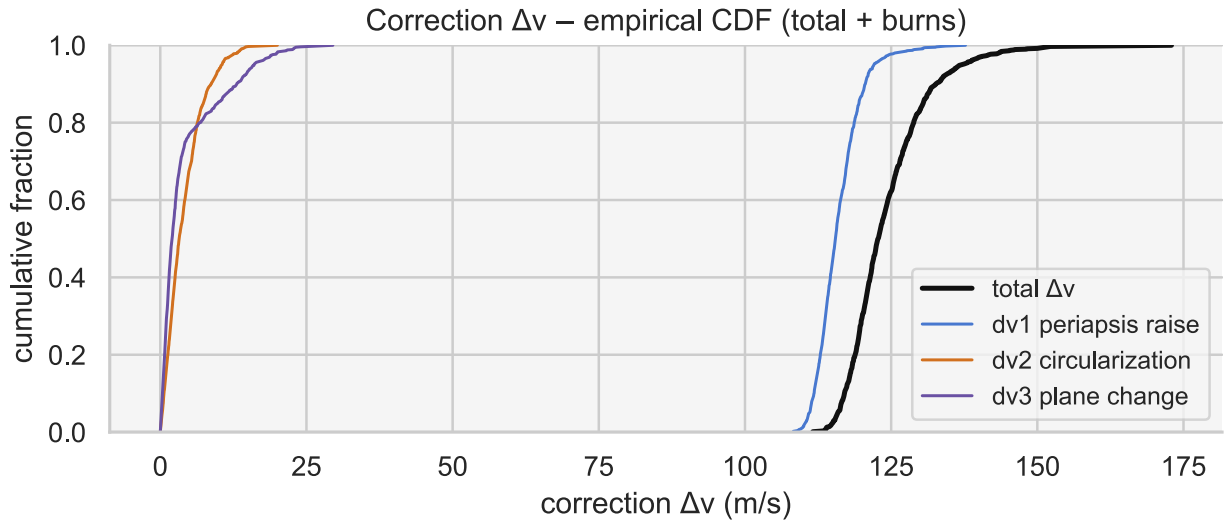
FNPAG

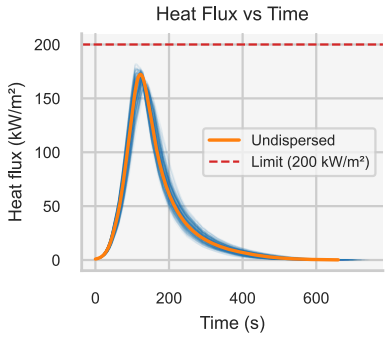

Corridor — Dynamic Pressure
reachable corridor
Undispersed
Dynamic pressure (kPa)
2.5
2.0
1.5
1.0
0.5
0.0
−4
−2
0
2
4
Energy (MJ/kg)

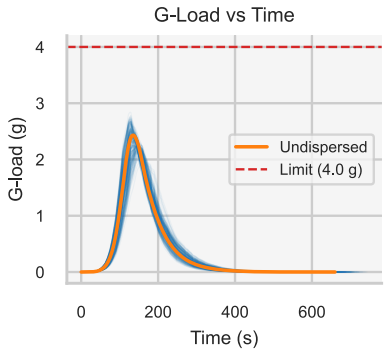

Corridor — Inclination
Captured envelope
Undispersed
Inclination (deg)
50
49
48
47
46
−6
−4
−2
0
2
4
Energy (MJ/kg)

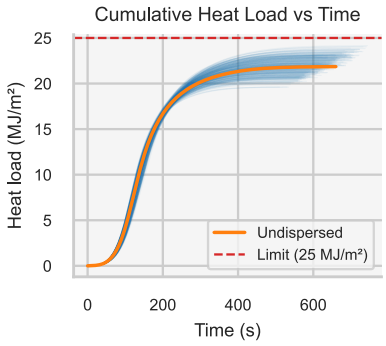

Corridor — Bank Angle
Captured envelope
Undispersed
Bank angle (deg)
150
100
50
0
−50
−100
−150
−6
−4
−2
0
2
4
Energy (MJ/kg)

FNPAG (continued)



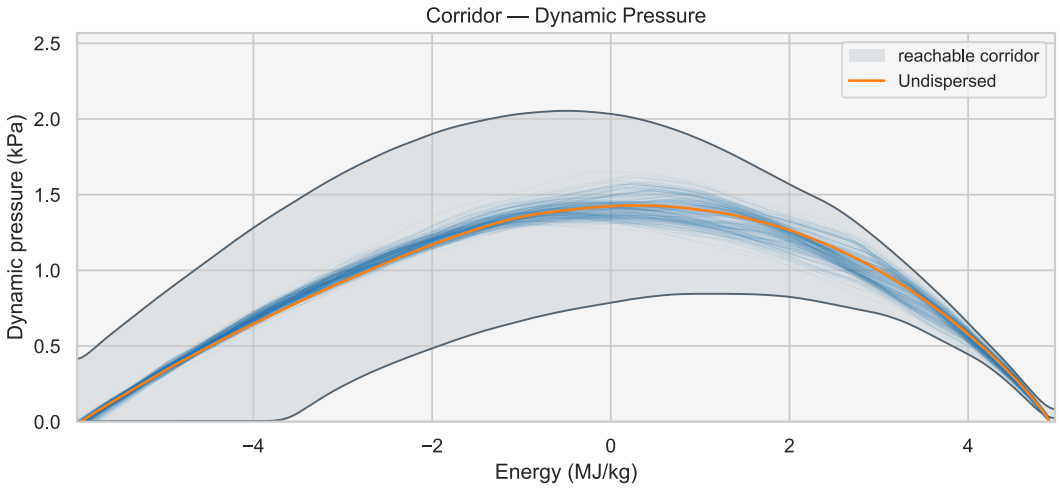


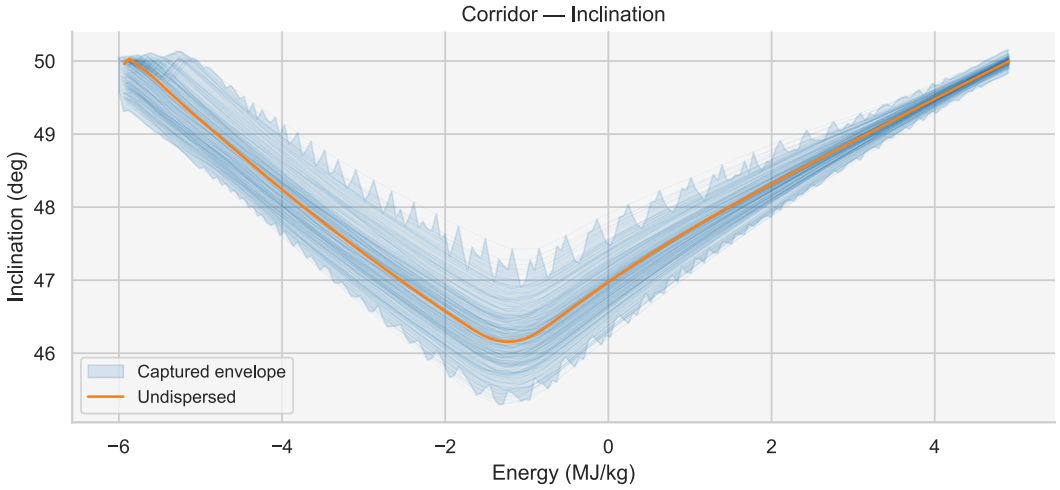


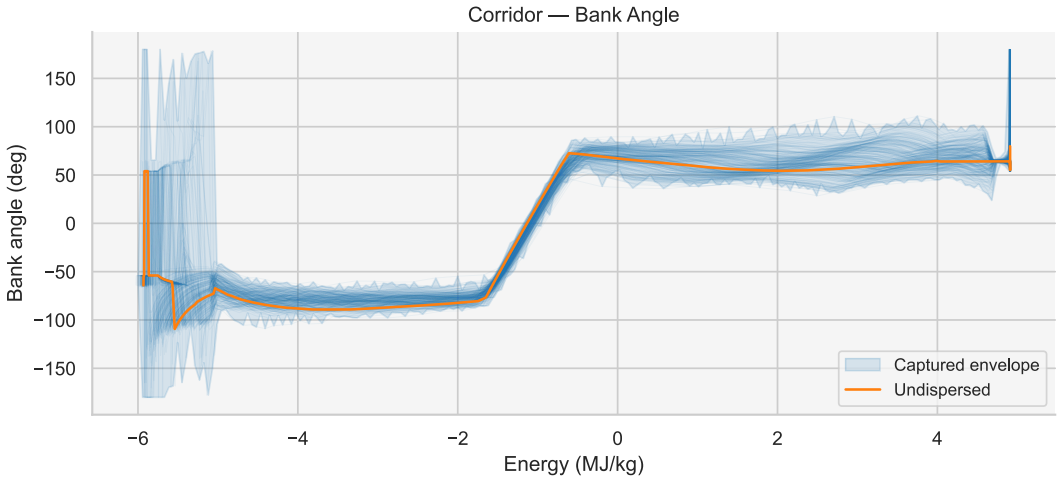


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	122.8	137.4	124.3	mean
Δv CVaR95 / p99 / max	144	148.1	172.9	tail
dv1 periapsis raise	115.5	122.1	137.7	m/s
dv2 circularization	3.3	10.6	20	m/s
dv3 plane change	2	16	29.5	m/s
Apoapsis error (km)	3.8	45.5	4.2	mean
Periapsis error (km)	−10	6.9	−12.1	mean
Inclination error (deg)	−0.03	0.04	−0.06	mean
Heat flux (kW/m²)	170.3	177.6	186.3	viol 0%
G-load (g)	2.38	2.68	2.89	viol 0%
Heat load (MJ/m²)	22	23.5	24.8	viol 0%
Capture rate				100%

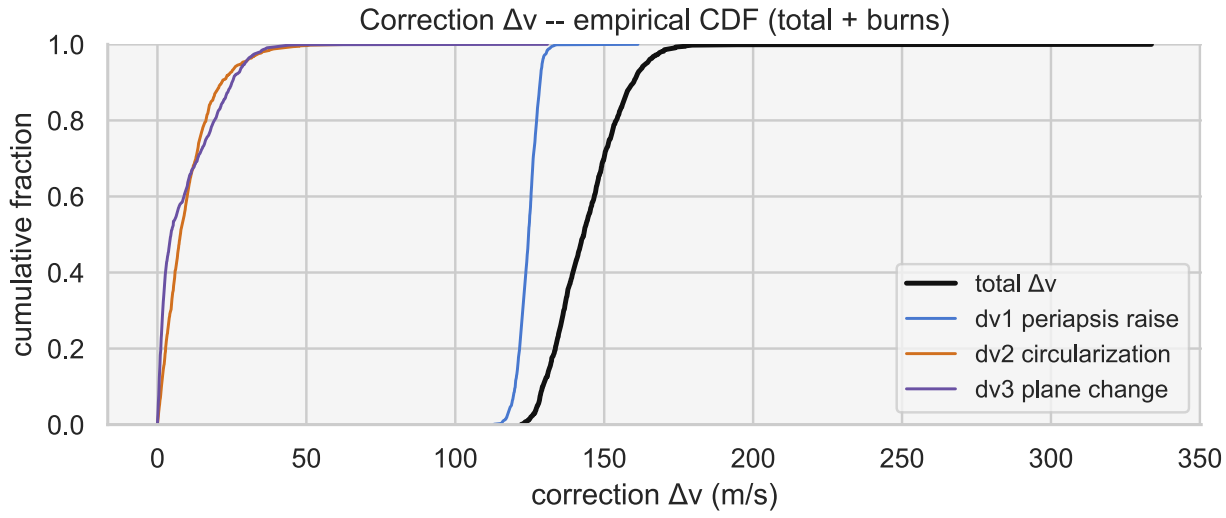
PredGuid (joint reference)

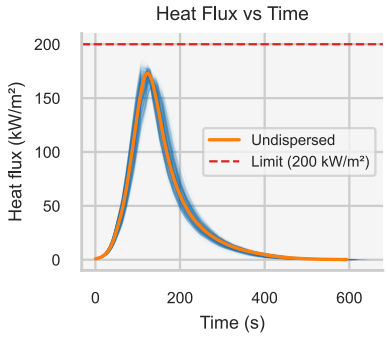


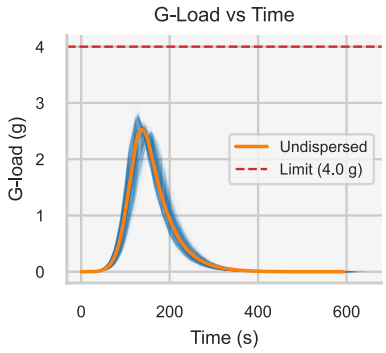


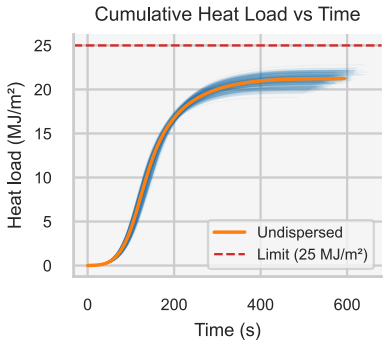


PredGuid (joint reference) (continued)



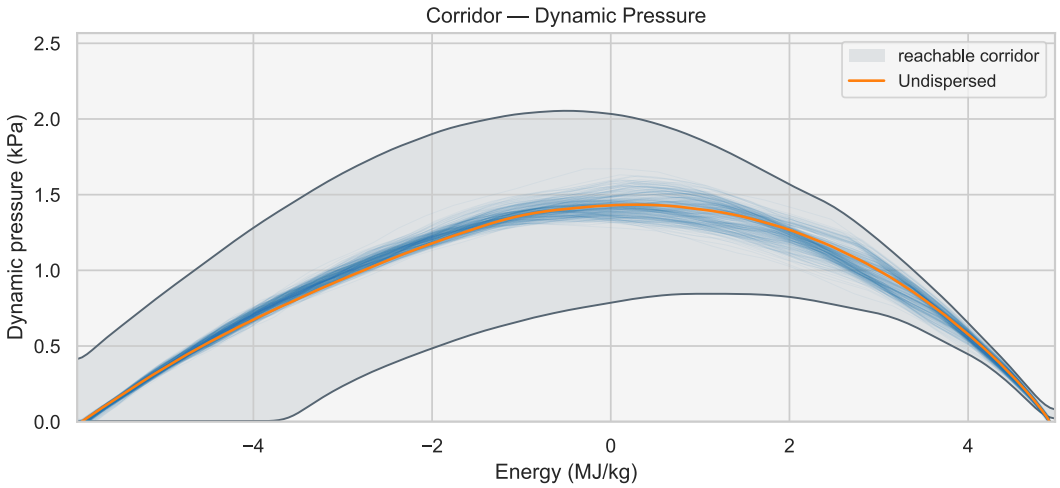


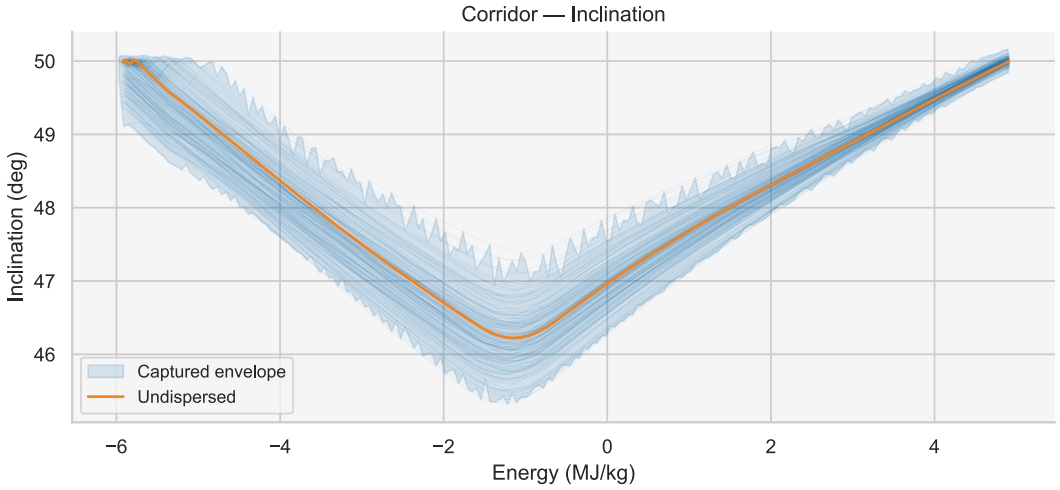


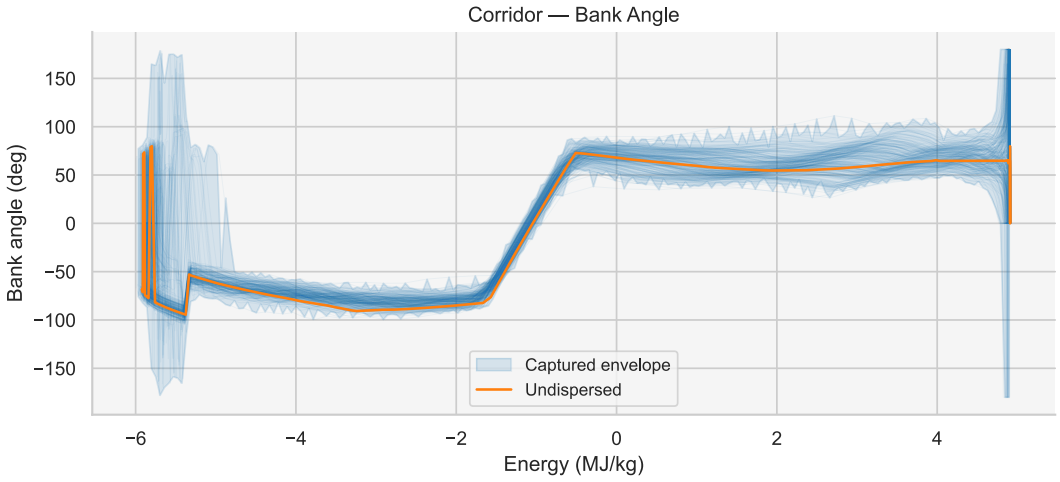


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	143.1	164.2	144.2	mean
Δv CVaR95 / p99 / max	172.8	173.7	333.8	tail
dv1 periapsis raise	124.6	129.2	161.3	m/s
dv2 circularization	7.7	28.4	65.3	m/s
dv3 plane change	4.5	29.3	130.9	m/s
Apoapsis error (km)	2.7	135.8	11.4	mean
Periapsis error (km)	−46.2	−26.9	−45.7	mean
Inclination error (deg)	−0.08	0.03	−0.16	mean
Heat flux (kW/m²)	172	178.9	186.9	viol 0%
G-load (g)	2.53	2.76	2.91	viol 0%
Heat load (MJ/m²)	21.2	22.2	22.9	viol 0%
Capture rate				100%

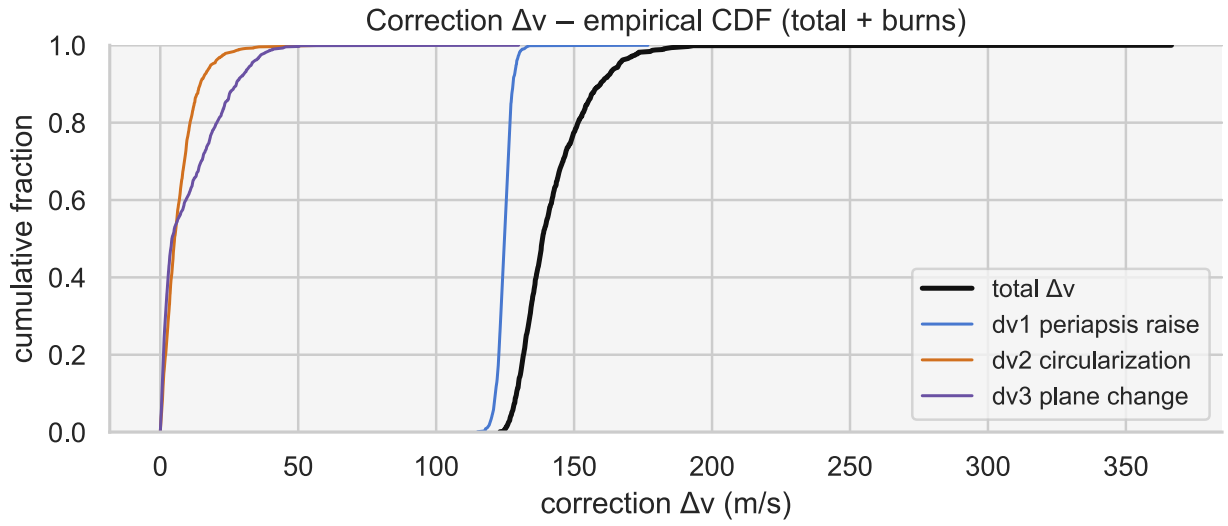
Energy controller (joint reference)

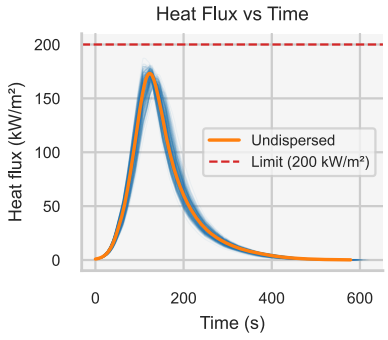


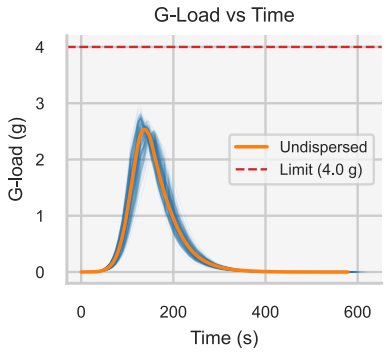


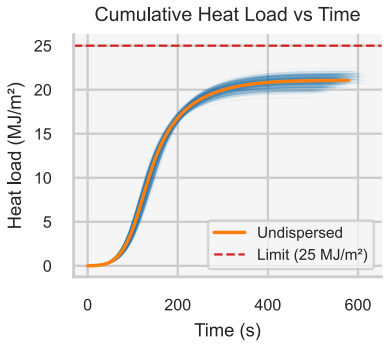


Energy controller (joint reference) (continued)



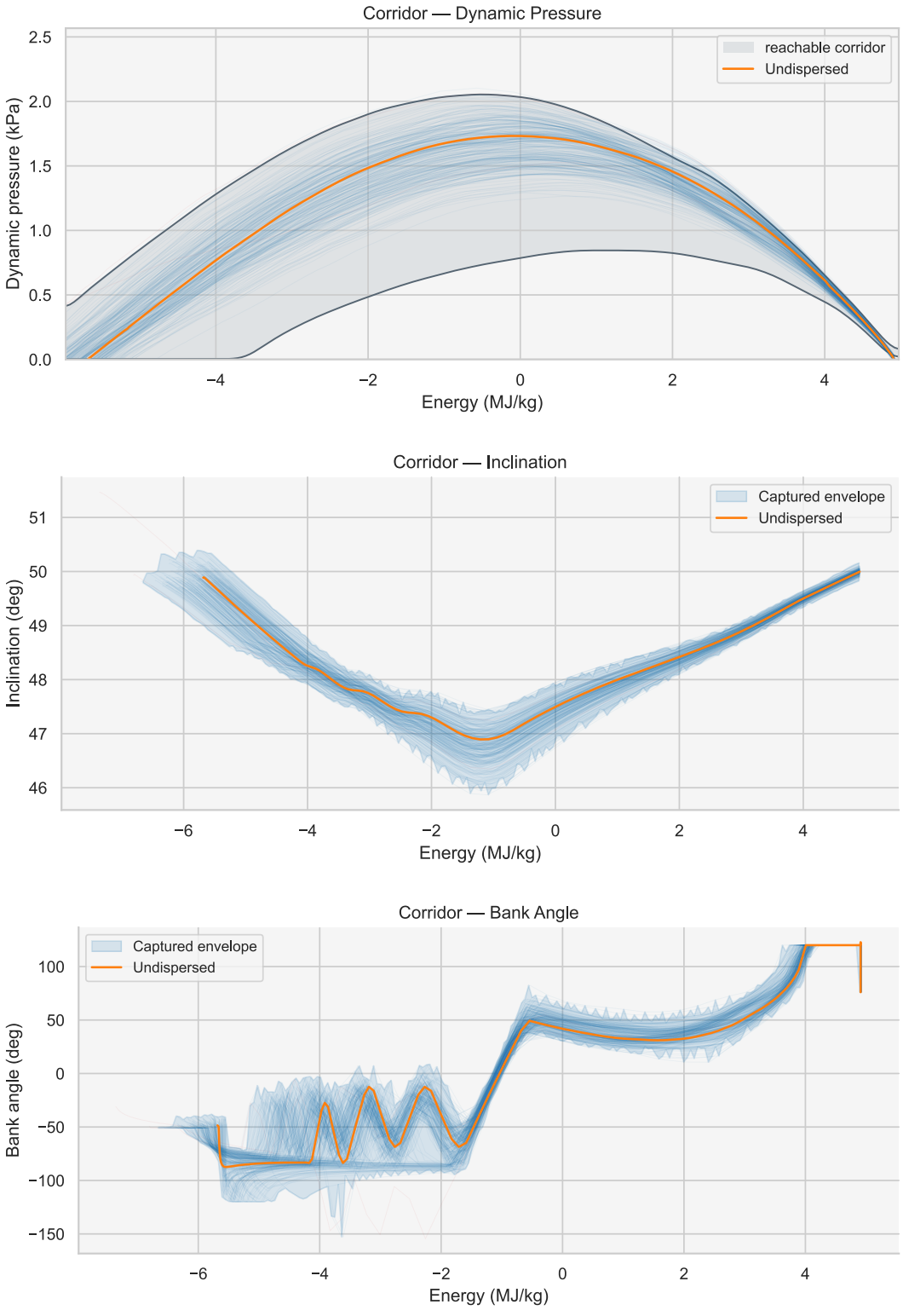




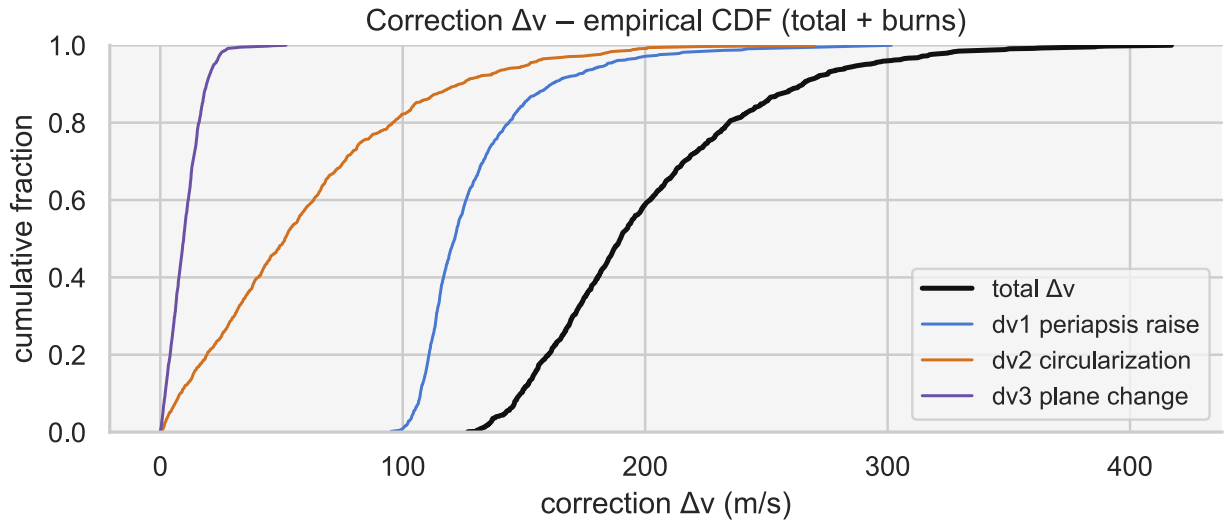


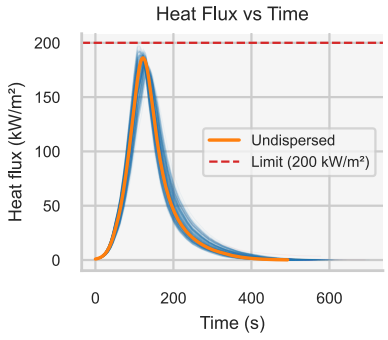
Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	138.6	166.3	142.1	mean
Δv CVaR95 / p99 / max	178.3	182.5	366.5	tail
dv1 periapsis raise	124.8	129.1	176.8	m/s
dv2 circularization	5.2	18.5	59.9	m/s
dv3 plane change	4.2	33.1	129.9	m/s
Apoapsis error (km)	15.9	87.1	19.5	mean
Periapsis error (km)	−46.9	−32.6	−47.3	mean
Inclination error (deg)	−0.07	0.03	−0.18	mean
Heat flux (kW/m²)	172.5	179.7	188.4	viol 0%
G-load (g)	2.55	2.78	2.96	viol 0%
Heat load (MJ/m²)	21	21.9	22.5	viol 0%
Capture rate				100%

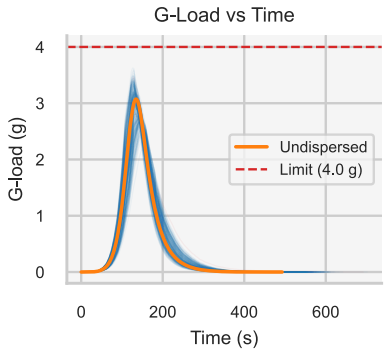
Equilibrium glide

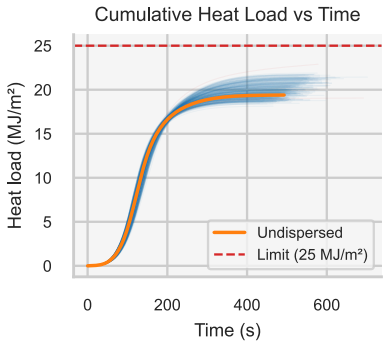


Equilibrium glide (continued)



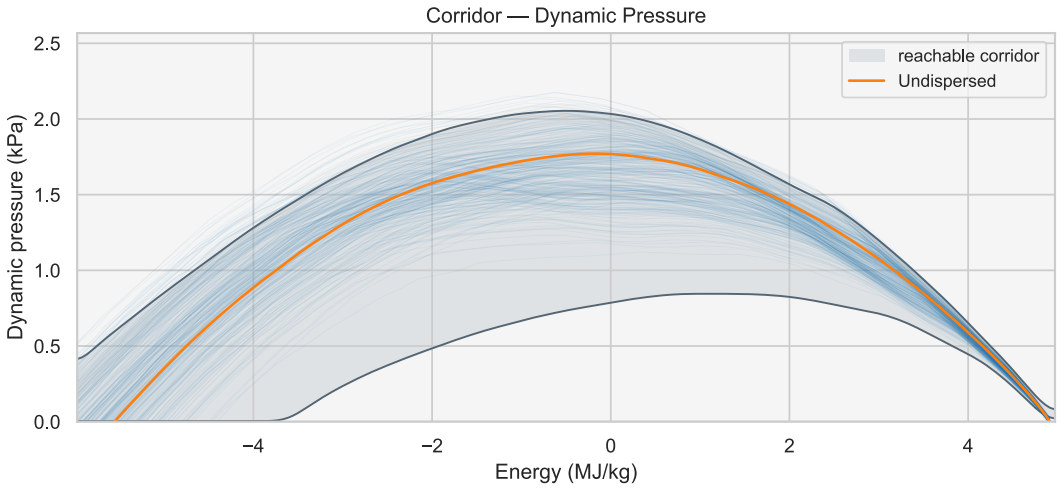


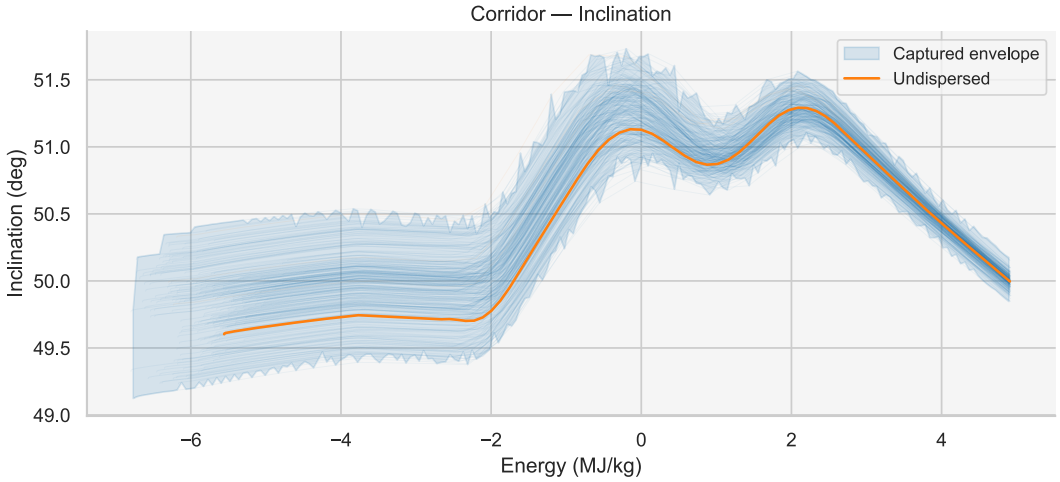


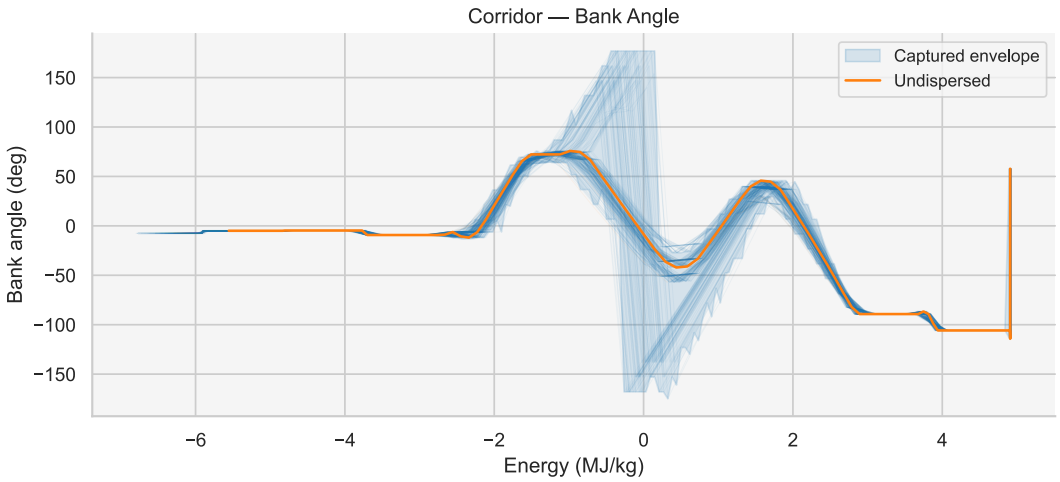


Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	189.9	290	200.3	mean
Δv CVaR95 / p99 / max	327.6	349.2	417.1	tail
dv1 periapsis raise	121.5	184.2	301.5	m/s
dv2 circularization	51.4	152.1	270	m/s
dv3 plane change	9.6	22.1	51.8	m/s
Apoapsis error (km)	174.5	806.2	205.2	mean
Periapsis error (km)	−39.7	9.9	−70.1	mean
Inclination error (deg)	−0.14	0.19	−0.13	mean
Heat flux (kW/m²)	184.3	192.7	202.9	viol 0.5%
G-load (g)	3.01	3.41	3.69	viol 0%
Heat load (MJ/m²)	19.7	21.4	24	viol 0%
Capture rate				99.5%

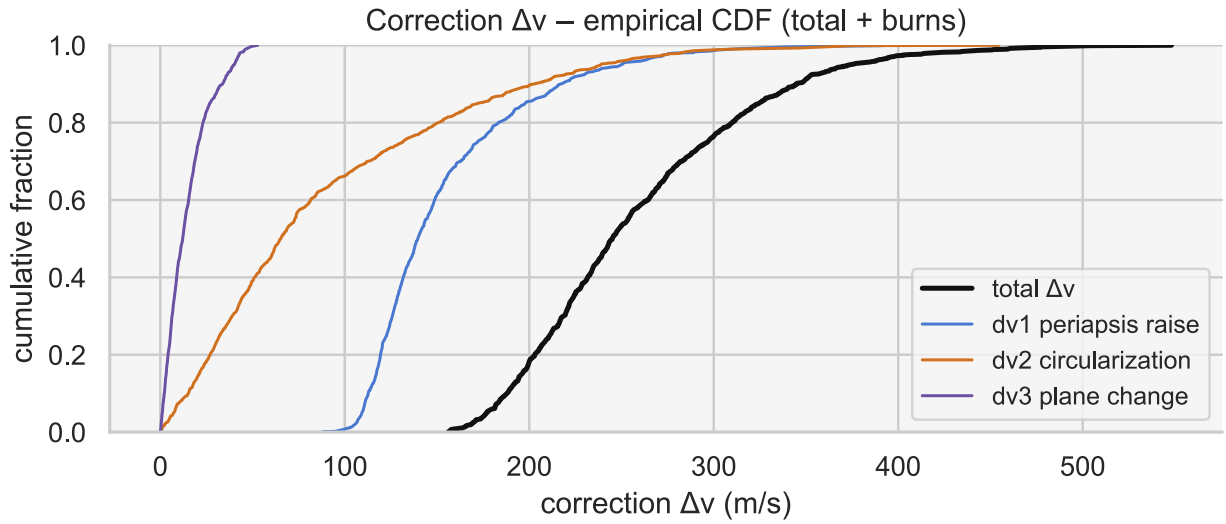
Piecewise constant

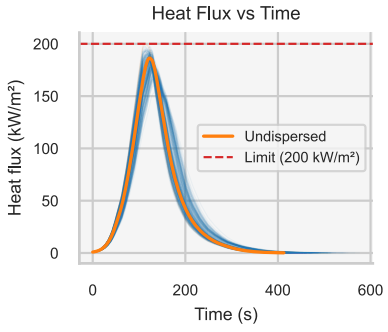


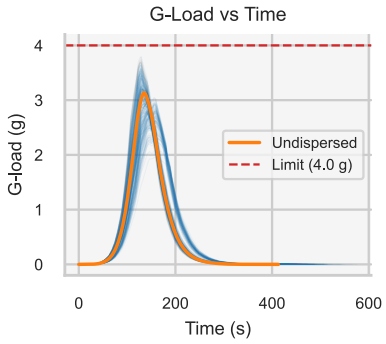


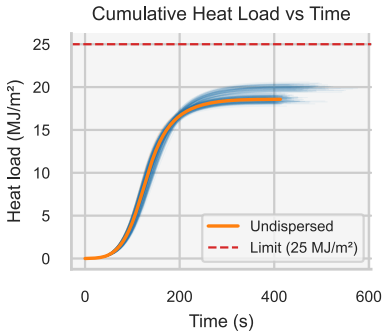


Piecewise constant (continued)









Statistic	p50	p95	mean/max	note
Correction Δv (m/s)	245	374.6	258.3	mean
Δv CVaR95 / p99 / max	421.1	459.3	548.1	tail
dv1 periapsis raise	139.6	249.8	391.5	m/s
dv2 circularization	65.4	238.8	454.5	m/s
dv3 plane change	12.1	40.2	52.9	m/s
Apoapsis error (km)	253.9	1365.6	363.1	mean
Periapsis error (km)	−113.9	−31.3	−166.4	mean
Inclination error (deg)	−0.16	0.22	−0.2	mean
Heat flux (kW/m²)	184.5	194.8	205.6	viol 1.1%
G-load (g)	3.03	3.62	3.94	viol 0%
Heat load (MJ/m²)	18.8	20.2	20.7	viol 0%
Capture rate				99.8%

Appendix E: the shared noise path – a conditioning defect and its repair

The defect

Every per-seed pool in this paper – training, validation, final evaluation, the re-quote pools, and the 10^6-scenario confirmatory pools – is executed as a batch of one-scenario runs, one per Monte Carlo seed. In the historical simulator, the seed drives the 26 static dispersion draws of Table 1 but **not** the stochastic streams: the Ornstein–Uhlenbeck density innovations (and the EKF sensor noise, unused here) are seeded from a separate constant plus the within-batch scenario index, which is zero for every one-scenario batch. The result is that all scenarios in every pool share a single sample path of the time-varying density perturbation. The process is genuinely time-varying within each run – the guidance laws do fight it – but across scenarios it is a constant, not a random variable. We found this while validating a demonstration script whose multi-scenario batches (which do increment the index) refused to reproduce the per-seed numbers; the discrepancy was 19 m/s of median, far outside sampling noise, and the audit traced every per-seed execution path to the same conditioning.

Three properties keep the main body meaningful. First, the conditioning is **identical across schemes**: every classical law and every network trains, validates, and evaluates under the same shared path, so the comparisons are fair as stated. Second, it is identical across the paper's pools, so the pre-registered confirmatory protocol of Section 4.3 is internally consistent. Third, the shared path is not an adversarial or a benign outlier – it is one honest draw of the process. What the conditioning does change is the **marginal** interpretation: a mission flies one fresh realization per entry, and the quoted absolute statistics understate that spread.

Quantification

We score each deployed policy on a paired pool ($n = 1000$ shared scenario seeds) under both regimes: the historical shared path, and a marginal regime in which each scenario receives its own realization. All values are CVaR_{95} of the correction Δv in m/s; capture and the heat-load feasibility of Section 6.2 are noted where they move.

Table 12: Shared-path versus per-scenario CVaR_{95} (m/s, paired $n = 1000$) for the deployed policies. The networks, trained on the shared path, give up 2–4 × more tail than the classical schemes; on the per-scenario tail the deployed FNPAG beats every shared-path-trained network.

Deployed policy	Shared path	Per-scenario	Δ
Mamba 962 (headline)	115.9	194.4 (98.0% capture)	+78.5
LSTM 1082	116.3	170.4 (17% heat-load viol.)	+54.1
GRU 1014	119.3	198.1	+78.8
Dense 972	121.3	194.9	+73.5
Dense 515	126.2	228.0	+101.9
FNPAG	143.3	154.3	+11.0
PredGuid / FTC	221–231	244–261	+22–+31

The asymmetry is the finding: the analytic laws lose 11–31 m/s – ordinary distribution widening – while the networks lose 54–102 and shed capture or feasibility. A policy with internal state can fit the one density history it ever sees, and did. On the per-scenario tail the deployed FNPAG (154.3 m/s, clean constraints) beats every shared-path-trained network, inverting the Section 7 margin.

The repair, and retraining under it

The repair is a seeding mode (`noise_seeding = "per_draw"`) that derives each scenario's stream seed from a hash of its own dispersion draw: identical draw, identical realization – runs stay reproducible – but distinct scenarios receive distinct paths, so per-seed pools marginalize. The historical mode remains the default; every main-body number is reproducible as published.

We then retrained the five headline cells from scratch under per-scenario noise for the deployed generation count (20,000 generations, three trainer-seed repeats each) at the sweep configuration's allocation – a population of 60 with ten scenarios per individual, not the headline's 512×2 – and separately fine-tuned each shared-path champion for 2000 further generations under the repaired seeding (the resume keeps the best 60 of the champion's 512 individuals). Because the optimizer study showed the genetic algorithm to be population-sensitive, the scratch rows of Table 13 test the repaired seeding at a smaller population than the headline, and part of their compressed architecture separation may be allocation rather than noise; the fine-tunes and the far-tail confirmatory start from the headline-allocation champions. Fourteen of the fifteen scratch repeats (the fifteenth, a dense-515 seed, loses one scenario in 1000) and three of the five fine-tunes restore 100% capture with zero constraint violations on the per-scenario pool; notably, scratch retraining fixes the LSTM cell whose shared-path champion had been heat-load infeasible.

Table 13: Per-scenario CVaR_{95} (m/s, $n = 1000$) after retraining under the repaired seeding. Scratch means are over three trainer-seed repeats (20,000 generations, population 60, ten scenarios per individual); fine-tunes are single runs of the shared-path champion continued 2000 generations under per-scenario noise at population 60.

Cell	Scratch, 3 seeds (mean $\pm\sigma_{\text{run}}$)	Fine-tune	Note
Mamba 962	148.1 ± 1.6	138.6	fine-tune replicates its pilot (138.3)
GRU 1014	150.2 ± 5.7	153.4	fine-tune **regresses**
Dense 972	150.4 ± 7.1	145.2	
LSTM 1082	153.8 ± 12.7	128.3	fine-tune inherits 10.8% heat-load viol.
Dense 515	158.7 ± 3.8	129.8	best CVaR_{95}; far tail says otherwise, see Table 14

Three conclusions. First, training on the right distribution repairs the damage: the scratch retrains beat the shared-path-trained networks by 17–69 m/s on the marginal tail and edge FNPAG's 154.3 by roughly one σ_{run} – a real but modest margin. The decisive margin comes from the fine-tune recipe: continuing a shared-path champion briefly under per-scenario noise yields the two best feasible policies of the study (129.8 and 138.6 m/s, 16–25 below FNPAG), though the recipe is not universal – the GRU regresses and the LSTM fine-tune inherits its parent's infeasibility, so it must be validated per cell. Second, the inter-architecture ordering of Section 6 compresses: the Mamba, GRU, and dense-972 scratch means sit within 3 m/s of one another, inside σ_{run}, and only the dense-515 cell is significantly worse. What survives for the recurrent cell at three repeats is **consistency** – ± 1.6 m/s against ± 5.7–12.7 – which is suggestive, not conclusive. Third, the scope of the main-body claims: the methodology results of Sections 4–5 (seed strategy, cost transform, optimizer ordering) compare like against like under identical conditioning and are unaffected in kind; the absolute Δv magnitudes and the Section 6 architecture margins are shared-path quantities and should be read with this appendix.

The far-tail confirmatory under per-scenario noise

The CVaR_{95} statistics above are too shallow for the sizing metric, so we re-ran the confirmatory protocol of Section 4.3 – the same ten pre-registered pools of 100,000 scenarios – under per-scenario noise, for the two fine-tuned champions, the shared-path headline champion, and FNPAG.

Table 14: Far-tail confirmatory under per-scenario noise: $10 \times 100,000$ scenarios per policy, the pre-registered pools of Section 4.3. Δv statistics in m/s over captured scenarios; standard errors over the ten replicates. FNPAG's 0.63% non-captures are physical crashes: a 500-seed sample re-runs as crashes with no timeout censoring.

Policy	Capture	CVaR_{95}	$\mathrm{CVaR}_{99.9}$ ($\pm$ s.e.)	Max
Mamba 962 fine-tune	99.9995%	138.7	$\mathbf{163.0} \pm 0.3$	249
Shared-path Mamba champion	97.93%	188.2	221.3 ± 0.5	270
Dense 515 fine-tune	100.00%	**128.8**	236.3 ± 2.5	405
FNPAG	99.37%	152.5	236.7 ± 2.3	579

The million-scenario depth reverses the shallow-tail verdict and restores the Section 6 thesis on the marginal distribution. The dense fine-tune, best at CVaR_{95}, pays for it at depth: its far tail reaches 236 m/s with a worst case of 405. The fine-tuned recurrent policy holds $\mathrm{CVaR}_{99.9} = 163.0 \pm 0.3$ m/s – more than 73 m/s below both the dense fine-tune and FNPAG – loses 5 of 10^6 scenarios, and posts the smallest worst case of the study (249 m/s). As could be expected from the shared-path results, the shared-path-trained champion is confirmed broken at this depth (97.9% capture); what could not be seen at $n = 1000$ is that the recurrent advantage survives the honest regime precisely where the paper always located it: the extreme tail that sizes the tanks.

The far-tail claim is seed-robust. Two further fine-tunes from the same shared-path checkpoint under independent trainer seeds land at $\mathrm{CVaR}_{99.9} = 164.6$ and 161.9 m/s – a three-seed mean of 163.2 ± 1.3, an order of magnitude below the 73 m/s margin. The three seeds lose 5, 30 and 80 of the 10^6 scenarios (5×10^{-6} to 8×10^{-5}; all genuine crashes, none a timeout), so the recipe's capture guarantee is seed-dependent at the 10^{-4} level and a deployed policy must be confirmatory-screened, exactly as the shared-path protocol always required. (A reproducibility note: a checkpoint resume restores the saved trainer RNG state, which silently overrides the seed flag – our first "repeats" were bit-identical replays; the campaign runner now strips the saved RNG state for repeat runs.) The remaining limit: the fine-tune recipe still requires per-cell feasibility validation (Table 13).